

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»  
Нижегородский филиал

Факультет информатики, математики и компьютерных наук

Паршин Даниил Денисович

**ОПТИМИЗАЦИЯ КАЧЕСТВА ОБЛАЧНОГО ВИДЕОСТРИМИНГА В  
УСЛОВИЯХ ОГРАНИЧЕННОСТИ ВЫЧИСЛИТЕЛЬНЫХ РЕСУРСОВ С  
ПРИМЕНЕНИЕМ КЛАССИЧЕСКИХ ЭВРИСТИЧЕСКИХ МЕТОДОВ И  
ПОДХОДОВ НА ОСНОВЕ МАШИННОГО ОБУЧЕНИЯ**

Выпускная квалификационная работа - бакалаврская работа  
по направлению подготовки 01.03.02 Прикладная математика и информатика  
образовательная программа «Прикладная математика и информатика»

Рецензент:  
приглашенный преподаватель  
С. М. Досов

Руководитель:  
д-р наук, проф.  
П. А. Колданов

Консультант:  
В. Г. Жислина

Нижний Новгород, 2026

## Аннотация

В выпускной квалификационной работе рассматривается задача повышения воспринимаемого качества облачного видеостриминга при ограниченных вычислительных ресурсах. Предлагаемый подход основан на выделении областей интереса в кадре и передаче информации о них кодировщику в качестве управляющего сигнала для пространственного перераспределения качества. В отличие от классических подходов, ориентированных преимущественно на выбор битрейта во времени, работа рассматривает качество внутри кадра как неоднородное: текст, лица и визуально значимые фактурные области имеют разную пользовательскую ценность.

Центральным инженерным результатом является библиотека IDet — CPU-ориентированный C++ слой для детекции и нормализации ROI. В архитектуре IDet рассматриваются три режима анализа: текстовые области, лица и текстурированные области одежды. Для текстовых областей используются идеи семейства DBNet/DBNet++, для лиц — семейства SCRFD, для текстурированных областей подготовлен отдельный YOLO-детектор на согласованной таксономии DeepFashion2 и Fashionpedia.

Работа фокусируется не только на точности нейросетевой модели, но и на системной пригодности решения: задержке обработки кадра, предсказуемости p95/p99, управлении буферами, политике потоков, переносимости ONNX-артефактов и возможности интеграции с видеокодировщиком. Фактический публичный API IDet возвращает геометрический результат в формате VecQuad; расширенная запись с типом ROI, приоритетом и qoffset рассматривается как контракт последующего слоя интеграции с кодировщиком, а не как уже существующий публичный класс библиотеки.

Дополнительно описан воспроизводимый конвейер обучения YOLO, обеспечивающий подготовку данных, аудит датасетов, обучение, сравнение моделей и экспорт PT-модели в ONNX-артефакт для последующего CPU-инференса через ONNX Runtime. Экспериментальная часть подтверждает качество детектора текстурированных ROI и показывает, что выбор модели должен определяться не только mAP, но и вычислительной стоимостью, размером модели и стабильностью задержки.

Финальная проверка выполнена как покадровое сравнение baseline и ROI-aware HEVC-кодирования через Ffmpeg addroi и libx265. При сопоставимом размере выходного потока ROI-aware ветка показала устойчивый локальный выигрыш в областях интереса для текстовых областей, лиц и текстурированных областей одежды. При этом снижение части global/background метрик интерпретируется как ожидаемый компромисс фиксированного битового бюджета: качество перераспределяется от фона к ROI, а не равномерно повышается по всему кадру.

**Ключевые слова:** облачный видеостриминг; ROI-кодирование; видеокодирование; IDet; CPU-инференс; ONNX Runtime; DBNet; SCRFD; YOLO; DeepFashion2; Fashionpedia; текстурированные ROI; VMAF; SSIM.

# Содержание

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>Аннотация</b>                                                           | <b>2</b>  |
| <b>Словарь терминов и сокращений</b>                                       | <b>6</b>  |
| <b>Введение</b>                                                            | <b>13</b> |
| <b>1 Предметная область и постановка задачи</b>                            | <b>15</b> |
| 1.1 Облачный видеостриминг как задача распределения ресурсов . . . . .     | 15        |
| 1.2 Почему глобальные метрики недостаточны . . . . .                       | 15        |
| 1.3 ROI-aware кодирование в существующих исследованиях . . . . .           | 16        |
| 1.4 Три режима ROI в предлагаемой системе . . . . .                        | 16        |
| 1.5 Выводы по главе . . . . .                                              | 17        |
| <b>2 Формальная постановка задачи ROI-aware оптимизации</b>                | <b>18</b> |
| 2.1 Ограниченная оптимизация качества . . . . .                            | 18        |
| 2.2 Бюджет площади ROI . . . . .                                           | 18        |
| 2.3 Временная стабильность ROI . . . . .                                   | 19        |
| 2.4 От ROI к qoffset кодировщика . . . . .                                 | 19        |
| 2.5 Метрики детекции . . . . .                                             | 20        |
| 2.6 Выводы по главе . . . . .                                              | 20        |
| <b>3 Проектирование IDet и трехрежимного ROI-конвейера</b>                 | <b>21</b> |
| 3.1 IDet как инженерное ядро работы . . . . .                              | 21        |
| 3.2 Жизненный цикл обработки кадра . . . . .                               | 23        |
| 3.3 Ответственность адаптеров детекторов . . . . .                         | 23        |
| 3.4 Публичный контракт IDet . . . . .                                      | 24        |
| 3.5 Последующий контракт метаданных ROI . . . . .                          | 27        |
| 3.6 Приоритеты и разрешение конфликтов . . . . .                           | 27        |
| 3.7 Память, буферы и критический путь без динамических аллокаций . . . . . | 27        |
| 3.8 Политика потоков . . . . .                                             | 28        |
| 3.9 Инварианты и отказоустойчивость . . . . .                              | 28        |
| 3.10 Выводы по главе . . . . .                                             | 29        |
| <b>4 Детектор текстурированных ROI: данные, обучение и экспорт</b>         | <b>30</b> |
| 4.1 Мотивация детекции текстурированных ROI . . . . .                      | 30        |
| 4.2 Конвейер обучения YOLO . . . . .                                       | 30        |
| 4.3 Исходные датасеты и единая таксономия . . . . .                        | 31        |

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| 4.4      | Аудит датасетов после конвертации . . . . .                                 | 32        |
| 4.5      | Дисбаланс классов . . . . .                                                 | 33        |
| 4.6      | Геометрия изображений и bbox . . . . .                                      | 34        |
| 4.7      | Режимы обучения . . . . .                                                   | 35        |
| 4.8      | Контракт PT-to-ONNX артефакта . . . . .                                     | 35        |
| 4.9      | ONNX Runtime и квантизация . . . . .                                        | 36        |
| 4.10     | Выводы по главе . . . . .                                                   | 37        |
| <b>5</b> | <b>Экспериментальные результаты YOLO-моделей на объединенной таксономии</b> | <b>38</b> |
| 5.1      | Граница экспериментальных выводов . . . . .                                 | 38        |
| 5.2      | Аппаратная конфигурация . . . . .                                           | 38        |
| 5.3      | Итоговые метрики YOLO26n . . . . .                                          | 39        |
| 5.4      | Поклассовая точность детектора . . . . .                                    | 39        |
| 5.5      | Сравнение YOLO26n и YOLO26s . . . . .                                       | 41        |
| 5.6      | Матрица ошибок . . . . .                                                    | 41        |
| 5.7      | Сводка артефактов для развертывания . . . . .                               | 42        |
| 5.8      | Почему выбран YOLO26n . . . . .                                             | 43        |
| 5.9      | Выводы по главе . . . . .                                                   | 43        |
| <b>6</b> | <b>Интеграция ROI в видеокодирование</b>                                    | <b>45</b> |
| 6.1      | ROI как управляющий сигнал . . . . .                                        | 45        |
| 6.2      | Адаптер кодировщика как граница между IDet и кодировщиком . . . . .         | 45        |
| 6.3      | Политика qoffset . . . . .                                                  | 46        |
| 6.4      | Ограничение площади и разрешение конфликтов . . . . .                       | 47        |
| 6.5      | Резервные режимы . . . . .                                                  | 48        |
| 6.6      | Связь ROI-интеграции с дальнейшим экспериментом . . . . .                   | 48        |
| 6.7      | Выводы по главе . . . . .                                                   | 49        |
| <b>7</b> | <b>Покадровое сравнение baseline и ROI-aware кодирования</b>                | <b>50</b> |
| 7.1      | Цель и границы эксперимента . . . . .                                       | 50        |
| 7.2      | Данные и подготовка FHD-наборов . . . . .                                   | 50        |
| 7.3      | Протокол кодирования . . . . .                                              | 51        |
| 7.4      | Выбранные detector-конфигурации . . . . .                                   | 52        |
| 7.5      | Метрики и знак improvement . . . . .                                        | 53        |
| 7.6      | Результаты ROI-aware кодирования . . . . .                                  | 55        |
| 7.7      | Почему эффект ограничен . . . . .                                           | 56        |
| 7.8      | Интерпретация по режимам . . . . .                                          | 57        |
| 7.9      | Выводы по главе . . . . .                                                   | 58        |
| <b>8</b> | <b>Экспериментальное исследование производительности IDet</b>               | <b>59</b> |
| 8.1      | Назначение benchmark-эксперимента . . . . .                                 | 59        |
| 8.2      | Дисциплина измерения задержки . . . . .                                     | 59        |



|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 8.3      | Рабочие точки IDet . . . . .                                         | 60        |
| 8.4      | Компромисс между задержкой и качеством покрытия . . . . .            | 61        |
| 8.5      | Интерпретация по режимам . . . . .                                   | 62        |
| 8.6      | I/O Binding и локальность памяти . . . . .                           | 62        |
| 8.7      | NUMA-аспекты CPU-only исполнения . . . . .                           | 63        |
| 8.8      | Top-Down Microarchitecture Analysis . . . . .                        | 63        |
| 8.9      | Ограничения текущих benchmark-данных . . . . .                       | 65        |
| 8.10     | Выводы по главе . . . . .                                            | 65        |
| <b>9</b> | <b>Производственная интеграция, ограничения и план развития IDet</b> | <b>67</b> |
| 9.1      | Граница производственной интеграции . . . . .                        | 67        |
| 9.2      | Режимы эксплуатации . . . . .                                        | 67        |
| 9.3      | Профили среды выполнения . . . . .                                   | 68        |
| 9.4      | Планировщик трех режимов ROI . . . . .                               | 68        |
| 9.5      | Инварианты production-конвейера . . . . .                            | 69        |
| 9.6      | Ограничения текущей реализации и данных . . . . .                    | 70        |
| 9.7      | Риски ложных ROI . . . . .                                           | 70        |
| 9.8      | Практическая значимость работы . . . . .                             | 71        |
| 9.9      | План дальнейшего развития . . . . .                                  | 71        |
| 9.10     | Выводы по главе . . . . .                                            | 72        |
|          | <b>Заключение</b>                                                    | <b>73</b> |
|          | <b>Список литературы</b>                                             | <b>76</b> |
|          | <b>Приложения</b>                                                    | <b>80</b> |
| A.1      | Расширенная сводка аудита датасетов . . . . .                        | 80        |
| A.2      | Распределение классов по датасетам . . . . .                         | 80        |
| A.3      | Ключевые параметры обучающих запусков . . . . .                      | 81        |
| A.4      | Сводка результатов обучения YOLO . . . . .                           | 81        |
| A.5      | Матрицы ошибок и метрики обучения YOLO-экспериментов . . . . .       | 82        |
| A.6      | Визуализация labels для датасетных конфигураций . . . . .            | 87        |
| B.1      | Команда чистого измерения задержки . . . . .                         | 89        |
| B.2      | Отдельный запуск для ROI-геометрии . . . . .                         | 89        |
| B.3      | Алгоритмический протокол измерений . . . . .                         | 90        |
| C.1      | Чек-лист PT-to-ONNX артефакта . . . . .                              | 91        |
| D.1      | Визуальное сравнение single-shot и tiled-режимов . . . . .           | 92        |

## Словарь терминов и сокращений

В данном разделе приведены основные термины, обозначения и сокращения, используемые в работе. Терминология относится к трем связанным областям: видеокодированию, детекции областей интереса и CPU-ориентированному инференсу нейросетевых моделей. Введение словаря необходимо для единообразной интерпретации экспериментальных результатов, архитектурных решений и ограничений предлагаемого ROI-aware конвейера.

### **ROI, область интереса**

Фрагмент кадра, которому в рамках рассматриваемой системы присваивается повышенная зрительная или прикладная значимость. В работе к таким областям относятся текст, лица и визуально насыщенные фактурные области одежды. Выделение ROI используется как основание для пространственного перераспределения качества при видеокодировании.

### **ROI-aware кодирование**

Подход к видеокодированию, при котором кодировщик получает дополнительную информацию о пространственном расположении областей интереса. Эта информация используется для более мягкого квантования значимых областей и более агрессивного сжатия менее важных фрагментов кадра при сохранении общего битового бюджета.

### **ABR, Adaptive Bitrate**

Механизм адаптивного выбора битрейта во времени. ABR-алгоритмы выбирают представление видеосегмента с учетом состояния сети, буфера воспроизведения и целевой функции качества, однако сами по себе не задают распределение качества внутри отдельного кадра.

### **BOLA, Pensieve**

Представители класса ABR-подходов. BOLA использует буферно-ориентированную модель полезности качества, а Pensieve применяет методы обучения с подкреплением для выбора битрейта видеосегментов.

### **Битрейт**

Объем данных, выделяемый на кодирование видео за единицу времени. При фиксированном битрейте ROI-aware кодирование не увеличивает общий бюджет, а перераспределяет его: области интереса получают большую долю битов, тогда как для фоновых областей допускается более сильное сжатие.

### **Кодек, кодировщик, транскодирование**

Кодек определяет формат и принципы сжатия видеоданных, например H.264/AVC или H.265/HEVC. Кодировщик является программной или аппаратной реализацией соответствующего метода сжатия. Транскодирование означает преобразование видеопотока в другой профиль качества, битрейт, разрешение или формат.

**IDet** CPU-ориентированная C++ библиотека для детекции и нормализации областей интереса. В рамках работы IDet рассматривается как инженерный слой между моделями компьютерного зрения и видеокодировщиком. Публичный выход библиотеки представлен геометрией найденных областей.

### **Детектор**

Модель или программный модуль, предназначенный для локализации объектов или областей определенного типа на изображении. В работе используются специализированные детекторы текста, лиц и текстурированных областей одежды.

### **Text ROI, Face ROI, Textured ROI**

Три типа областей интереса, рассматриваемые в предлагаемой системе. Text ROI соответствует текстовым областям, Face ROI — областям лиц, Textured ROI — фрагментам изображения с выраженной визуальной структурой, приближенно представленным через области одежды. Последний тип не является задачей модной классификации, а используется как практический способ локализации высокочастотных деталей, чувствительных к сжатию.

**BBox** Ограничивающий прямоугольник объекта на изображении. В YOLO-разметке bbox задает положение и размер объекта относительно изображения и используется как базовая геометрическая форма для оценки локализации.

**IoU** Intersection over Union — отношение площади пересечения двух областей к площади их объединения. Метрика применяется для оценки степени совпадения предсказанной области с эталонной разметкой.

### **Quad, VecQuad**

Quad — четырехугольник, заданный четырьмя точками. VecQuad — список таких четырехугольников, используемый как публичный геометрический формат выхода IDet.

### **YOLO**

Семейство нейросетевых моделей для детекции объектов. В данной работе YOLO используется для локализации областей одежды, интерпретируемых как textured ROI.

### **DBNet/DBNet++**

Семейство моделей для детекции текста на изображениях. Модели этого класса формируют карту вероятности текстовых областей и затем преобразуют ее в геометрические контуры или многоугольники.

### **SCRFD**

Семейство быстрых детекторов лиц, ориентированных на компромисс между точностью локализации и вычислительной стоимостью. В работе SCRFD рассматривается как один из практических вариантов для CPU-first сценариев.

### **ONNX**

Формат представления нейросетевых моделей, предназначенный для переноса моде-

ли между обучающим фреймворком и средой выполнения. В работе обученная PT-модель экспортируется в ONNX для последующего инференса через ONNX Runtime.

### **PT-модель, checkpoint**

Файл весов модели, сохраненный после обучения в PyTorch/Ultralytics. Такой файл удобен для продолжения обучения и анализа результатов, однако для развертывания в IDet требуется переносимый ONNX-артефакт.

### **Артефакт развертывания**

Проверенный файл модели и сопутствующие метаданные, пригодные для использования в среде выполнения. К существенным характеристикам такого артефакта относятся форма входного тензора, порядок выходов, список классов, версия opset и совместимость с ONNX Runtime.

### **ONNX Runtime**

Среда выполнения ONNX-моделей. В IDet ONNX Runtime используется для запуска нейросетевых графов на CPU с контролем потоков, памяти и параметров инференса.

### **CPU Execution Provider**

Режим выполнения ONNX Runtime, при котором вычисления производятся на центральном процессоре без использования GPU.

### **I/O Binding**

Механизм ONNX Runtime, позволяющий заранее связать входные и выходные тензоры с выделенными буферами памяти. Использование I/O Binding снижает число лишних выделений памяти и повышает предсказуемость задержки.

### **Конвейер, runtime, hot path**

Конвейер — последовательность этапов обработки данных. Runtime — среда выполнения модели или приложения. Hot path — критический путь обработки кадра, в который не должны входить отладочная визуализация, файловый вывод и подробное логирование.

### **Benchmark, grid search**

Benchmark — измерительный эксперимент с фиксированным протоколом. Grid search — перебор набора конфигураций, например входного размера модели, числа потоков и схемы плиточного инференса, с целью выбора рабочей точки по задержке и качеству покрытия ROI.

### **Latency, throughput, tail latency**

Latency — задержка обработки одного кадра. Throughput — производительность системы, выраженная числом обработанных кадров или задач за единицу времени. Tail latency — задержка в хвосте распределения, обычно характеризующаяся процентилиями p95 и p99.

### **Tiled inference**

Режим инференса, при котором изображение обрабатывается не целиком, а набором перекрывающихся плиток. Такой подход может повысить покрытие мелких объектов,

однако требует сшивания результатов и может увеличивать задержку в зависимости от числа запусков модели.

**NMS** Non-Maximum Suppression — этап постобработки, удаляющий дублирующиеся предсказания, если они относятся к одному объекту и имеют значительное взаимное перекрытие.

### **Letterbox, overlap, remap**

Letterbox — изменение размера изображения с сохранением пропорций и добавлением полей. Overlap — перекрытие соседних плиток при tiled inference. Remap — обратное преобразование координат из пространства модели или плитки в координаты исходного изображения.

### **Precision и Recall**

Precision показывает долю корректных срабатываний среди всех найденных объектов. Recall показывает долю найденных целевых объектов среди всех объектов, присутствующих в разметке.

### **AP, AP50, AP50-95 и mAP**

AP оценивает качество детекции для одного класса по precision–recall кривой. mAP является средним значением AP по классам. AP50 вычисляется при пороге IoU 0.5, тогда как AP50-95 усредняет качество локализации по нескольким IoU-порогам от 0.5 до 0.95. В работе эти метрики используются для оценки YOLO-моделей, но дополняются системными характеристиками: задержкой, размером модели и пригодностью к CPU-инференсу.

### **Confusion matrix**

Матрица ошибок классификации. Диагональные элементы показывают корректные предсказания, а внедиагональные элементы отражают случаи смешения классов между собой или с фоновым классом.

### **Params, GFLOPs**

Params — число параметров модели. GFLOPs — оценка вычислительной стоимости одного прямого прохода модели в миллиардах операций с плавающей точкой. Эти показатели не равны фактической задержке, но позволяют предварительно оценить относительную тяжесть модели.

### **Epoch, batch, learning rate, loss**

Epoch — один полный проход по обучающей выборке. Batch — группа примеров, обрабатываемая за один шаг оптимизации. Learning rate задает величину шага обновления параметров. Loss — функция ошибки, минимизируемая в процессе обучения.

### **Аугментации, mosaic, close mosaic**

Аугментации — искусственные преобразования обучающих изображений, повышающие устойчивость модели к вариативности данных. Mosaic объединяет несколько изображений в один обучающий пример. Close mosaic означает отключение данной аугментации на заключительных эпохах для стабилизации обучения.

## **TextOCR, WIDER FACE, DeepFashion2, Fashionpedia, df2-fp-7c**

TextOCR используется для проверки текстовых ROI на естественных сценах с текстом. WIDER FACE содержит изображения с лицами различных масштабов и условий съемки. DeepFashion2 и Fashionpedia являются датасетами изображений одежды; в работе они приведены к единой 7-классовой таксономии df2-fp-7c для обучения и оценки textured ROI.

## **PSNR, SSIM, VMAF, $Q_{JOD}$ , LPIPS, DISTS**

Метрики качества изображения и видео. PSNR, SSIM, VMAF и  $Q_{JOD}$  обычно интерпретируются как метрики, где большее значение соответствует лучшему качеству. Для LPIPS и DISTS, напротив, меньшее значение означает большее сходство с эталоном. В работе  $Q_{JOD}$  используется как относительная полнореференсная метрика качества в шкале JOD при одинаковой нормализации входных изображений.

## **One-frame HEVC stream**

Покадровая экспериментальная постановка, при которой каждое изображение кодируется как отдельный однокадровый HEVC-поток. Такая постановка изолирует пространственный эффект ROI-aware кодирования, но не оценивает межкадровые механизмы полноценного видеопотока.

## **match\_baseline**

Режим сравнения, при котором параметры ROI-aware кодирования подбираются так, чтобы размер выходного потока был сопоставим с baseline. Это обеспечивает более корректное сравнение при близком битовом бюджете.

## **ROI-only, background, thesis-weighted, ROI-crop-weighted**

Группы метрик, используемые для анализа ROI-aware эксперимента. ROI-only оценивает качество только внутри маски области интереса. Background характеризует качество вне ROI. Thesis-weighted повышает вклад ROI в агрегированную оценку. ROI-crop-weighted рассчитывает качество по кропам ROI с учетом их площади.

## **Improvement, positive share**

Improvement — разница качества ROI-aware результата относительно baseline. В работе положительное значение соответствует преимуществу ROI-aware режима; для LPIPS и DISTS знак разницы инвертируется, поскольку для этих метрик меньшее значение означает лучшее качество. Positive share показывает долю случаев, в которых ROI-aware режим превосходит baseline.

## **qoffset**

Смещение параметра квантования, передаваемое кодировщику для области ROI. Отрицательное значение обычно соответствует более мягкому квантованию и, следовательно, более высокому качеству выбранной области.

## **CRF, preset, GOP**

CRF задает целевой уровень качества в кодировщике. Preset определяет компромисс между скоростью кодирования и эффективностью сжатия. GOP описывает структуру группы кадров и расположение ключевых кадров.

## **FFmpeg, x265, ROI side data**

FFmpeg — мультимедийный фреймворк для обработки аудио- и видеоданных. x265 — программный кодировщик HEVC/H.265. ROI side data — метаданные кадра, через которые кодировщику может быть передана информация о координатах и силе защиты областей интереса.

## **Baseline**

Контрольный режим кодирования без использования ROI. Baseline необходим для сопоставления ROI-aware подхода с обычным кодированием при одинаковых или близких условиях эксперимента.

## **Area budget, area cap, extra area**

Ограничения на суммарную площадь защищаемых ROI. Они предотвращают ситуацию, при которой значительная часть кадра объявляется областью повышенного качества. Extra area характеризует избыточную защищаемую площадь относительно эталонной или ожидаемой геометрии.

## **Tracking, temporal smoothing**

Tracking связывает одну и ту же область интереса между соседними кадрами. Temporal smoothing сглаживает координаты и уверенность ROI во времени, снижая риск мерцания качества при видеокодировании.

## **Rate-control**

Политика управления битрейтом в видеокодировщике. Она определяет распределение битового бюджета между кадрами, блоками и пространственными областями.

## **FPS, end-to-end FPS**

FPS характеризует скорость обработки в кадрах в секунду. End-to-end FPS учитывает весь путь обработки, включая предобработку, инференс, постобработку и вспомогательные операции критического конвейера.

## **CPU affinity, cpuset**

Механизмы закрепления процесса или потоков за определенными ядрами CPU. Они используются для снижения шума измерений и повышения воспроизводимости benchmark-экспериментов.

## **p50, p95, p99**

Процентили распределения задержки. Например, p99 означает, что 99% измерений имеют задержку не выше указанного значения. Для видеоконвейера хвостовые процентили особенно важны, поскольку редкие всплески задержки могут нарушать стабильность обработки.

## **NUMA**

Архитектура памяти, при которой разные группы процессорных ядер имеют неодинаковую близость к различным областям памяти. Нарушение NUMA-локальности может увеличивать задержку CPU-инференса.

## **SIMD/NEON**

Векторные инструкции процессора, позволяющие выполнять одну операцию сразу над несколькими элементами данных. Они могут ускорять предобработку, инференс и постобработку.

## **Top-Down Microarchitecture Analysis**

Методика анализа производительности CPU, показывающая, в каких категориях теряются слоты исполнения: полезные вычисления, подача инструкций, ожидание данных или неверные предсказания ветвлений.



## Введение

**Актуальность темы.** Облачный видеостриминг является одним из наиболее ресурсоемких классов мультимедийных систем. Качество итогового видеопотока ограничивается не только скоростью сети пользователя, но и вычислительной стоимостью транскодирования, задержкой доставки, пропускной способностью инфраструктуры, типом кодировщика и сложностью видеоконтента. В реальных сервисах необходимо одновременно поддерживать приемлемое визуальное качество, ограниченный битрейт, низкую задержку и высокую плотность обработки потоков на серверной инфраструктуре.

Классические алгоритмы адаптивного выбора битрейта решают часть этой задачи на уровне временных сегментов видео. Например, BOLA выбирает представление потока с учетом буфера и качества, а Pensieve применяет learning-based подход к адаптации битрейта [1, 2]. Однако эти методы практически не отвечают на вопрос о том, как распределять качество внутри отдельного кадра. Между тем пользователь воспринимает кадр неравномерно: лицо, текст, субтитры, элементы интерфейса и визуально богатые фактуры имеют большую значимость, чем однородный фон.

**Идея работы** состоит в построении ROI-aware конвейера. Сначала на кадре выделяются области интереса, затем они нормализуются и передаются кодировщику как управляющая информация для повышения качества в более значимых областях. Временная стабилизация ROI рассматривается как задача последующего слоя для полноценного видеопотока, а не как обязательная часть покадровой проверки, выполненной в данной работе. Такой подход не заменяет классическое видеокодирование и не отменяет ABR-логику, а дополняет их пространственным уровнем принятия решений.

**Практическая основа работы** — библиотека IDet [46]. Она проектируется как CPU-ориентированный C++ слой между компьютерным зрением и видеокодированием. В архитектуре IDet текстовые области обрабатываются через адаптеры детекции текста, лица — через адаптеры детекции лиц, а текстурированные области одежды — через отдельный YOLO-детектор. Дополнительным инженерным артефактом является воспроизводимый конвейер обучения YOLO [47], который отвечает за подготовку датасетов, обучение, аудит, экспорт и проверку ONNX-артефактов для развертывания.

**Цель работы** — разработать и обосновать инженерно-научный подход к повышению воспринимаемого качества облачного видеостриминга за счет выделения и защиты областей интереса в условиях ограниченного CPU-бюджета.

**Для достижения цели поставлены следующие задачи:**

- 1) проанализировать облачный видеостриминг как задачу распределения битовых и вычислительных ресурсов;
- 2) формализовать ROI-aware оптимизацию качества как ограниченную задачу с учетом битрейта, задержки и вычислительной стоимости;

- 3) разработать трехрежимную архитектуру ROI-детекции: текст, лица и текстурированные области одежды;
- 4) описать IDet как CPU-oriented C++ библиотеку для формирования геометрического ROI-контракта;
- 5) подготовить YOLO-детектор текстурированных ROI на общей таксономии DeepFashion2 и Fashionpedia;
- 6) провести аудит датасетов, оценить дисбаланс классов и геометрию bounding boxes;
- 7) сравнить YOLO26n и YOLO26s как кандидатов для текстурированных ROI;
- 8) обосновать выбор легкой модели с учетом качества и вычислительной стоимости;
- 9) описать схему интеграции ROI с кодировщиком через приоритеты, бюджет площади и qoffset-политику;
- 10) провести покадровое сравнение baseline и ROI-aware кодирования при сопоставимом размере выходного потока.

**Объект исследования** — облачный конвейер видеостриминга и транскодирования.

**Предмет исследования** — методы выделения и использования областей интереса для повышения воспринимаемого качества видео при ограниченных вычислительных ресурсах.

**Терминологическая оговорка.** В работе сохраняются англоязычные названия технологий и метрик, если они являются устоявшимися именами: ONNX Runtime, FFmpeg, VMAF, LPIPS, YOLO, DBNet, SCRFD. Для общих инженерных понятий используется русская терминология: **конвейер** вместо pipeline, **среда выполнения** вместо runtime, **артефакт для развертывания** вместо deployable artifact. Краткие имена режимов **text**, **face** и **textured ROI** используются в таблицах и графиках; в основном тексте им соответствуют текстовые ROI, ROI лиц и текстурированные ROI.

**Научно-практическая новизна** работы заключается в том, что ROI-детекция рассматривается не как изолированная задача компьютерного зрения, а как элемент системной оптимизации видеокодирования. В отличие от подхода, где модель оценивается только по mAP, в данной работе качество детектора анализируется вместе с задержкой, размером модели, предсказуемостью предобработки и постобработки, а также возможностью интеграции в CPU-ориентированный C++ конвейер.

# Глава 1. Предметная область и постановка задачи

## 1.1 Облачный видеостриминг как задача распределения ресурсов

Облачный видеостриминг строится вокруг компромисса между качеством, стоимостью и задержкой. В типовом сценарии платформа получает исходный видеопоток, транскодирует его в несколько представлений и доставляет пользователю версию, соответствующую текущим сетевым условиям. На практике этот процесс ограничен вычислительными ресурсами, числом параллельных потоков, стоимостью хранения, пропускной способностью сети и требованиями к задержке.

Классические ABR-алгоритмы, такие как BOLA [1], и learning-based подходы, такие как Pensieve [2], оптимизируют выбор битрейта во времени. Их задача состоит в том, чтобы выбрать подходящее представление сегмента, не допустить rebuffering и сохранить стабильность воспроизведения. Однако даже правильный выбор битрейта не гарантирует, что кодировщик распределит качество внутри кадра так, как это важно пользователю. На одном и том же битрейте можно сохранить фон, но потерять лицо или текст; можно защитить лицо, но ухудшить второстепенные области. Поэтому пространственное распределение качества становится отдельной задачей.

Современные видеокодеки, включая H.264/AVC и HEVC/H.265, основаны на блочном представлении кадра, предсказании, преобразовании, квантовании и энтропийному кодированию [11, 12, 13, 14]. Значительная часть потерь качества связана с квантованием. Следовательно, управление параметром квантования является естественной точкой воздействия: если некоторая область кадра важнее остальных, для нее можно использовать более мягкое квантование, а для менее важных областей — более агрессивное. На этой идее основано ROI-aware кодирование.

## 1.2 Почему глобальные метрики недостаточны

Глобальные метрики качества, такие как PSNR, SSIM, VMAF и LPIPS, полезны для сравнения кодеков и режимов кодирования, но не всегда отражают пользовательскую значимость локальных дефектов. Если кодировщик ухудшил небольшую область с лицом, текстом или важной фактурой, средняя метрика по кадру может измениться незначительно. При этом пользователь может сразу заметить потерю читаемости текста или деградацию лица.

Поэтому в данной работе важно перейти от оценки равномерного качества кадра к ROI-weighted quality. Пусть кадр содержит множество областей  $R_i$  с весами  $w_i$ , отражающими их зрительную или прикладную значимость. Тогда агрегированное качество можно записать как

$$Q_{\text{ROI}}(F) = \frac{\sum_{i=1}^N w_i \cdot Q(F | R_i)}{\sum_{i=1}^N w_i}, \quad (1.1)$$

где  $F \mid R_i$  означает фрагмент кадра внутри области  $R_i$ , а  $Q(\cdot)$  — локальная метрика качества. Эта формула не заменяет VMAF или SSIM, а задает способ интерпретировать их в привязке к областям интереса.

Особенность ROI-aware подхода состоит в том, что глобальная метрика может измениться умеренно даже при полезном локальном эффекте. Если ROI занимает 10–20% кадра, улучшение в этих областях может быть частично компенсировано ухудшением фона. Это не является ошибкой постановки: цель ROI-aware кодирования состоит не в равномерном повышении качества всего кадра, а в перераспределении качества в пользу областей с большей зрительной ценностью.

### 1.3 ROI-aware кодирование в существующих исследованиях

Идея защиты пространственно важных областей используется в работах по ROI-based и saliency-aware video coding. В таких исследованиях рассматриваются схемы, где битовый бюджет перераспределяется между частями кадра с разной семантической или зрительной значимостью [18, 19, 20, 21]. Обычно задача делится на два уровня: определение ROI и внедрение этой информации в rate-control или quantization decision кодировщика.

В данной работе эти уровни также разделяются. IDet отвечает за построение списка областей интереса и их геометрическую нормализацию. Последующий интеграционный слой должен преобразовать эти области в приоритеты и подсказки кодировщику: qoffset, ROI side data или резервную политику. Такое разделение важно, потому что модели компьютерного зрения и видеокодировщик имеют разные контракты, разные ограничения и разные критерии качества.

Системная сложность ROI-aware подхода состоит в том, что локальное улучшение качества не должно достигаться любой ценой. Если ROI занимает слишком большую часть кадра, кодировщик теряет возможность перераспределять биты. Если ROI нестабилен во времени, качество может мерцать. Если детектор слишком медленный, выигрыш качества будет куплен нарушением бюджета задержки. Поэтому ROI-aware кодирование в работе рассматривается как ограниченная оптимизационная задача, а не как простое добавление еще одной нейросетевой модели.

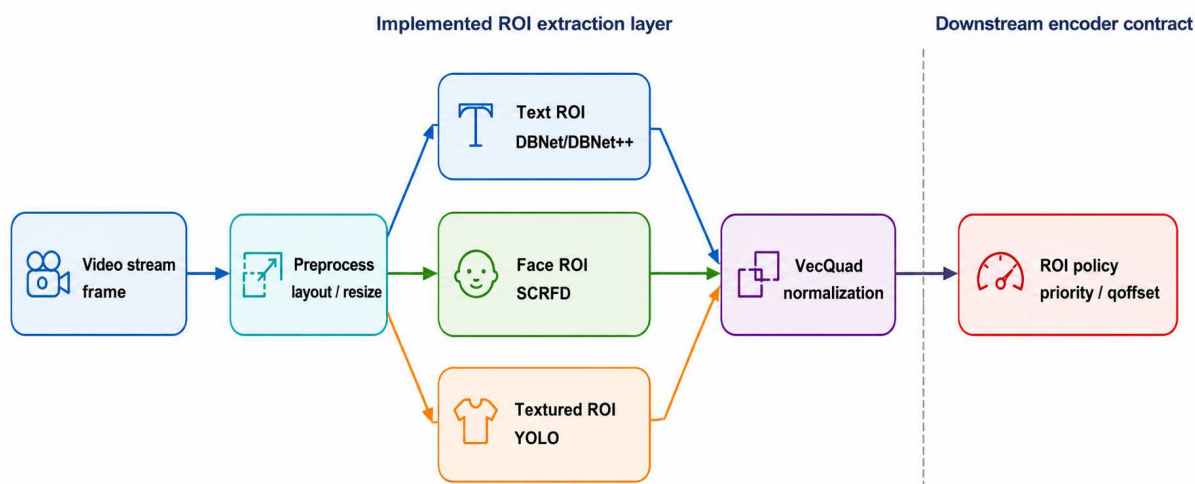
### 1.4 Три режима ROI в предлагаемой системе

В предлагаемой архитектуре IDet рассматриваются три режима ROI:

- 1) **текстовые ROI** — области текста, субтитров, интерфейсных элементов и мелких надписей;
- 2) **ROI лиц** — лица ведущих, участников и людей, заметных в кадре;
- 3) **текстурированные ROI** — области одежды и визуально богатых фактур, приближенно представленные через классы одежды.

Текстовая ветка опирается на идеи сегментационной детекции текста. DBNet встраивает дифференцируемую бинаризацию в модель, что уменьшает зависимость от внешней постобработки и делает подход удобным для потокового сценария [22]. DBNet++ развивает

эту идею через объединение признаков разных масштабов, что полезно для текста разных размеров и пропорций [23]. Ветвь лиц основана на SCRFD: это семейство детекторов проектировалось как эффективный компромисс точности и вычислительной стоимости за счет перераспределения обучающих примеров и вычислений [24]. Третья ветка добавляет текстурированные ROI на базе YOLO-детектора одежды. Она не заменяет детекцию текста и лиц, а расширяет систему на визуально богатые области, где потеря высокочастотной структуры ухудшает воспринимаемую резкость.



**Рисунок 1.1:** Архитектура ROI-aware конвейера на базе IDet: текстовые ROI, ROI лиц и текстурированные ROI объединяются в геометрический выход, а последующий слой преобразует его в приоритеты для кодировщика.

Рисунок 1.1 фиксирует главное архитектурное разделение: IDet отвечает за геометрию текста, лиц и одежды, а политика кодирования работает уже с нормализованными областями и не зависит от внутреннего формата конкретной модели.

## 1.5 Выводы по главе

В первой главе задача облачного видеостриминга сведена к управлению ограниченными ресурсами: битрейтом, задержкой и вычислительным бюджетом. Обычный ABR-подход выбирает качество во времени, но не решает, какие части одного кадра важнее для зрителя.

Для этой работы ключевым является пространственное перераспределение качества. Текст, лица и фактурные области одежды выделяются как три класса ROI, потому что их деградация заметна даже при небольшой занимаемой площади. Дальнейшие главы поэтому рассматривают не абстрактное улучшение всего кадра, а проверяемый путь от детекции этих областей к управляющему сигналу для кодировщика.

## Глава 2. Формальная постановка задачи ROI-aware оптимизации

### 2.1 Ограниченная оптимизация качества

В работе задача улучшения качества видеопотока формулируется как ограниченная оптимизация. Пусть  $B$  — доступный битрейт,  $T$  — допустимая задержка анализа кадра,  $C_{\max}$  — вычислительный бюджет, а  $\theta$  — параметры ROI-детектора и политики кодирования. Тогда постановку можно записать так:

$$\max_{\theta} \mathbb{E}[Q_{\text{ROI}}(F; \theta)] \quad \text{при условиях} \quad R(F; \theta) \leq B, \quad L(F; \theta) \leq T, \quad C(F; \theta) \leq C_{\max}. \quad (2.1)$$

Здесь  $R$  — фактический расход битрейта,  $L$  — задержка обработки кадра,  $C$  — вычислительная стоимость, а  $Q_{\text{ROI}}$  — ROI-взвешенное качество. В отличие от лабораторной постановки, где можно максимизировать только mAP модели, в данной работе важна системная метрика: улучшение качества должно быть достигнуто без выхода за бюджет задержки и без чрезмерного роста вычислительной стоимости.

Эта постановка отражает практическую природу облачного видеостриминга. Даже если более тяжелая модель точнее на валидационном разбиении, она может оказаться непригодной для конвейера реального времени. Напротив, легкая модель с немного меньшей mAP может быть предпочтительнее, если она обеспечивает стабильное p95/p99 и достаточное покрытие ROI.

### 2.2 Бюджет площади ROI

Если защищать слишком большую часть кадра, кодировщик теряет возможность перераспределять биты. Поэтому вводится ограничение на долю защищаемой площади:

$$\frac{\left| \bigcup_{i=1}^N R_i \right|}{|F|} \leq \alpha, \quad (2.2)$$

где  $\alpha$  — максимально допустимая доля кадра, например 0.15–0.35 в зависимости от типа контента. Это ограничение важно не только математически, но и инженерно: оно защищает систему от ложных срабатываний и от ситуации, когда ROI-aware режим фактически превращается в кодирование всего кадра с повышенным качеством.

Бюджет площади должен применяться после объединения результатов разных детекторов. Несколько корректных ROI по отдельности могут вместе занять слишком большую часть кадра. Особенно это важно для текстурированных ROI: области одежды могут быть крупными, поэтому без ограничения площади они начнут конкурировать с текстом и лицами за битовый бюджет. В типовой политике текст имеет самый высокий приоритет, лица — высокий, а текстурированные ROI — средний.

## 2.3 Временная стабильность ROI

Для видео важна не только точность ROI в отдельном кадре, но и стабильность во времени. Если область резко меняет положение между соседними кадрами, это может привести к мерцанию качества после кодирования. Для сглаживания уверенности можно использовать рекуррентную формулу:

$$\hat{s}_t = \lambda \hat{s}_{t-1} + (1 - \lambda) s_t, \quad (2.3)$$

где  $s_t$  — текущая уверенность детектора,  $\hat{s}_t$  — сглаженная уверенность, а  $\lambda \in [0, 1]$  — коэффициент памяти.

Дополнительно можно ввести штраф за нестабильность геометрии ROI между соседними кадрами. Для этого используется метрика, основанная на пересечении областей:

$$P_{\text{flicker}} = \frac{1}{T-1} \sum_{t=2}^T (1 - \text{IoU}(R_t, R_{t-1})). \quad (2.4)$$

Здесь  $R_t$  обозначает ROI на кадре  $t$ , а  $\text{IoU}(R_t, R_{t-1})$  показывает, насколько сильно область на текущем кадре совпадает с областью на предыдущем кадре. Если две области почти совпадают, значение IoU близко к единице, а вклад в штраф близок к нулю. Если область резко смещается, уменьшается или появляется в другом месте кадра, IoU становится малым, а величина  $1 - \text{IoU}$  возрастает.

Таким образом,  $P_{\text{flicker}}$  можно интерпретировать как среднюю величину межкадрового изменения ROI. Чем выше это значение, тем менее стабильна геометрия областей интереса и тем выше риск визуального мерцания качества после передачи ROI кодировщику. Поэтому временное сглаживание, сопровождение областей между кадрами и подавление кратковременных ложных срабатываний должны рассматриваться как часть последующего ROI-слоя, а не как обязанность конкретной нейросетевой модели.

## 2.4 От ROI к qoffset кодировщика

FFmpeg предоставляет структуру `AVRegionOfInterest`, где область задается координатами и параметром `qoffset`; отрицательное значение `qoffset` означает повышение качества кодирования внутри ROI [15]. На уровне фильтров аналогичная идея доступна через `addroi`: фильтр прикрепляет к кадру метаданные об областях интереса для последующего кодирования, не изменяя сами пиксели [16].

Упрощенное преобразование приоритета  $p_i$  в смещение QP можно записать так:

$$\Delta QP_i = -\gamma \cdot p_i \cdot (QP_{\text{max}} - QP_{\text{min}}), \quad (2.5)$$

где  $p_i \in [0, 1]$  — нормированный приоритет области, а  $\gamma$  — коэффициент силы защиты. Формула не задает конкретную реализацию кодировщика, но фиксирует общий принцип: чем важнее область, тем сильнее отрицательное смещение QP.

Важно, что `qoffset` является подсказкой для rate-control механизма, а не абсолютным запретом на потери внутри ROI. Кодировщик по-прежнему ограничен общим битовым бюджетом, структурой кадра и собственными эвристиками распределения битов. Поэтому корректная ROI-политика должна одновременно задавать приоритет, ограничивать суммарную площадь ROI и оставлять кодировщику достаточно свободы для обработки фона.

## 2.5 Метрики детекции

Для модели текстурированных ROI используются стандартные метрики детекции объектов: точность, полнота, F1, AP, mAP@0.5 и mAP@0.5:0.95 [34]. Точность и полнота вычисляются по формулам:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (2.6)$$

а F1-мера определяется как

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2.7)$$

В контексте ROI-задачи полнота особенно важна: пропуск значимой области одежды хуже, чем неточная классификация типа одежды, потому что цель детектора текстурированных ROI — не модная аналитика, а защита визуально богатой области от агрессивного сжатия. Однако точность также нельзя игнорировать: ложные текстурированные ROI увеличивают защищаемую площадь и могут отнимать битовый бюджет у текста и лиц.

AP измеряет площадь под precision–recall кривой при заданном IoU-пороге, а mAP@0.5:0.95 усредняет AP по нескольким порогам IoU от 0.5 до 0.95. Для ROI-aware кодирования это различие принципиально: mAP@0.5 показывает, насколько модель в целом находит объекты, а mAP@0.5:0.95 строже оценивает точность bbox. Слишком широкий bbox защищает лишний фон, а слишком узкий может не покрыть значимую фактуру. Поэтому detection metrics должны дополняться area-based метриками покрытия ROI.

## 2.6 Выводы по главе

Формальная постановка фиксирует главный компромисс работы: ROI-aware система должна повышать локальное качество значимых областей, не выходя за ограничения по битрейту, задержке, вычислительной стоимости и площади ROI. Поэтому одной mAP детектора недостаточно для вывода о полезности системы.

Для дальнейших экспериментов это задает два уровня проверки. Первый уровень — качество детекции и геометрии ROI: precision, recall, AP, mAP и area-based метрики. Второй уровень — видеокодирование: `qoffset`, ограничение площади, стабильность ROI во времени и ROI-взвешенные метрики качества. Только совместная проверка этих уровней позволяет сделать аккуратный вывод о практической пользе подхода.



## Глава 3. Проектирование IDet и трехрежимного ROI-конвейера

### 3.1 IDet как инженерное ядро работы

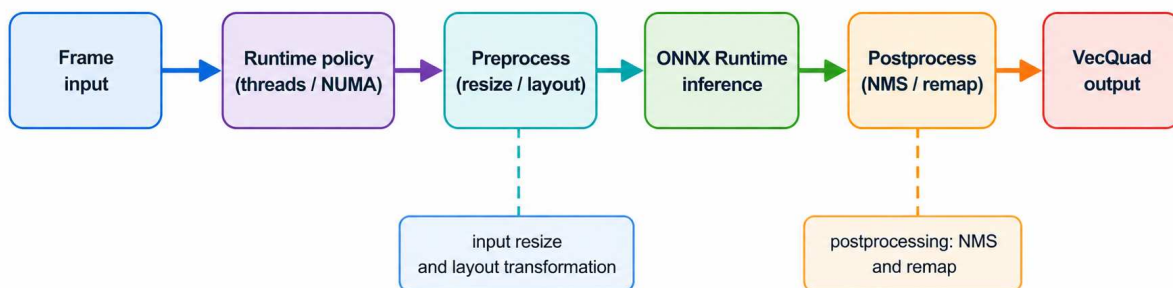
IDet — это CPU-ориентированная C++ библиотека для детекции и нормализации областей интереса [46]. В данной работе она рассматривается как центральный инженерный результат: задача библиотеки состоит не в обучении единственной модели, а в связывании нескольких специализированных детекторов с видеокодировщиком через устойчивый геометрический контракт.

Фактический публичный контракт IDet возвращает список четырехугольников `VecQuad`. Типизация ROI, приоритеты и `qoffset` относятся к последующему слою интеграции с кодировщиком. Такое разделение важно: IDet отвечает за компьютерное зрение и геометрию, а адаптер кодировщика — за конкретную политику кодирования.

Архитектурно IDet опирается на несколько принципов:

- 1) критический путь обработки должен работать на CPU без зависимости от Python;
- 2) модели должны быть приведены к ONNX-формату, пригодному для развертывания;
- 3) инференс должен выполняться через ONNX Runtime [25, 26];
- 4) входные и выходные буферы должны переиспользоваться;
- 5) число потоков должно задаваться явно;
- 6) модельно-зависимые детали не должны протекать в пользовательский код.

ONNX Runtime поддерживает оптимизацию вычислительного графа, включая слияние операций и удаление лишних узлов [28]. Механизм I/O Binding позволяет заранее связать входы и выходы с выбранными буферами, что уменьшает число копирований и делает размещение памяти более предсказуемым [27].



**Рисунок 3.1:** Путь обработки одного кадра в IDet: подготовка изображения, CPU-инференс, постобработка и возврат списка `VecQuad`.

**Таблица 3.1:** Основные модули IDet в критическом пути обработки кадра

| Модуль                        | Ответственность                                                                                                      | Выходной результат                                           |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Слой входного кадра           | Прием кадра, проверка формата, масштабирование и подготовка рабочего представления.                                  | Нормализованный кадр или набор плиток.                       |
| Слой предобработки            | Letterbox, изменение размера, нормализация каналов, преобразование раскладки и заполнение входных тензорных буферов. | Готовые входные буферы для ONNX Runtime.                     |
| Адаптеры моделей              | Обертки над детекторами текста, лиц и текстур; скрывают формат выходов конкретной модели.                            | Сырые кандидаты: область, оценка уверенности, класс или тип. |
| Слой постобработки            | Пороговая фильтрация, декодирование координат, NMS, сшивание плиток и обратное преобразование координат.             | Кандидаты ROI в координатах исходного кадра.                 |
| Публичный геометрический слой | Приведение результата к публичному контракту библиотеки.                                                             | VecQuad.                                                     |

**Таблица 3.2:** Последующие слои интеграции IDet с видеокодировщиком

| Слой                 | Ответственность                                                                                              | Выходной результат                      |
|----------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Последующий ROI-слой | Добавление типа ROI, уверенности, приоритета, ограничения площади и временного сглаживания поверх геометрии. | Метаданные ROI для кодировщика.         |
| Адаптер кодировщика  | Преобразование метаданных ROI в <code>qoffset</code> , ROI side data или резервную политику.                 | Управляющая информация для кодировщика. |
| Слой измерений       | Измерение задержки, числа ROI, площади ROI и стабильности результатов.                                       | p50/p95/p99 и диагностические отчеты.   |

Рисунок 3.1 показывает, что на критическом пути нет обучающего Python-конвейера: кадр проходит через подготовку входа, ONNX Runtime и модельно-зависимую постобработку, после чего внешний слой получает только унифицированную геометрию.

## 3.2 Жизненный цикл обработки кадра

Жизненный цикл кадра в IDet начинается до запуска нейросетевой модели. Сначала система получает кадр в исходном разрешении, проверяет формат пикселей и выбирает режим обработки: полный кадр, tiled inference, downscale или повторное использование ROI с предыдущих кадров. Затем выполняется предобработка: resize/letterbox, нормализация и преобразование layout в формат, ожидаемый ONNX-моделью.

После инференса каждый движок декодирует выходы своей модели. Движок текста преобразует карты вероятностей DBNet-подобной модели в четырехугольники, движок лиц декодирует выходы SCRFD, а движок текстур преобразует выходы YOLO в кандидаты bbox/quad. На публичном уровне результат нормализуется в VecQuad; последующий слой уже поверх этой геометрии добавляет тип, приоритет и qoffset-рекомендацию.

**Таблица 3.3:** Жизненный цикл кадра в ROI-aware конвейере на базе IDet

| Шаг | Этап                           | Смысл этапа                                                                                                 |
|-----|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1   | Получение кадра                | Получение кадра и фиксация размеров, временной метки и формата пикселей.                                    |
| 2   | План обработки                 | Выбор режима обработки: полный кадр, плитки, пропуск тяжелого детектора или перенос ROI.                    |
| 3   | Предобработка                  | Подготовка входных буферов: изменение размера, letterbox, нормализация, преобразование раскладки.           |
| 4   | Инференс                       | Выполнение одной или нескольких сессий ONNX Runtime с контролируемым числом потоков.                        |
| 5   | Постобработка модели           | Декодирование выходов моделей, пороговая фильтрация, NMS, сшивание плиток и обратное отображение координат. |
| 6   | Публичный геометрический выход | Возврат унифицированной геометрии в формате VecQuad.                                                        |
| 7   | Интеграция ROI                 | Добавление типа ROI, сортировка, слияние пересечений, ограничение площади и временное сглаживание.          |
| 8   | Управление кодировщиком        | Преобразование метаданных ROI в qoffset, side data или резервные параметры.                                 |
| 9   | Телеметрия                     | Запись задержки, числа ROI, площади ROI и диагностической информации.                                       |

## 3.3 Ответственность адаптеров детекторов

Адаптер в IDet отвечает за преобразование конкретной модели в общий контракт библиотеки. Он не должен принимать решения о том, какие ROI важнее для кодировщика: это задача последующего ROI-слоя. Такой принцип уменьшает связность и позволяет заменять модели без переписывания слоя интеграции с кодировщиком.

**Таблица 3.4:** Ответственность адаптеров IDet

| Адаптер             | Внутренняя ответственность                                                                                                | Что не входит в ответственность                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Адаптер текста      | Предобработка под DBNet/DBNet++, декодирование карт текста, извлечение многоугольников, фильтрация по уверенности.        | Глобальный приоритет текста относительно лица и текстурированных ROI. |
| Адаптер лиц         | Изменение размера и нормализация под SCRFD, декодирование кандидатов, NMS лиц, нормализация уверенности.                  | Решение о <code>qoffset</code> и ограничение общей площади ROI.       |
| Адаптер текстур     | Предобработка под YOLO, декодирование координат и классов, NMS, обратное отображение координат из letterbox-пространства. | Интерпретация классов одежды как поли-тик кодирования.                |
| Адаптер кодировщика | Преобразование метаданных ROI в FFmpeg ROI side data, карту <code>qoffset</code> или резервные параметры.                 | Запуск детекторов и изменение выходов моделей.                        |

### 3.4 Публичный контракт IDet

Важное архитектурное решение IDet состоит в том, что конкретная модель не протекает в пользовательский код. Публичный слой выбирает задачу и движок через `Task` и `EngineKind`, а пользователь получает геометрию областей как `VecQuad`. Ниже приведен сокращенный фрагмент публичного интерфейса библиотеки.

**Листинг 3.1:** Сокращенный фрагмент публичного API IDet

```

1 namespace idet {
2
3 enum class Task : std::uint8_t {
4     None = 0,
5     Text,
6     Face,
7     Cloth,
8 };
9
10 enum class EngineKind : std::uint8_t {
11     None = 0,
12     DBNet,
13     SCRFD,
14     Yolo,
15 };
16
17 struct Point2f {
18     float x = 0.0f;

```

```

19     float y = 0.0f;
20 };
21
22 using Quad = std::array<Point2f, 4>;
23 using VecQuad = std::vector<Quad>;
24
25 struct GridSpec {
26     int rows = 1;
27     int cols = 1;
28 };
29
30 struct InferenceOptions {
31     bool bind_io = false;
32
33     float bin_thresh = 0.3f;
34     float box_thresh = 0.5f;
35     float nms_iou = 0.3f;
36
37     int min_roi_size_w = 5;
38     int min_roi_size_h = 5;
39
40     GridSpec fixed_input_dim{0, 0};
41     GridSpec tiles_dim{1, 1};
42     float tile_overlap = 0.1f;
43 };
44
45 struct RuntimePolicy {
46     int ort_intra_threads = 1;
47     int ort_inter_threads = 1;
48     int tile_omp_threads = 1;
49
50     bool soft_mem_bind = true;
51     bool suppress_opencv = true;
52 };
53
54 struct DetectorConfig final {
55     Task task = Task::None;
56     EngineKind engine = EngineKind::None;
57
58     InferenceOptions infer{};
59     RuntimePolicy runtime{};
60
61     std::string model_path{};
62     bool verbose = true;
63
64     [[nodiscard]] Status validate() const noexcept;
65
66     static DetectorConfig setup(Task task, std::string model_path);
67 };

```

```

68
69 class Detector final {
70 public:
71     Detector() noexcept = default;
72     ~Detector() noexcept;
73
74     Detector(Detector&& other) noexcept;
75     Detector& operator=(Detector&& other) noexcept;
76
77     Detector(const Detector&) = delete;
78     Detector& operator=(const Detector&) = delete;
79
80     explicit operator bool() const noexcept;
81
82     Task task() const noexcept;
83     EngineKind engine() const noexcept;
84
85     void reset() noexcept;
86
87     static Result<Detector> create(
88         const DetectorConfig& config
89     ) noexcept;
90
91     Status prepare_binding(
92         int width,
93         int height,
94         int contexts
95     ) noexcept;
96
97     Result<VecQuad> detect(
98         const Image& image
99     ) noexcept;
100
101     Result<VecQuad> detect_bound(
102         const Image& image,
103         int ctx_idx
104     ) noexcept;
105
106 private:
107     void* impl_ = nullptr;
108     const detail::DetectorVTable* vtbl_ = nullptr;
109 };
110
111 } // namespace idet

```

Этот контракт фиксирует фактическую границу между C++ библиотекой и внешним видеоконвейером. Модельно-зависимые детали — letterbox, преобразование раскладки, нормализация входа, декодирование выходов, пороговая фильтрация и NMS — скрыты внутри

движков. Последующий слой, необходимый для ROI-aware кодирования, должен добавлять к возвращенной геометрии тип области, приоритет и `qoffset`; в текущем публичном API это не отдельный класс библиотеки.

### 3.5 Последующий контракт метаданных ROI

Для кодировщика одной геометрии недостаточно. Ему важно знать тип области, уверенность, приоритет и рекомендуемую силу защиты. Поэтому поверх `VecQuad` вводится последующая запись метаданных ROI. Она может быть реализована в слое интеграции с кодировщиком и не должна смешиваться с текущим публичным API `IDet`.

**Таблица 3.5:** Последующий контракт метаданных ROI между детекторами, ROI-слоем и адаптером кодировщика

| Поле                     | Пример                                                      | Назначение                                                           |
|--------------------------|-------------------------------------------------------------|----------------------------------------------------------------------|
| <code>kind</code>        | <code>text, face, texture</code>                            | Тип ROI, определяющий приоритет и политику <code>qoffset</code> .    |
| <code>quad/bbox</code>   | <code>TL, TR, BR, BL</code> или <code>x1, y1, x2, y2</code> | Геометрия области в координатах исходного кадра.                     |
| <code>confidence</code>  | <code>0.0–1.0</code>                                        | Уверенность конкретного детектора.                                   |
| <code>priority</code>    | <code>0.0–1.0</code>                                        | Системный приоритет после учета типа и уверенности.                  |
| <code>area</code>        | доля кадра                                                  | Используется для бюджета площади и отсеечения чрезмерно больших ROI. |
| <code>qoffsetHint</code> | отрицательное значение                                      | Предлагаемое смещение квантования для адаптера кодировщика.          |
| <code>source</code>      | <code>dbnet, scrfd, yolo26n</code>                          | Источник ROI для диагностики и ablation study.                       |
| <code>trackId</code>     | целое число или <code>none</code>                           | Связь ROI между кадрами для временного сглаживания.                  |

### 3.6 Приоритеты и разрешение конфликтов

В системе вводится иерархия приоритетов: текст > лицо > текстурированные ROI > фон. Текст получает максимальный приоритет, потому что даже небольшие артефакты могут сделать его нечитаемым. Лицо получает высокий приоритет из-за чувствительности пользователя к искажениям человеческой внешности. Текстурированные ROI получают средний приоритет: фактура повышает воспринимаемую резкость, но не должна вытеснять текст и лица при ограниченном бюджете площади.

### 3.7 Память, буферы и критический путь без динамических аллокаций

Для видеостриминга в реальном времени важна не только средняя задержка, но и устойчивость p95/p99. Динамические аллокации в критическом пути приводят к джиттеру: отдельные кадры внезапно обрабатываются дольше из-за выделения памяти, освобождения памяти или перераспределения контейнеров. Поэтому `IDet` должен использовать явную модель владения буферами.

**Таблица 3.6:** Алгоритм слияния ROI

| Шаг | Действие                                                                                 |
|-----|------------------------------------------------------------------------------------------|
| 1   | Получить кандидаты ROI от адаптеров текста, лиц и текстур.                               |
| 2   | Нормировать координаты, уверенность и тип области к последующей записи метаданных ROI.   |
| 3   | Удалить кандидаты с уверенностью ниже порога, зависящего от типа ROI.                    |
| 4   | Отсортировать ROI по приоритету: текст > лицо > текстура > фон.                          |
| 5   | Объединить сильно пересекающиеся области с учетом IoU и типа области.                    |
| 6   | Ограничить суммарную защищаемую площадь кадра заданным бюджетом площади.                 |
| 7   | Применить временное сглаживание к координатам и уверенности для уменьшения мерцания ROI. |
| 8   | Вернуть упорядоченный список метаданных ROI для адаптера кодировщика.                    |

**Таблица 3.7:** Модель владения буферами в IDet

| Буфер                         | Владелец                      | Правило использования                                                              |
|-------------------------------|-------------------------------|------------------------------------------------------------------------------------|
| Исходный кадр                 | Слой входного кадра           | Только чтение после получения кадра; исходное изображение не меняется детекторами. |
| Предобработанный вход         | Слой предобработки            | Переиспользуется между кадрами одного размера; заполняется перед инференсом.       |
| Входные/выходные тензоры ONNX | Бэкенд инференса              | Выделяются при инициализации сессии или при смене формы входа.                     |
| Массивы кандидатов            | Адаптер модели                | Очищаются и переиспользуются; вместимость растет только вне критического пути.     |
| Рабочий буфер NMS             | Слой постобработки            | Общий временный буфер для сортировки и подавления кандидатов.                      |
| VecQuad                       | Публичный геометрический слой | Финальный геометрический результат публичного API.                                 |
| Список метаданных ROI         | Последующий ROI-слой          | Список областей с типом и приоритетом для адаптера кодировщика.                    |

### 3.8 Политика потоков

IDet должен контролировать два уровня параллелизма: внешний уровень обработки кадров/плиток и внутренний уровень потоков ONNX Runtime. Если оба уровня включены без ограничений, возникает переподписка потоков: несколько пулов потоков конкурируют за одни и те же CPU-ядра, что ухудшает p95/p99 задержку. Поэтому политика потоков должна задаваться явно и быть частью конфигурации.

### 3.9 Инварианты и отказоустойчивость

Для производственного конвейера важно определить не только идеальный путь, но и инварианты, которые должны сохраняться при ошибках. IDet и последующий ROI-слой должны гарантировать следующие свойства:

- 1) координаты ROI находятся внутри границ исходного кадра;



**Таблица 3.8:** Политика потоков в IDet

| Уровень                   | Настраиваемый параметр                       | Инженерный смысл                                                                 |
|---------------------------|----------------------------------------------|----------------------------------------------------------------------------------|
| Параллелизм кадров        | Число одновременно обрабатываемых кадров.    | Увеличивает пропускную способность, но может ухудшить задержку отдельного кадра. |
| Параллелизм плиток        | Число плиток, обрабатываемых параллельно.    | Полезно для крупных кадров, но требует аккуратного сшивания результатов.         |
| Потоки ORT intra-op       | Число потоков внутри оператора ONNX Runtime. | Влияет на скорость сверточных и матричных операций.                              |
| Потоки ORT inter-op       | Параллелизм между независимыми узлами графа. | Эффективен не для всех моделей; требует профилирования.                          |
| Частота запуска детектора | Частота запуска каждой ветки ROI.            | Позволяет запускать тяжелые детекторы реже и переносить ROI во времени.          |

- 2) суммарная площадь защищаемых областей не превышает заданный бюджет  $\alpha$ ;
- 3) пересекающиеся ROI не приводят к конфликтующим `qoffset` одного и того же блока;
- 4) при сбое одного детектора остальные режимы могут продолжить работу;
- 5) при превышении бюджета задержки система может временно перейти к переносу ROI предыдущих кадров или к более дешевой ветке;
- 6) отсутствие нативной поддержки ROI в кодировщике не должно ломать кодирование: адаптер кодировщика обязан перейти к резервной политике.

Эти инварианты показывают, что IDet проектируется не как демонстрационный скрипт, а как слой, который может быть встроен в реальный видеоконвейер с ограничениями по времени, памяти и надежности.

### 3.10 Выводы по главе

IDet в работе выступает не как одиночная модель, а как C++ слой, который приводит разные детекторы к одному геометрическому контракту. Публичный выход `VecQuad` намеренно минимален: он скрывает DBNet-, SCRFD- и YOLO-специфику и позволяет внешнему видеоконвейеру работать с единым представлением областей.

Последующие метаданные ROI, приоритеты и `qoffset` отделены от публичного API библиотеки. Такое разделение делает архитектуру последовательной: IDet отвечает за CPU-инференс, буферы, потоки, tiled inference и геометрию, а адаптер кодировщика отвечает за конкретную политику качества в H.264/HEVC/FFmpeg или другом backend.

## Глава 4. Детектор текстурированных ROI: данные, обучение и экспорт

### 4.1 Мотивация детекции текстурированных ROI

ROI-aware видеокодирование часто связывают с лицами и текстом, поскольку эти области имеют очевидную семантическую ценность. Однако в пользовательском видео существенную часть ощущения резкости формируют текстуры: одежда ведущего, ткань, узор, ворс и другие мелкие регулярные структуры. При сильном сжатии такие области теряют высокочастотные детали и превращаются в сглаженные пятна. Поэтому текстурированные ROI рассматриваются как третий режим IDet: он не конкурирует с текстовыми ROI и ROI лиц, а расширяет систему на визуально богатые области.

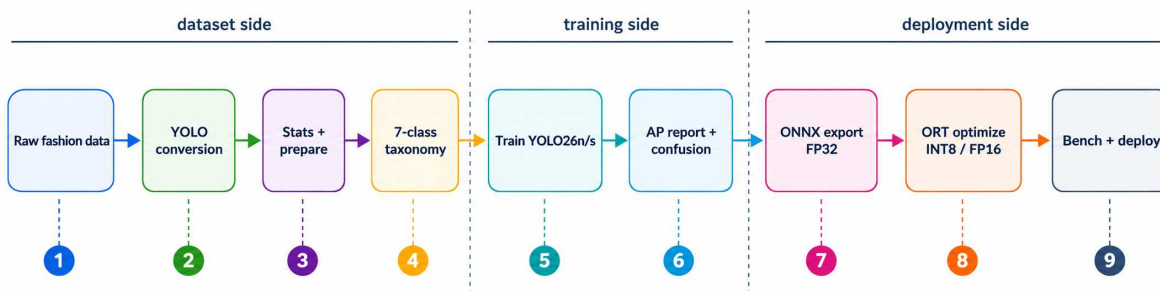
Для этой ветки используется объектный детектор семейства YOLO, обученный на согласованной таксономии одежды. Выбор детекции объектов вместо общей модели визуальной значимости связан с инженерной управляемостью: `bbbox` легко преобразовать в ROI кодировщика, ошибки можно анализировать через AP и матрицу ошибок, а задержку можно профилировать в CPU-ориентированном контуре.

Текстурированная ROI имеет другой смысл, чем текст и лицо. Текст обычно либо читается, либо теряет полезность; лицо связано с социальной значимостью и быстро привлекает внимание пользователя. Одежда и фактуры дают более мягкий, но устойчивый вклад в воспринимаемую резкость сцены. Поэтому этот режим рассматривается как дополнительный слой качества: он должен улучшать важные детали, но не вытеснять более приоритетные области из битового бюджета кодировщика.

### 4.2 Конвейер обучения YOLO

Для обучения детектора текстурированных ROI используется отдельный воспроизводимый конвейер обучения YOLO [47]. Его задача — обеспечить полный цикл подготовки и проверки модели: перевод исходных аннотаций в YOLO-представление, согласование таксономии, аудит разметки, запуск обучения, построение отчетов, экспорт ONNX и последующую проверку, ориентированную на среду выполнения. Это отличает конвейер от разового обучающего скрипта: модель не считается готовой после появления контрольной точки, пока не проверены данные, метрики, переносимость ONNX-графа и контракт развертывания.

Конвейер разделяет доменную логику и CLI-слой. Датасетная часть отвечает за конвертацию исходных схем, статистический аудит и подготовку YAML-описания данных. Обучающая часть строит план обучения и запускает Ultralytics YOLO. Отчетная часть формирует поклассовые AP и анализ матриц ошибок. ONNX-часть выполняет экспорт, проверку графа, оптимизацию и подготовку артефактов под CPU-среду выполнения. Поддержка ONNX-экспорта важна, поскольку Ultralytics рассматривает режим экспорта как основной путь переноса обученной модели в форматы развертывания, включая ONNX [32, 33].



**Рисунок 4.1:** Конвейер обучения YOLO для детектора текстурированных ROI: конвертация и аудит датасетов, обучение, анализ AP и матриц ошибок, экспорт в ONNX, оптимизация среды выполнения и проверка артефакта для развертывания.

Рисунок 4.1 важен для границы ответственности: подготовка данных, обучение и deployment-ветка отделены друг от друга. ONNX-артефакт появляется только после проверки обученной модели, поэтому он не подменяет валидационные отчеты и не смешивается с исходными runs/.

Конвейер обучения не является частью критического пути IDet и не выполняется при обработке кадра. Его результат — проверенный набор обучающих данных, отчетов и артефактов для развертывания, которые затем передаются в контур C++/ONNX Runtime. Благодаря этому IDet остается CPU-ориентированной библиотекой без зависимости от Python-фреймворка обучения.

### 4.3 Исходные датасеты и единая таксономия

Использовались два исходных датасета: DeepFashion2 и Fashionpedia. DeepFashion2 содержит изображения одежды с аннотациями для detection, segmentation, landmarks и retrieval [38]. Fashionpedia построена вокруг fashion ontology и содержит категории одежды, части одежды и fine-grained attributes [39]. Оба датасета были переведены в YOLO-формат, после чего исходные категории были приведены к общей 7-классовой таксономии.

Целевая таксономия включает классы:

- 1) top\_sleeved;
- 2) outerwear;
- 3) top\_sleeveless;
- 4) shorts;
- 5) trousers;
- 6) skirt;
- 7) dress.

Смысл этой таксономии не в сохранении полной семантической детализации fashion-датасетов. Для кодировщика важнее другой вопрос: есть ли в области визуально значимая фактура, которую нежелательно разрушать агрессивным квантованием. Поэтому близкие ка-

тегории объединяются до уровня, достаточного для ROI-политики. Это снижает несовместимость DeepFashion2 и Fashionpedia, уменьшает число редких классов и делает выход модели стабильнее для последующего решения о приоритете.

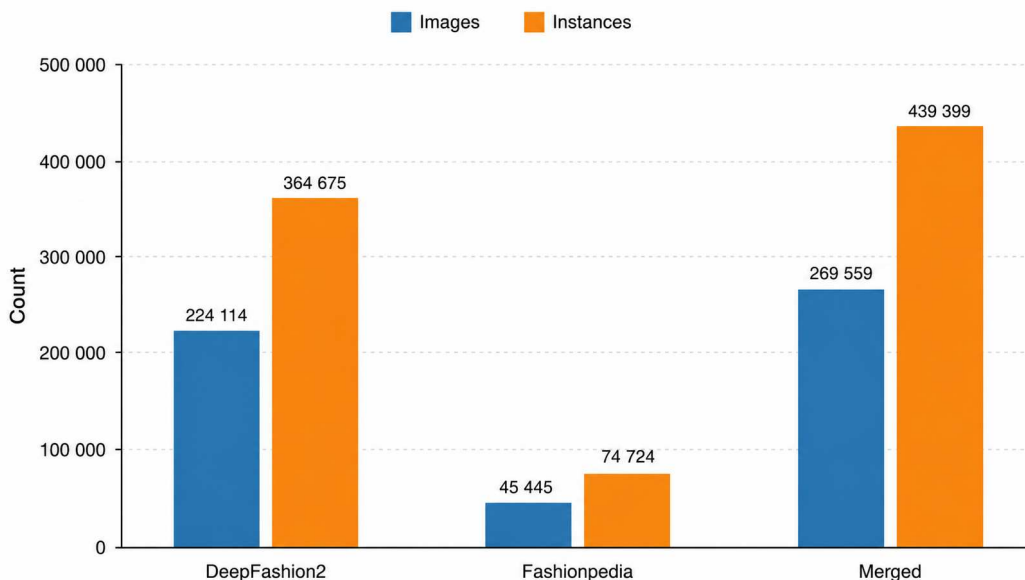
#### 4.4 Аудит датасетов после конвертации

В таблице 4.1 приведена сводка по трем конфигурациям: DeepFashion2, Fashionpedia и объединенному набору. Во всех случаях отсутствуют пустые разметочные записи, отсутствующие метки, «осиротевшие» метки и нечитаемые изображения. Это важно: качество модели начинается не с архитектуры, а с корректности подготовленного корпуса.

**Таблица 4.1:** Сводная статистика датасетов после конвертации в YOLO-формат

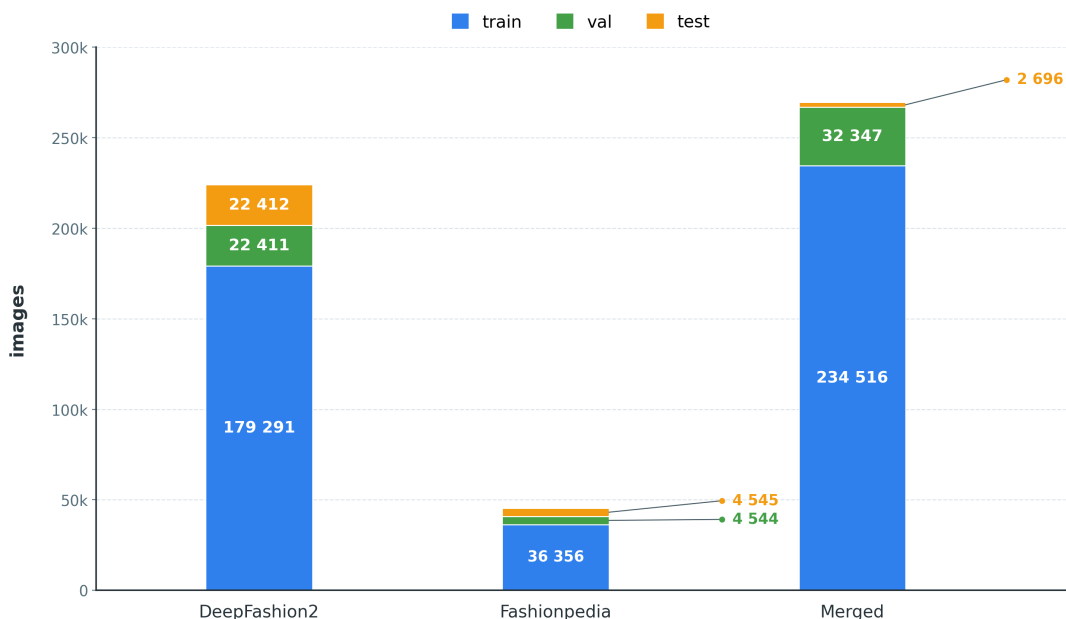
| Датасет      | Изображения | Объекты | Train  | Val   | Test  |
|--------------|-------------|---------|--------|-------|-------|
| DeepFashion2 | 224114      | 364675  | 179291 | 22411 | 22412 |
| Fashionpedia | 45445       | 74724   | 36356  | 4544  | 4545  |
| DF2+FP       | 269559      | 439399  | 234516 | 32347 | 2696  |

Аудит выполняет две функции. Во-первых, он подтверждает воспроизводимость training pipeline: каждый split содержит изображения, разметку и активные классы. Во-вторых, он задает границы интерпретации результатов. Если после конвертации меняются пропорции train/val/test или исчезает часть объектов, итоговая mAP уже не отражает качество исходного корпуса. Поэтому до обучения важно проверять не только количество изображений, но и число instances, распределение классов, геометрию bbox и наличие некорректных строк разметки.



**Рисунок 4.2:** Сравнение масштаба датасетов после приведения к общей таксономии: DeepFashion2 задает основной объем корпуса, Fashionpedia расширяет разнообразие, а DF2+FP используется как итоговый объединенный набор.

Рисунок 4.2 показывает практический смысл объединения датасетов: корпус растет с 224114 до 269559 изображений и с 364675 до 439399 объектов. Для обучения textured ROI важен не только прирост размера, но и большее разнообразие примеров одежды.



**Рисунок 4.3:** Распределение изображений по train/val/test-разбиениям для DeepFashion2, Fashionpedia и DF2+FP: график проверяет, что расширение корпуса не нарушает воспроизводимую структуру обучения и валидации.

На рисунке 4.3 видно, что train-разбиение доминирует во всех вариантах, а у объединенного набора test-часть существенно меньше train/val. Поэтому итоговые выводы по модели нужно читать вместе с указанием конкретного split, на котором считались метрики.

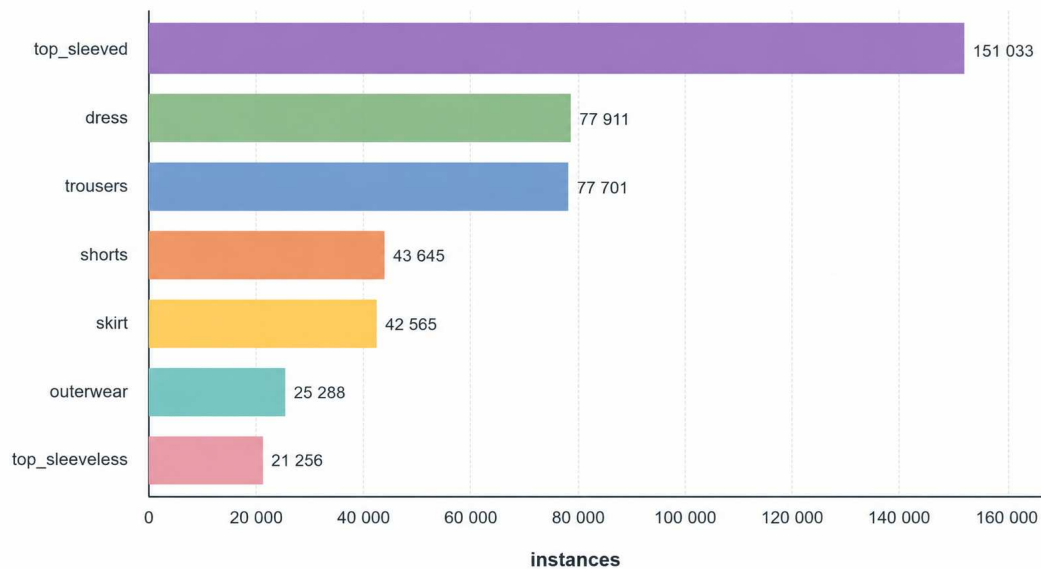
## 4.5 Дисбаланс классов

Класс `top_sleeved` доминирует во всех конфигурациях. В DeepFashion2 он составляет 126230 объектов, в Fashionpedia — 24803, а в объединенном наборе — 151033. При этом в Fashionpedia класс `top_sleeveless` содержит только 741 объект, что создает сильный imbalance. После объединения редкие классы получают больше примеров, а отношение крупнейшего класса к минимальному становится мягче.

**Таблица 4.2:** Распределение объектов по классам в объединенном датасете

| Класс                       | Train  | Val   | Test | Total  |
|-----------------------------|--------|-------|------|--------|
| <code>top_sleeved</code>    | 132382 | 17116 | 1535 | 151033 |
| <code>dress</code>          | 67265  | 9833  | 813  | 77911  |
| <code>trousers</code>       | 67226  | 9631  | 844  | 77701  |
| <code>shorts</code>         | 36958  | 6203  | 484  | 43645  |
| <code>skirt</code>          | 35909  | 6163  | 493  | 42565  |
| <code>outerwear</code>      | 20407  | 4556  | 325  | 25288  |
| <code>top_sleeveless</code> | 17033  | 3925  | 298  | 21256  |

Дисбаланс влияет не только на итоговую mAP, но и на поведение модели в streaming-сценарии. Частые классы формируют большую часть градиента при обучении, поэтому модель может быть устойчивой к типичным верхним предметам одежды и менее уверенной на редких силуэтах. Для ROI-задачи это означает риск систематического недо выделения некоторых визуально значимых областей. Поэтому в работе отдельно анализируются per-class AP и confusion matrix.



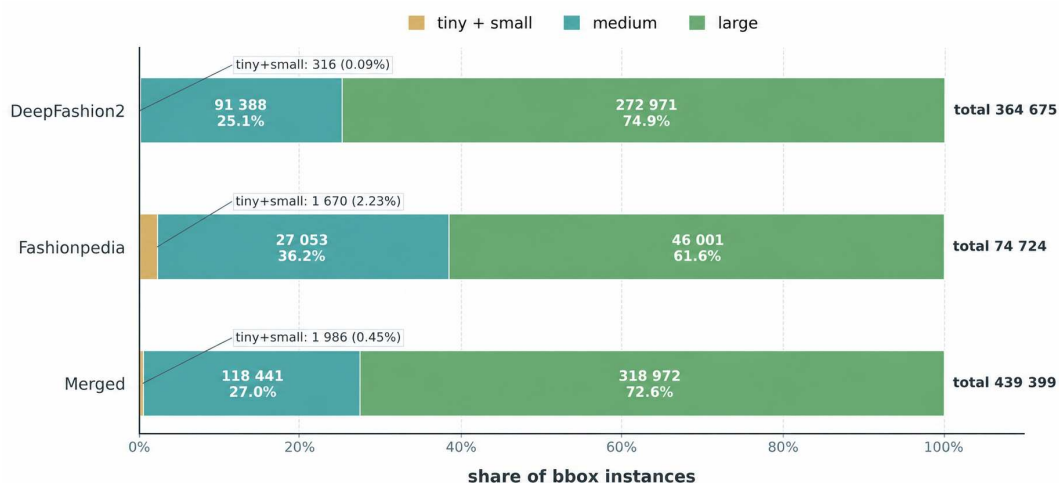
**Рисунок 4.4:** Распределение объектов по классам в объединенном датасете: график показывает остаточный дисбаланс 7-классовой таксономии и объясняет необходимость поклассовой оценки AP.

Рисунок 4.4 уточняет масштаб дисбаланса: класс `top_sleeved` примерно в семь раз больше `top_sleeveless`. Поэтому поклассовые AP и confusion matrix важнее одной агрегированной mAP.

## 4.6 Геометрия изображений и bbox

Fashion-домены имеют выраженную вертикальную ориентацию. Для DeepFashion2 средний размер изображения составляет примерно  $602.5 \times 756.8$  на обучающем разбиении, для Fashionpedia —  $755.3 \times 986.1$ , для объединенного набора —  $628.3 \times 795.7$ . Центры bbox устойчиво находятся около центра кадра: для объединенного обучающего разбиения средний центр bbox равен (0.5008, 0.5237), а средняя площадь bbox — 0.2709.

Это свойство имеет два следствия. С одной стороны, модель может переобучиться на центральное расположение объектов. Поэтому при обучении применялись геометрические аугментации: перенос, масштабирование, отражение и mosaic-подобные преобразования [35]. С другой стороны, центральность bbox частично соответствует реальному стриминговому контенту с человеком в кадре: одежда ведущего часто действительно находится в центральной области, а значит текстурированные ROI можно стабилизировать во времени.



**Рисунок 4.5:** Распределение bbox по размерным группам после конвертации в YOLO-формат: преобладание medium/large областей упрощает поиск одежды, но усиливает требование к ограничению площади ROI.

Рисунок 4.5 показывает преобладание medium/large bbox во всех датасетных вариантах. Это упрощает использование одежды как ROI-сигнала, но одновременно требует area budget: без ограничения площади крупные области одежды могут вытеснить текст и лица.

## 4.7 Режимы обучения

В работе рассматривались следующие режимы:

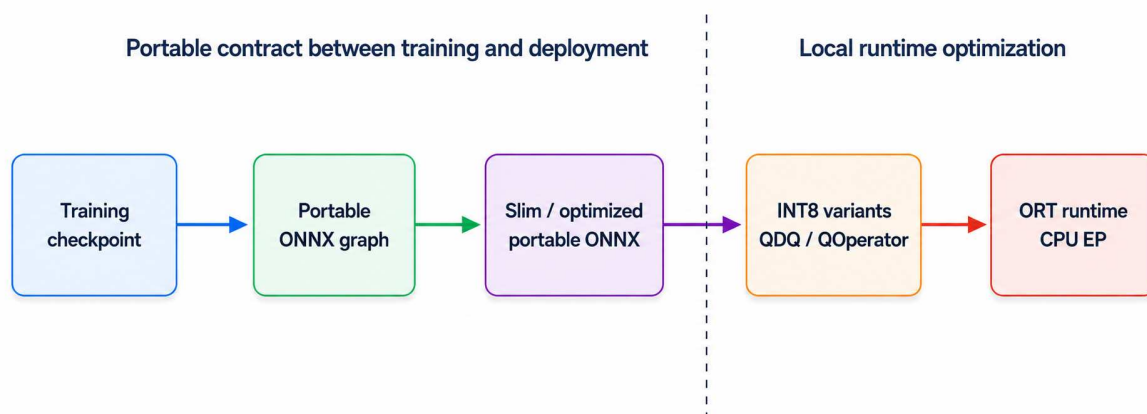
- 1) обучение только на DeepFashion2-7c;
- 2) обучение только на Fashionpedia-7c;
- 3) transfer learning: DeepFashion2 → Fashionpedia;
- 4) обучение на объединенном наборе DeepFashion2 и Fashionpedia.

Эти режимы разделяют несколько возможных источников качества. Обучение на одном датасете показывает, насколько модель зависит от домена исходной разметки. Transfer learning проверяет, можно ли использовать один корпус как предварительную специализацию перед дообучением на другом. Объединенный набор проверяет более практичную гипотезу: для ROI-задачи полезнее иметь больше разнообразных примеров одежды и фактур, чем сохранять жесткую привязку к одному источнику аннотаций.

На этом этапе важно подчеркнуть границу главы: здесь описываются данные, конфигурации и воспроизводимый конвейер. Численное сравнение обученных моделей, выбор YOLO26n и анализ качества вынесены в следующую главу, чтобы не смешивать подготовку данных с экспериментальными выводами.

## 4.8 Контракт PT-to-ONNX артефакта

Отдельным этапом конвейера является экспорт обученной PT-модели в ONNX. Этот этап нужен для замыкания цикла между обучением и IDet: библиотека среды выполнения не должна зависеть от Python-фреймворка обучения, поэтому в нее передается стабильный ONNX-артефакт с известной формой входа, набором выходов и настройками предобработки.



**Рисунок 4.6:** Путь подготовки артефакта для развертывания: от обученной контрольной точки к переносимому ONNX и исполнению через ONNX Runtime CPU Execution Provider.

Рисунок 4.6 показывает две обязательные проверки между обученной PT-точкой и CPU-инференсом: сначала фиксируется переносимый ONNX-контракт, затем выполняется локальная ORT-оптимизация под конкретную среду выполнения.

**Таблица 4.3:** Контракт PT-to-ONNX артефакта для детектора текстурированных ROI

| Элемент контракта      | Требование                                                                                                                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Входной артефакт       | Лучшая обучающая контрольная точка, полученная после обучения YOLO-модели на согласованной таксономии.                                                                                          |
| Экспортируемый формат  | ONNX-артефакт модели, совместимый с ONNX Runtime CPU Execution Provider.                                                                                                                        |
| Размер входа           | Фиксированная или явно задокументированная динамическая форма входа; размер должен совпадать с предобработкой в IDet.                                                                           |
| Параметры экспорта     | <code>format=onnx</code> , выбранный <code>opset</code> , режим <code>dynamic</code> , <code>simplify</code> , размер <code>batch</code> и флаг встроенного NMS фиксируются в отчете конвейера. |
| Проверка корректности  | Сравнение выходов PT и ONNX на калибровочных изображениях: форма, диапазоны уверенности, число кандидатов и близость координат.                                                                 |
| Проверка переносимости | Переносимый ONNX не должен содержать целевые узлы оптимизации раскладки ONNX Runtime, например <code>com.microsoft.nchwc::Conv</code> .                                                         |
| Оптимизация            | Сохранение версии до оптимизации и версии после оптимизации графа ORT для воспроизводимых измерений.                                                                                            |
| Метаданные             | Версия конвейера, YAML-описание датасета, имена классов, хэш весов и параметры предобработки.                                                                                                   |
| Интеграция             | Артефакт регистрируется как модель адаптера текстур и используется IDet без обращения к Python.                                                                                                 |

## 4.9 ONNX Runtime и квантизация

После обучения модель должна пройти отдельную проверку как артефакт для развертывания. Для этого недостаточно сохранить лучшую контрольную точку: необходимо



убедиться, что экспортированный ONNX-граф совместим с ONNX Runtime CPU Execution Provider, не содержит непереносимых внутренних операторов раскладки и сохраняет ожидаемые формы выходов. ONNX Runtime поддерживает несколько классов оптимизации графа и механизмы квантизации, включая QDQ и QOperator-представления [28, 31]. Однако наличие INT8-артефакта само по себе не гарантирует ускорение: итоговая скорость зависит от ядер вычисления, структуры графа, накладных расходов деквантизации и целевой архитектуры CPU.

**Таблица 4.4:** Классификация ONNX/ORT-артефактов в конвейере

| Артефакт               | Роль                                              | Основная проверка                                                           |
|------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| Raw ONNX               | Базовый экспорт из обучающего фреймворка.         | Формы входов/выходов, opset, отсутствие ошибок загрузки.                    |
| Slimmed ONNX           | Упрощенный переносимый граф.                      | Сохранение численной близости и структуры выходов.                          |
| FP32 ONNX              | Базовая модель для развёртывания.                 | р99 задержка, размер модели, число операторов.                              |
| QDQ INT8 ONNX          | Квантизованный граф с Quantize/DeQuantize узлами. | Потеря качества, накладные расходы деквантизации, CPU-ядра вычисления.      |
| QOperator INT8 ONNX    | Граф с квантизованными операторами.               | Поддержка операторов CPU EP и отсутствие перехода на медленные пути.        |
| ORT-optimized artifact | Локально оптимизированный артефакт.               | Привязка к среде выполнения/CPU; недопустимость как переносимого контракта. |

Особое внимание нужно уделять операциям домена `com.microsoft.nchwc`. Они могут появляться в результате внутренних оптимизаций ONNX Runtime для конкретной машины. Такой граф полезен как целевой оптимизированный артефакт, но не должен подменять переносимый ONNX-контракт между конвейером обучения и IDet.

## 4.10 Выводы по главе

Глава фиксирует воспроизводимый путь подготовки textured ROI детектора: исходные датасеты приводятся к единой 7-классовой таксономии, проверяются на пропуски и некорректную разметку, после чего используются для обучения YOLO. Локальные summary показывают, что после конвертации нет пустых label-файлов, orphan-labels и некорректных строк разметки.

Главное ограничение данных — дисбаланс классов и преобладание крупных bbox. Для ROI-задачи это не делает данные непригодными, но задает условия интерпретации: нужна поклассовая проверка, а крупные textured ROI должны проходить через ограничение площади. Экспорт в ONNX рассматривается как отдельный deployment-контракт между YOLO-pipeline и IDet, а не как часть обучения.

## Глава 5. Экспериментальные результаты YOLO-моделей на объединенной таксономии

### 5.1 Граница экспериментальных выводов

В этой главе приведены результаты обучения и валидации YOLO-моделей как детекторов текстурированных ROI. Эти результаты подтверждают качество детекции объектов на подготовленной 7-классовой таксономии, но сами по себе не доказывают рост качества видеокодирования. Для таких выводов нужны отдельные CPU-only прогоны ONNX Runtime с p50/p90/p95/p99 задержкой и отдельный эксперимент кодирования при одинаковом битрейте.

Поэтому численные выводы этой главы следует читать как характеристику ML-компонента. Валидация YOLO отвечает на вопрос, способен ли детектор находить области одежды и фактуры на выбранной таксономии. Она не отвечает на вопросы о стоимости letterbox-преобразования в C++, задержке NMS на CPU, поведении ONNX Runtime после квантизации и том, как кодировщик перераспределит биты по ROI.

Такое разграничение делает структуру экспериментов последовательной. Сначала проверяется, что текстурированный ROI-детектор достаточно надежен как модель компьютерного зрения. Затем оценивается стоимость его CPU-исполнения в составе IDet. Только после этого результат детектора используется как управляющий сигнал для ROI-aware кодирования. Без этого разделения высокий mAP можно было бы ошибочно трактовать как прямое доказательство выигрыша видеокачества.

### 5.2 Аппаратная конфигурация

Обучение выполнялось на рабочей станции с NVIDIA RTX 4090, Intel Core i9-12900F и 64 GB RAM. Такая конфигурация позволила запускать обучение YOLO26n/YOLO26s с тяжелыми аугментациями и большим merged dataset. RTX 4090 имеет 24 GB видеопамяти, что делает ее практичной для обучения современных моделей компьютерного зрения [40].

Эта конфигурация относится именно к обучению и валидации YOLO-моделей. CPU-only измерения IDet выполнялись отдельно, уже после экспорта моделей в ONNX и переноса в C++/ONNX Runtime контур. Такое разделение важно: GPU-станция проверяет качество ML-компонента, а серверные CPU-прогоны показывают стоимость будущего runtime-исполнения.

YOLO26n выбран как основной кандидат не потому, что он максимизирует все метрики, а потому что дает достаточное качество при меньшей вычислительной стоимости. Для CPU-ориентированной библиотеки этот критерий важнее небольшого прироста mAP у более тяжелой модели.

**Таблица 5.1:** Аппаратная конфигурация обучения

| Компонент                  | Значение                                 |
|----------------------------|------------------------------------------|
| GPU                        | NVIDIA RTX 4090, 24 GB VRAM              |
| CPU                        | Intel Core i9-12900F, 16 ядер, 24 потока |
| RAM                        | 64 GB DDR5                               |
| Модели                     | YOLO26n, YOLO26s                         |
| Основной кандидат для IDet | YOLO26n                                  |

### 5.3 Итоговые метрики YOLO26n

Итоговые метрики YOLO26n на объединенной 7-классовой таксономии приведены в таблице 5.2. По результатам валидационного отчета модель достигла  $mAP@0.5 = 0.938$  и  $mAP@0.5:0.95 = 0.818$  при  $precision = 0.903$  и  $recall = 0.873$ . Для легкой модели это сильный результат, особенно с учетом дальнейшей CPU-интеграции. При этом в доступных отчетах нет подтвержденного значения оптимального F1-порога, поэтому F1 не используется как отдельный численный вывод.

**Таблица 5.2:** Итоговые метрики YOLO26n на объединенной 7-классовой таксономии

| Метрика   | Значение | Метрика        | Значение |
|-----------|----------|----------------|----------|
| Precision | 0.903    | Recall         | 0.873    |
| $mAP@0.5$ | 0.938    | $mAP@0.5:0.95$ | 0.818    |

Сочетание высокой  $AP@0.5$  и заметно более строгой  $mAP@0.5:0.95$  показывает, что модель уверенно находит объекты одежды, но качество точной геометрии bbox остается отдельным фактором риска для ROI-кодирования.

### 5.4 Поклассовая точность детектора

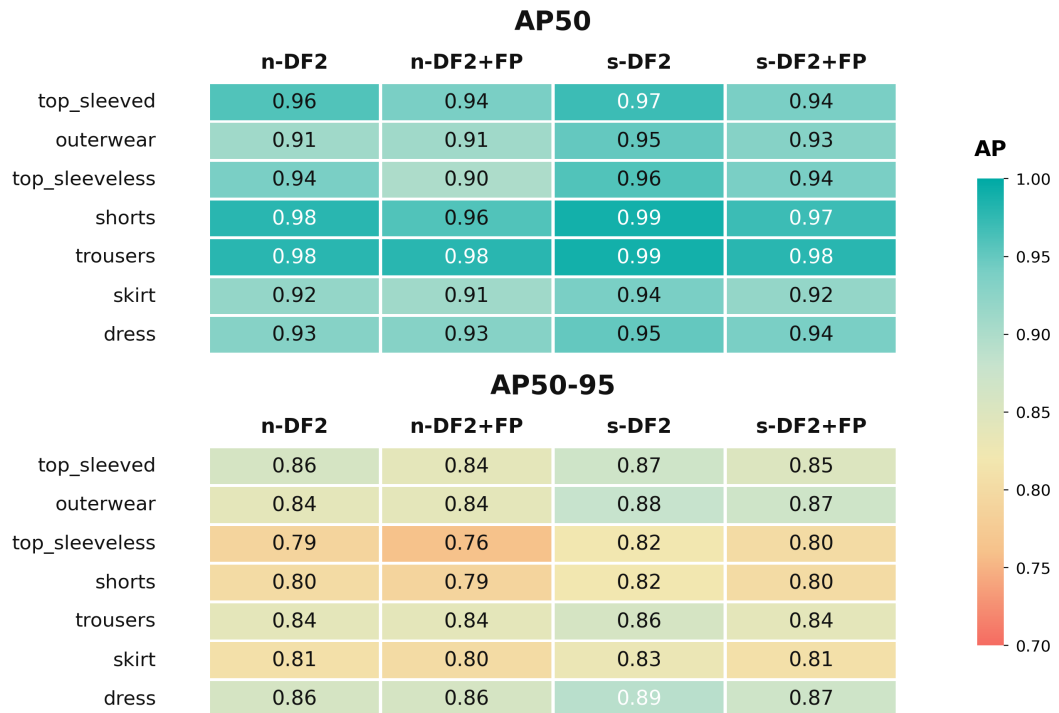
Поклассовая AP показывает, что модель наиболее уверенно распознает `trousers`, `dress`, `top_sleeved`, `shorts` и `skirt`. Более сложными являются `top_sleeveless` и `outerwear`. Это ожидаемо: `top_sleeveless` менее представлен и визуально близок к другим верхним классам, а `outerwear` часто пересекается с `top_sleeved` и `dress`.

Для ROI-задачи особенно важно сравнивать  $AP@0.5$  и  $AP@0.5:0.95$  вместе. Если  $AP@0.5$  высокий, а строгая AP заметно ниже, модель в целом находит объект, но геометрия bbox менее точна. В обычной классификации одежды это может быть вторичным, но для кодировщика геометрия напрямую задает защищаемую площадь. Поэтому классы с низкой строгой AP требуют downstream-ограничений: `clipping`, `area cap` или более консервативный `qoffset`.

На рисунке 5.1 приведено поклассовое сравнение метрик AP50 и AP50-95 для четырех обучающих конфигураций YOLO26. В отличие от диагональных значений *confusion matrix*, данные метрики учитывают не только правильность выбора класса, но и качество ло-

кализации bbox. AP50 отражает качество детекции при пороге IoU = 0.5, тогда как AP50-95 является более строгой метрикой, усредненной по нескольким порогам IoU от 0.5 до 0.95.

Такое сравнение позволяет оценить, какие классы остаются устойчивыми при смене модели и датасета, а какие сильнее теряют качество при более строгой оценке локализации.



**Рисунок 5.1:** Поклассовые значения AP50 и AP50-95 для YOLO26n/YOLO26s на DeepFashion2-7c и DF2+FP-7c: сравнение показывает, какие классы устойчивы к смене датасета и где строгая локализация bbox остается слабым местом.

Рисунок 5.1 показывает, что во всех конфигурациях значения AP50 заметно выше, чем AP50-95. Для textured ROI это существенный результат: модель часто находит нужный объект, но строгая геометрия остается отдельным риском для кодировщика.

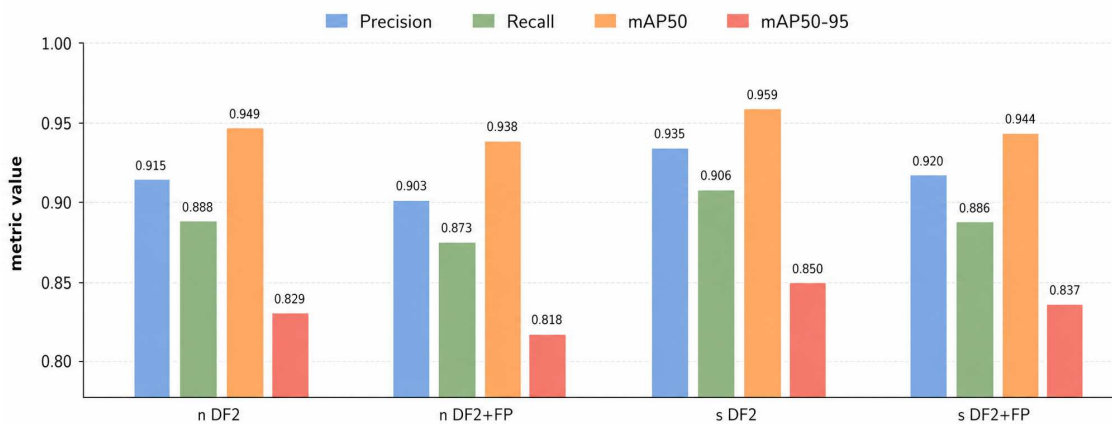
Наиболее устойчивыми классами остаются top\_sleeved, trousers и dress: они сохраняют высокие значения AP в разных конфигурациях. Более сложными оказываются top\_sleeveless, shorts и часть случаев skirt, где снижение AP50-95 указывает на более чувствительную локализацию bbox.

Сравнение YOLO26n и YOLO26s показывает, что более тяжелая модель в целом дает более высокое качество, особенно по строгой метрике AP50-95. Однако этот прирост необходимо сопоставлять с вычислительной стоимостью: YOLO26s имеет больше параметров и GFLOPs, поэтому для CPU-first сценария YOLO26n может оставаться более предпочтительным компромиссом. Объединенный набор DeepFashion2+Fashionpedia делает проверку модели более строгой за счет большего разнообразия данных, поэтому небольшое снижение отдельных поклассовых AP не противоречит его практической ценности для textured ROI.

## 5.5 Сравнение YOLO26n и YOLO26s

Сравнение YOLO26n и YOLO26s показывает ожидаемый компромисс между качеством и вычислительной стоимостью. YOLO26s имеет около 9.47 млн параметров и 20.5 GFLOPs против 2.38 млн параметров и 5.2 GFLOPs у YOLO26n, то есть примерно четырехкратную вычислительную стоимость по валидационным отчетам. На DeepFashion2 это дает рост  $mAP@0.5:0.95$  с 0.829 до 0.850. На объединенном наборе сохраненные отчеты показывают 0.818 для YOLO26n и 0.837 для YOLO26s.

Это сравнение необходимо трактовать осторожно, потому что число валидационных изображений и объектов в отдельных сохраненных отчетах различается. Тем не менее общий вывод устойчив: более тяжелая модель дает умеренный прирост качества, но хуже соответствует CPU-ориентированной роли IDet. Для ВКР важнее не абсолютный максимум  $mAP$ , а возможность встроить детектор в конвейер реального времени с жестким бюджетом на всю предобработку и постобработку.



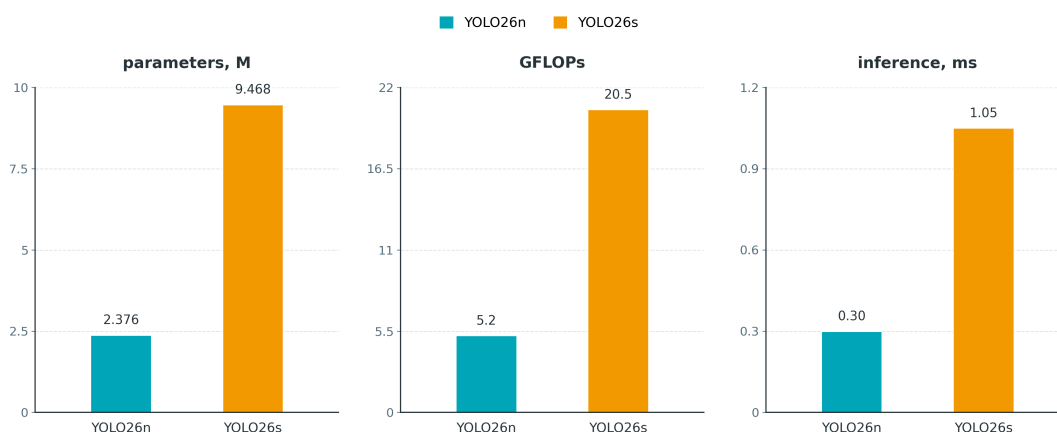
**Рисунок 5.2:** Сравнение precision, recall,  $mAP@0.5$  и  $mAP@0.5:0.95$  для YOLO26n/YOLO26s: график отделяет общий прирост качества от более строгой оценки локализации bbox.

Рисунок 5.2 показывает умеренное преимущество YOLO26s по строгой  $mAP$ , но этот прирост не меняет порядок величины качества. Для CPU-first интеграции его недостаточно, чтобы автоматически выбрать более тяжелую модель.

На рисунке 5.3 видно, что вычислительная цена YOLO26s примерно в четыре раза выше по параметрам и GFLOPs. Поэтому даже небольшой прирост  $mAP$  нужно проверять через p99 полного IDet-конвейера, а не только через валидационный отчет Ultralytics.

## 5.6 Матрица ошибок

Нормализованная матрица ошибок показывает, что на диагонали модель имеет сильные значения: 0.92 для `top_sleeved`, 0.95 для `trousers`, 0.90 для `shorts` и 0.87 для `dress`. Более низкие значения у `top_sleeveless` (0.80) и `outerwear` (0.83) объясняются визуальной близостью классов и дисбалансом данных. Заметная часть ошибок связана не с заменой одного класса одежды другим, а с фоном: это означает, что часть одежды может не получить текстурированную ROI, если пороги и политика площади выбраны слишком агрессивно.



**Рисунок 5.3:** Сравнение Params и GFLOPs для YOLO26n и YOLO26s: вычислительная цена более точной модели сопоставляется с CPU-бюджетом IDet.

|                | n-DF2 | n-DF2+FP | s-DF2 | s-DF2+FP | AP   |
|----------------|-------|----------|-------|----------|------|
| top_sleeved    | 0.94  | 0.92     | 0.95  | 0.92     | 1.00 |
| outerwear      | 0.83  | 0.83     | 0.88  | 0.86     | 0.95 |
| top_sleeveless | 0.85  | 0.80     | 0.88  | 0.85     | 0.90 |
| shorts         | 0.92  | 0.90     | 0.93  | 0.91     | 0.85 |
| trousers       | 0.95  | 0.95     | 0.96  | 0.94     | 0.80 |
| skirt          | 0.87  | 0.86     | 0.90  | 0.87     | 0.75 |
| dress          | 0.87  | 0.87     | 0.89  | 0.88     | 0.70 |

**Рисунок 5.4:** Сравнение диагональных значений нормализованных матриц ошибок для четырех YOLO-экспериментов: диагональ показывает устойчивость классификации, но для ROI также важны переходы объектов в background.

Рисунок 5.4 показывает, что наиболее устойчивы trousers, top\_sleeved и dress; слабее выглядят top\_sleeveless и outerwear. Эти классы должны оставаться в фокусе при выборе порогов уверенности и area policy.

Для задачи формирования ROI-кандидатов часть ошибок между близкими классами одежды менее критична, чем полный пропуск области. Если модель перепутала юбку и платье, но корректно выделила область одежды, кодировщик все равно получает полезный сигнал. Более опасны ошибки другого типа: одежда как фон или фон как текстурированная ROI. Поэтому матрица ошибок в данной работе интерпретируется не как тонкая классификация модных категорий, а как диагностика отделения переднего плана от фона и покрытия ROI.

## 5.7 Сводка артефактов для развертывания

Помимо обучающих сводок, важен срез качества, ориентированный на развертывание. В опубликованной сводке артефактов для YOLO26 фиксируется входной размер 640, динамический пространственный вход, ONNX Runtime CPU Execution Provider и разные

FP32/INT8-представления. Такая сводка показывает не только mAP, точность и полноту, но и размер модели для развертывания, поведение квантизованных вариантов и пригодность артефакта для CPU-среды выполнения.

**Таблица 5.3:** FP32-артефакты YOLO26 для CPU-среды выполнения

| Модель  | Датасет                   | mAP50-95 | mAP50  | Precision | Recall | Mem, MB |
|---------|---------------------------|----------|--------|-----------|--------|---------|
| YOLO26n | DeepFashion2              | 0.8131   | 0.9343 | 0.8974    | 0.8767 | 9.3     |
| YOLO26n | DeepFashion2+Fashionpedia | 0.8202   | 0.9395 | 0.9319    | 0.8621 | 9.3     |
| YOLO26s | DeepFashion2              | 0.8364   | 0.9490 | 0.9167    | 0.8940 | 36.4    |
| YOLO26s | DeepFashion2+Fashionpedia | 0.8161   | 0.9225 | 0.8982    | 0.8620 | 36.4    |

**Таблица 5.4:** INT8-варианты YOLO26 для CPU-среды выполнения

| Модель  | Датасет                   | mAP50-95      | mAP50         | Precision     | Recall        | Mem, MB |
|---------|---------------------------|---------------|---------------|---------------|---------------|---------|
| YOLO26n | DeepFashion2              | 0.7466–0.7492 | 0.8848–0.8863 | 0.8211–0.8261 | 0.9038–0.9058 | 2.7–2.9 |
| YOLO26n | DeepFashion2+Fashionpedia | 0.7421–0.7437 | 0.8723–0.8736 | 0.9373–0.9404 | 0.7978–0.7984 | 2.7–2.9 |
| YOLO26s | DeepFashion2              | 0.8158–0.8167 | 0.9403–0.9414 | 0.9203–0.9256 | 0.8817–0.8826 | 9.6–9.8 |
| YOLO26s | DeepFashion2+Fashionpedia | 0.7707–0.7740 | 0.8968–0.9000 | 0.8980–0.9018 | 0.8465–0.8492 | 9.6–9.8 |

Из таблицы видно, что квантизация резко уменьшает размер артефакта: для YOLO26n примерно с 9.3 MB до 2.7–2.9 MB, для YOLO26s — с 36.4 MB до 9.6–9.8 MB. Однако это сопровождается заметной потерей mAP50-95, особенно для объединенного набора. Поэтому INT8-варианты в данной работе рассматриваются как гипотезы развертывания, требующие отдельной проверки задержки и качества, а не как автоматически лучший выбор.

## 5.8 Почему выбран YOLO26n

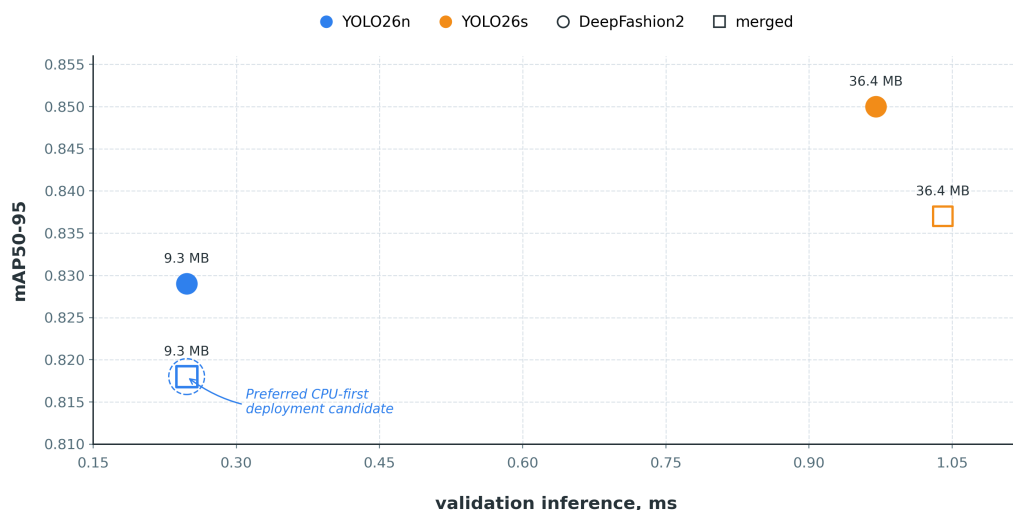
Финальное инженерное решение — выбрать YOLO26n как основной кандидат для текстурированных ROI в IDet. YOLO26s может давать более высокую точность в части экспериментов, но требует около 20.5 GFLOPs против 5.2 GFLOPs у YOLO26n. Цель ВКР — не построить максимально точный fashion-классификатор, а получить модель, которую реалистично встроить в CPU-ориентированную IDet и проверить в режиме с ограниченным бюджетом на всю предобработку и постобработку.

Этот выбор также подтверждается сводкой артефактов для развертывания. При одинаковом входном размере и CPU Execution Provider FP32-артефакт YOLO26n для объединенного набора занимает 9.3 MB и показывает mAP50-95 = 0.8202, тогда как FP32-артефакт YOLO26s занимает 36.4 MB и в этой сводке не дает устойчивого преимущества на объединенном наборе. Это не отменяет того, что в отдельных обучающих сводках YOLO26s может быть точнее; это показывает другое: после перевода в контракт развертывания легкая модель выглядит более подходящей для CPU-ориентированной среды выполнения.

Рисунок 5.5 суммирует deployment-компромисс: YOLO26n сохраняет качество, близкое к YOLO26s, но имеет существенно меньший размер и расчетную стоимость.

## 5.9 Выводы по главе

YOLO26n на объединенной таксономии достигает precision 0.903, recall 0.873, mAP@0.5 0.938 и mAP@0.5:0.95 0.818. Эти значения достаточны, чтобы рассматривать



**Рисунок 5.5:** Компромисс между качеством детектора и вычислительной стоимостью по валидационным отчетам.

модель как кандидат для textured ROI, но не являются доказательством улучшения видео: они проверяют только ML-компонент.

Сравнение с YOLO26s показывает последовательный компромисс. Более тяжелая модель может быть точнее в обучающих отчетах, но стоит примерно в четыре раза дороже по параметрам и GFLOPs. Поэтому для CPU-ориентированной IDet выбран YOLO26n: он дает близкое качество, меньший deployment-артефакт и более реалистичную основу для дальнейших p99-измерений.



## Глава 6. Интеграция ROI в видеокодирование

### 6.1 ROI как управляющий сигнал

В данной работе ROI-детектор не является финальным продуктом сам по себе. Его выход — это управляющий сигнал для кодировщика. Поэтому система должна оцениваться не только по *detection metrics*, но и по тому, как ROI может быть использован для перераспределения качества при фиксированном битрейте.

Пусть  $V$  — исходное видео,  $D$  — ROI-детектор,  $S$  — политика приоритизации и преобразования ROI в управляющую информацию для кодировщика, а  $E$  — кодировщик. Тогда полный конвейер можно записать как

$$\hat{V} = E(V, S(D(V))), \quad (6.1)$$

где  $\hat{V}$  — закодированный и восстановленный видеопоток. В этой постановке детектор  $D$  не оптимизируется изолированно: он должен быть полезен внутри кодировщика  $E$ .

Такое разделение важно для корректной интерпретации результатов. Высокая mAP детектора текстурированных ROI означает, что модель хорошо находит объекты одежды на валидационном разбиении. Но итоговая польза для видеостриминга возникает только тогда, когда эти области корректно преобразуются в `qoffset`/метаданные ROI и действительно влияют на распределение битового бюджета.

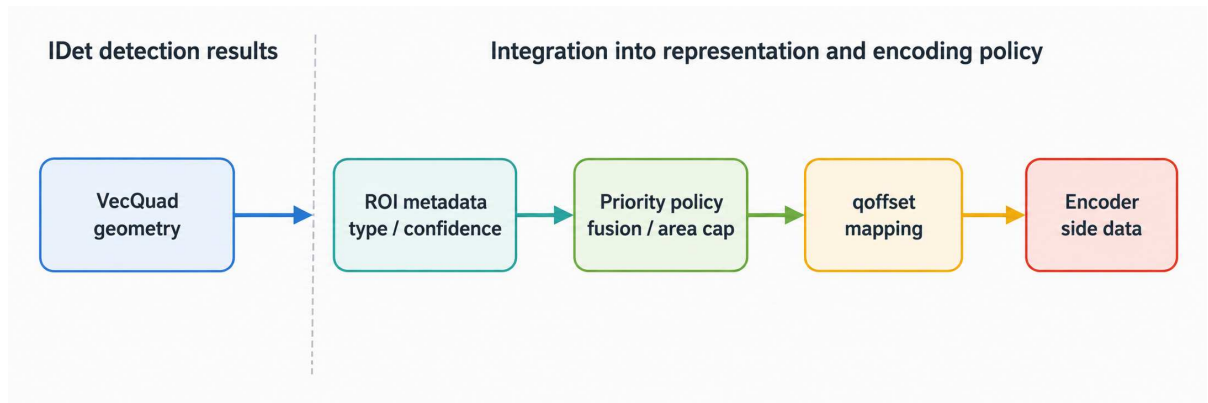
Формально выход детектора можно рассматривать как набор кандидатов  $d_i = (G_i, t_i, c_i)$ , где  $G_i$  — геометрия области,  $t_i$  — тип ROI, а  $c_i$  — уверенность детектора. Для IDet геометрия часто задается четырехугольником  $G_i = \{(x_{ik}, y_{ik})\}_{k=1}^4$ . Кодировщику при этом передается не произвольный многоугольник, а прямоугольная область:

$$R_i = [\min_k x_{ik}, \max_k x_{ik}] \times [\min_k y_{ik}, \max_k y_{ik}]. \quad (6.2)$$

Политика  $S$  преобразует  $(R_i, t_i, c_i)$  в пару  $(R_i, q_i)$ , где  $q_i$  — управляющее смещение квантования. Поэтому ошибка детектора влияет на кодирование двумя путями: пропущенная ROI не получает защиты, а чрезмерно широкая ROI тратит битовый бюджет на лишнюю площадь.

### 6.2 Адаптер кодировщика как граница между IDet и кодировщиком

Интеграционный слой должен быть отделен от детекторов. Модель не должна знать, работает ли кодировщик через метаданные ROI FFmpeg, адаптивное квантование или резервную предобработку фона. FFmpeg `addroi` прикрепляет к кадру метаданные об областях интереса, не изменяя пиксели самого кадра, а `AVRegionOfInterest` задает координаты и `qoffset` для таких областей [15, 16]. Это делает адаптер кодировщика естественной границей между компьютерным зрением и видеокодированием.



**Рисунок 6.1:** Преобразование метаданных ROI в управляющую информацию для кодировщика: ограничение площади, сортировка, отображение в `qoffset` и передача ROI side data.

Рисунок 6.1 показывает, что адаптер кодировщика должен получать уже отсортированные и ограниченные по площади ROI. Иначе `qoffset` будет применяться к слишком большим или конфликтующим областям, что ухудшит фон и израсходует битовый бюджет.

**Таблица 6.1:** Алгоритмический контракт адаптера кодировщика

| Операция                 | Семантика                                                          |
|--------------------------|--------------------------------------------------------------------|
| <code>supportsROI</code> | Проверка поддержки ROI side data или карты <code>qoffset</code> .  |
| <code>mapPriority</code> | Преобразование приоритета ROI в управляющую подсказку кодировщику. |
| <code>applyHints</code>  | Передача ROI после ограничения площади и сортировки.               |
| <code>fallback</code>    | Резервная стратегия при отсутствии нативной ROI-поддержки.         |

Этот адаптер позволяет сохранить модульность: IDet отвечает за поиск областей, ROI-слой — за слияние и стабилизацию, адаптер кодировщика — за конкретный способ влияния на кодирование.

### 6.3 Политика `qoffset`

Для каждой ROI задается приоритет  $p_i$ . Сильнее всего защищается текст, затем лица, затем текстурированные ROI. Практическая политика может быть задана таблицей 6.2.

Эти значения не являются универсальными константами. Они задают начальную инженерную политику для экспериментов. В реальной системе `qoffset` следует подбирать на валидационном видеокорпусе с учетом битрейта, кодека, типа контента и допустимого ухудшения фоновых областей.

В общем виде отображение приоритета в смещение квантования можно записать как

$$q_i = \text{clip}(-\beta p_i, q_{\min}, 0), \quad p_i = f(t_i, c_i, A_i), \quad (6.3)$$

где  $p_i \in [0, 1]$  — итоговый приоритет ROI,  $A_i = |R_i|/|F|$  — доля площади кадра, а  $\beta$  задает силу защиты. Верхняя граница равна нулю, потому что положительный `qoffset` ухудшал бы ROI. Нижняя граница  $q_{\min}$  нужна для контроля: слишком сильное отрицательное смеще-

**Таблица 6.2:** Пример политики приоритизации ROI

| Тип ROI              | Приоритет | qoffset | Причина                                           |
|----------------------|-----------|---------|---------------------------------------------------|
| Текстовая ROI        | 1.00      | -0.20   | Читаемость текста критична для пользователя.      |
| ROI лица             | 0.85      | -0.15   | Искажения лица заметны субъективно.               |
| Текстурированная ROI | 0.55      | -0.08   | Фактура важна, но не должна вытеснять текст/лицо. |
| Фон                  | 0.00      | 0.00    | Фон кодируется без дополнительной защиты.         |

ние повышает локальное качество ROI ценой резкого ухудшения фона и нарушает сопоставимость потока с baseline. Именно поэтому политика `qoffset` должна рассматриваться вместе с ограничением площади, а не как независимая ручная настройка.

## 6.4 Ограничение площади и разрешение конфликтов

При интеграции с кодировщиком необходимо учитывать не только тип ROI, но и суммарную защищаемую площадь. Если система защищает слишком большую часть кадра, ROI-aware подход теряет смысл: кодировщик фактически получает команду повысить качество почти всего изображения. Поэтому перед передачей ROI в адаптер кодировщика применяется ограничение площади:

$$A_{ROI} = \frac{|\bigcup_i R_i|}{|F|} \leq \alpha. \quad (6.4)$$

Если ROI пересекаются, приоритет должен разрешаться детерминированно. Например, область текста внутри лица или одежды должна сохранять текстовый приоритет. Лицо должно иметь приоритет над текстурированной ROI. Текстурированная ROI может быть обрезана или ослаблена, если она занимает слишком большую площадь или конфликтует с более важными областями. Такой подход предотвращает ситуацию, когда крупная область одежды вытесняет текст или лицо из битового бюджета.

Практически это удобно реализовывать как жадную процедуру. Сначала ROI сортируются по типу, уверенности и ожидаемой пользовательской значимости. Затем области добавляются в защищаемое множество до достижения `area cap`; пересечения с более приоритетными ROI вычитаются или получают более слабый `qoffset`. Такой порядок делает результат предсказуемым: один и тот же набор детекций приводит к одному и тому же набору подсказок кодировщику, а конфликт между крупной областью одежды и небольшим текстом решается в пользу текста.

## 6.5 Резервные режимы

Поддержка метаданных ROI различается между кодировщиками и сборками FFmpeg/x265 [15, 17]. Поэтому архитектура должна предусматривать резервные режимы:

- 1) использование нативных метаданных ROI, если кодировщик поддерживает `AVRegionOfInterest`;
- 2) настройка `adaptive quantization` и `rate-control` параметров;
- 3) слабая предобработка фона вне ROI для снижения его локальной сложности;
- 4) режим `offline-анализа`, где ROI используются только для оценки качества и построения отчетов.

Наличие резервных режимов важно для переносимости. В идеальном варианте IDet формирует метаданные ROI, а кодировщик напрямую применяет `qoffset` к соответствующим областям. Однако в реальной инфраструктуре могут отличаться версии FFmpeg, параметры сборки, доступные кодеки и способ передачи `side data` между фильтрами. Поэтому ROI-aware система не должна зависеть от единственного механизма кодировщика.

Резервные режимы не эквивалентны нативному ROI-кодированию, но они сохраняют архитектурный контракт: IDet продолжает выдавать геометрию и приоритеты, а downstream-слой выбирает доступный способ их использования. Это важно для `production-сценария`, где один и тот же анализ кадра может применяться в разных окружениях: в одном случае через `addroi`, в другом — только для оценки качества или для мягкой предобработки фона.

## 6.6 Связь ROI-интеграции с дальнейшим экспериментом

Глава описывает архитектурный путь от ROI-геометрии к управляющей информации для кодировщика. Она подводит к количественной оценке качества: следующий эксперимент сравнивает `baseline` и ROI-aware ветки при одинаковом исходном материале, одном кодировщике, фиксированном `preset` и сопоставимом размере выходного потока.

Таким образом, интеграция ROI в видеокодирование состоит из трех независимых частей: извлечения геометрии через IDet, формирования последующих метаданных ROI с приоритетами и ограничением площади, а также преобразования этих метаданных в конкретный механизм кодировщика. Именно это разделение позволяет сохранять IDet как универсальный CPU-ориентированный слой, а адаптер кодировщика — как заменяемую часть под конкретный видеокодек и окружение.

Такое разделение также определяет дизайн эксперимента. Чтобы проверить именно вклад ROI `side data`, дальше фиксируются кодек, `preset`, `baseline CRF` и режим сопоставимого размера потока. Тогда разница между ветками объясняется не сменой кодировщика или увеличением файла, а тем, что одинаковый битовый бюджет был распределен между ROI и фоном по-разному.

## 6.7 Выводы по главе

Глава отделяет геометрию ROI от действий кодировщика. IDet формирует области, последующий ROI-слой добавляет тип, уверенность, приоритет и ограничение площади, а адаптер кодировщика переводит это в `qoffset`, `side data` или резервную политику.

Ключевой практический риск — не отсутствие ROI, а чрезмерная или нестабильная ROI-политика. Если защищаемая площадь слишком велика, кодировщик фактически теряет свободу перераспределения битов. Если области скачут между кадрами, появляется мерцание качества. Поэтому `area cap`, разрешение конфликтов и `temporal smoothing` являются обязательными частями интеграции.

В рамках работы ROI-интеграция рассматривается как продуктовый механизм повышения качества: IDet находит значимые области, а адаптер кодировщика должен направить в них дополнительный битовый бюджет. Численная проверка этого механизма приведена в следующей главе на покадровом HEVC-эксперименте с `Ffmpeg addroi` и `libx265`.

## Глава 7. Покадровое сравнение baseline и ROI-aware кодирования

### 7.1 Цель и границы эксперимента

Финальная проверка ROI-aware подхода выполнена как покадровый эксперимент: каждое изображение кодировалось отдельно как one-frame HEVC stream. Такая постановка изолирует пространственный эффект ROI side data и показывает, перераспределяет ли кодировщик качество в пользу областей интереса при сопоставимом размере выходного потока.

Эксперимент не является полноценной end-to-end оценкой видеостриминга. Он не моделирует temporal smoothing, tracking ROI между кадрами, GOP-структуру, межкадровое предсказание, сетевую доставку и задержки реального live-конвейера. Такое сравнение выходит за рамки работы: оно требует отдельного видеокорпуса, пространственно-временного сглаживания ROI, политики межкадровой стабилизации, настройки GOP и существенных вычислительных затрат на повторяемую оценку. Поэтому результаты ниже подтверждают локальный эффект ROI-aware кодирования на отдельных FHD-кадрах, но не подменяют следующий видеотест.

Для каждого режима был подготовлен отдельный FHD-набор изображений: для Text ROI использовано 3000 кадров, для Face ROI — 3000 кадров, для Cloth ROI — 2696 кадров. Такой объем использован для агрегированной инженерной оценки трех независимых сценариев ROI: текстовых областей, лиц и текстурированных областей одежды; статистическая репрезентативность всех возможных видеосценариев при этом не утверждается. Численные результаты ниже приведены как агрегированные improvements относительно baseline при сопоставимом размере выходного потока.

### 7.2 Данные и подготовка FHD-наборов

Для проверки использовались три независимых набора изображений, соответствующие режимам IDet. Text ROI проверялся на test-варианте TextOCR [36]: этот набор содержит естественные сцены с текстом и подходит для оценки читаемости. Face ROI проверялся на test-варианте WIDER FACE [37], где присутствуют лица разных масштабов и условий съемки. Cloth ROI проверялся на test split набора `df2-fp-7c`, построенного на основе DeepFashion2 и Fashionpedia и приведенного к единой 7-классовой таксономии. В экспериментальной части режим textured ROI обозначается как Cloth ROI, поскольку он реализован через детектор областей одежды.

Разметка исходных датасетов не использовалась как oracle-разметка качества кодирования. ROI формировались непосредственно детекторами IDet, а test split служил независимым набором изображений, не совпадающим с обучающими данными соответствующих моделей. Все изображения были подготовлены единым конвейером: рекурсивный сбор из подпапок, приведение к FHD, формирование плоской структуры набора и сохранение слу-

жебного манифеста. Обработанные кадры имеют размер  $1920 \times 1080$ ; технический механизм четного сгор нужен только как страховка для yuv420p/HEVC.

### 7.3 Протокол кодирования

Baseline-ветка кодировалась через libx265 без ROI-метаданных. ROI-aware ветка использует выход IDet: детектор возвращает четырехугольники в порядке  $TL \rightarrow TR \rightarrow BR \rightarrow BL$ , после чего они преобразуются в axis-aligned bounding rectangle для фильтра Ffmpeg addroi. Фильтр передает ROI side data в libx265; отрицательный qoffset означает более мягкое квантование внутри ROI.

Оба one-frame HEVC потока декодировались обратно в PNG. Метрики считались между парами original vs baseline\_decoded и original vs roi\_aware\_decoded. Основной режим сравнения — match\_baseline: ROI-aware ветка подбирает CRF так, чтобы размер ROI-aware потока был близок к baseline. Поэтому зафиксированный выигрыш не объясняется увеличением файла, а отражает перераспределение битового бюджета внутри режима match\_baseline.

**Таблица 7.1:** Параметры покадрового HEVC-эксперимента

| Режим | Baseline CRF | Preset | Size mode      | qoffset | Интерпретация                              |
|-------|--------------|--------|----------------|---------|--------------------------------------------|
| Text  | 32           | medium | match_baseline | -1/4    | Наиболее сильная защита читаемых областей. |
| Face  | 32           | medium | match_baseline | -1/5    | Защита лиц при умеренном смещении битов.   |
| Cloth | 32           | medium | match_baseline | -1/10   | Более мягкая защита фактур и одежды.       |

Меньший CRF соответствует более высокому качеству и большему размеру, больший CRF — более сильному сжатию. Preset фиксировался как medium, чтобы не смешивать эффект ROI с изменением эффективности кодирования. Таблица 7.1 также показывает, что сила ROI-защиты намеренно различается по режимам. Текст получает самый сильный отрицательный qoffset, потому что читаемость чувствительна к локальным артефактам. Для лиц смещение сделано умеренным: лицо субъективно важно, но слишком агрессивная защита может заметно ухудшить фон при том же размере потока. Для Cloth ROI выбран наиболее мягкий qoffset; это консервативная настройка, которая проверяет сам механизм перераспределения качества без попытки получить максимальный локальный выигрыш ценой нестабильного фона.

**Таблица 7.2:** Сопоставимость размеров baseline и ROI-aware потоков

| Режим | Средний<br>size ratio | Медианный<br>size ratio | Диапазон<br>size ratio | Baseline,<br>КБ | ROI,<br>КБ |
|-------|-----------------------|-------------------------|------------------------|-----------------|------------|
| Text  | 1.0036                | 1.0000                  | 0.9625–1.0375          | 60.2            | 60.3       |
| Face  | 1.0058                | 1.0000                  | 0.9566–1.0383          | 90.2            | 90.8       |
| Cloth | 1.0029                | 1.0025                  | 0.9595–1.0424          | 35.8            | 35.8       |

Средний `size_ratio` находится около 1.0 во всех трех режимах: 1.0036 для Text, 1.0058 для Face и 1.0029 для Cloth. Следовательно, сравнение выполнено при сопоставимом битовом бюджете. Разброс отдельных изображений сохраняется, но медианные значения равны или близки к 1.0, что соответствует цели режима `match_baseline`. Поэтому таблица 7.2 служит контрольной проверкой эксперимента: ROI-aware ветка не получает преимущества за счет большего файла. Любой положительный эффект в ROI нужно трактовать именно как перераспределение битов внутри почти того же размера, а не как результат ослабления сжатия всего изображения.

## 7.4 Выбранные detector-конфигурации

ROI для эксперимента формировались выбранными рабочими точками IDet. Эти профили получены grid-search перебором detector-конфигураций с одновременным учетом p99-задержки и качества покрытия областей по площади.

**Таблица 7.3:** Выбранные рабочие точки IDet для ROI-aware эксперимента

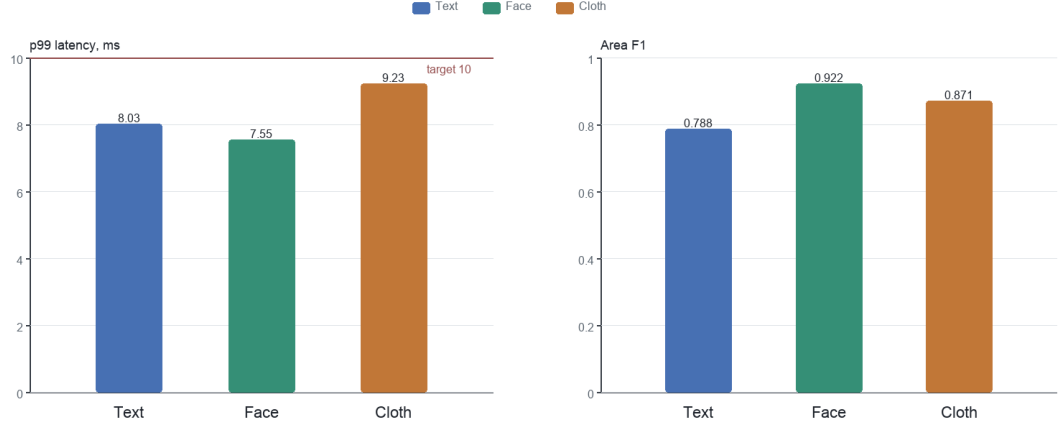
| Режим | Плитки | Вход      | Intra<br>threads | OMP<br>threads | p99, мс | Area Recall | Area<br>Precision | Area F1 | Extra area |
|-------|--------|-----------|------------------|----------------|---------|-------------|-------------------|---------|------------|
| Text  | 5 × 6  | 64 × 128  | 2                | 30             | 8.032   | 0.886       | 0.709             | 0.788   | 0.363      |
| Face  | 3 × 4  | 128 × 192 | 8                | 12             | 7.553   | 0.916       | 0.929             | 0.922   | 0.070      |
| Cloth | 2 × 4  | 128 × 128 | 8                | 8              | 9.230   | 0.920       | 0.827             | 0.871   | 0.192      |

Таблица 7.3 описывает не абстрактные модели, а конкретные runtime-профили, которыми формировались ROI для последующего кодирования. Поля «Плитки» и «Вход» задают способ разбиения FHD-кадра и размер входа детектора; Intra и OMP фиксируют потоковую схему; p99 показывает хвостовую задержку. Последние четыре колонки нужны для оценки качества ROI-геометрии: `recall` отвечает за полноту покрытия важных областей, `precision` — за аккуратность защищаемой площади, F1 объединяет эти две стороны, а Extra показывает лишнюю площадь относительно reference-геометрии. Важно подчеркнуть, что эти area-based показатели не являются сравнением с ручной oracle-разметкой исходных датасетов: reference-геометрия задается выбранным эталонным прогоном и используется как инженерный проху для сравнения быстрых runtime-конфигураций между собой.

Для Text ROI выбран профиль, который жертвует частью `precision` ради высокого `recall`: для текста пропуск хуже небольшого расширения области, потому что пропущенная надпись вообще не получает защиту при кодировании. Face ROI имеет наиболее ровный про-



фильм: Area F1 0.922 достигается при  $p99 = 7.553$  ms, поэтому лица защищаются точнее и без выхода за целевой бюджет задержки. Cloth ROI работает ближе к верхней границе 10 ms, но сохраняет высокий recall; умеренная лишняя площадь здесь допустима, поскольку фактуры одежды часто имеют сложные границы и не сводятся к компактным текстовым или лицевым областям.



**Рисунок 7.1:** p99-задержка и Area F1 для выбранных detector-конфигураций IDet: график фиксирует рабочие точки, где качество покрытия ROI сопоставляется с ограничением на хвостовую задержку.

Рисунок 7.1 показывает, что все выбранные профили остаются ниже порога 10 ms по p99, но отличаются характером компромисса: Face имеет лучший Area F1, Text жертвует precision ради recall, а Cloth удерживает наибольшую задержку из трех режимов.

## 7.5 Метрики и знак improvement

Обозначим исходный кадр через  $x$ , декодированный baseline — через  $b$ , а декодированный ROI-aware результат — через  $r$ . Пусть  $\Omega$  — множество пикселей кадра,  $M$  — ROI mask,  $B = \Omega \setminus M$  — фон вне ROI,  $C$  — число цветовых каналов, а  $L$  — максимальное значение канала. Для любой области  $D \subseteq \Omega$  среднеквадратичная ошибка и PSNR определяются как

$$\text{MSE}_D(y, z) = \frac{1}{|D|C} \sum_{p \in D} \sum_{c=1}^C (y_{p,c} - z_{p,c})^2, \quad (7.1)$$

$$\text{PSNR}_D(y, z) = 10 \log_{10} \frac{L^2}{\text{MSE}_D(y, z)}. \quad (7.2)$$

Индекс  $D$  показывает, где считается метрика: для global используется весь кадр  $D = \Omega$ , для ROI-only — маска  $D = M$ , для background — область  $D = B$ .

SSIM оценивает не только среднюю ошибку пикселей, но и согласованность яркости, контраста и структуры. В стандартной форме локальная SSIM-компонента для окна около пикселя записывается как

$$\text{SSIM}(u, v) = \frac{(2\mu_u\mu_v + C_1)(2\sigma_{uv} + C_2)}{(\mu_u^2 + \mu_v^2 + C_1)(\sigma_u^2 + \sigma_v^2 + C_2)}, \quad (7.3)$$

где  $\mu$  обозначает локальное среднее,  $\sigma^2$  — локальную дисперсию, а  $\sigma_{uv}$  — ковариацию двух изображений. Для области  $D$  берется среднее значение локальных SSIM-компонент по пикселям этой области. VMAF,  $Q_{JOD}$ , LPIPS и DISTS используются как full-reference perceptual metrics, то есть сравнивают восстановленное изображение с исходным эталоном [5, 7, 8, 9].  $Q_{JOD}$  рассматривается как относительная perceptual metric при одинаковой нормализации входных изображений для baseline и ROI-aware ветвей.

В работе используются пять групп метрик. Global metrics измеряют качество всего кадра: PSNR, SSIM, VMAF,  $Q_{JOD}$ , LPIPS и DISTS [3, 4]. ROI-only метрики считаются только внутри ROI mask, background metrics — только вне ROI. Thesis-weighted метрики задают компромиссную оценку, где ROI получает больший вес, чем ее доля в площади кадра:

$$Q_{tw}(x, y) = \lambda Q_M(x, y) + (1 - \lambda) Q_B(x, y), \quad \lambda > \frac{|M|}{|\Omega|}. \quad (7.4)$$

Такая форма не утверждает, что фон не важен; она отражает прикладную постановку, где артефакты в ROI заметнее пользователю, чем равная по площади ошибка в фоне.

ROI-crop-weighted метрики считаются по кропам отдельных ROI и агрегируются с учетом площади кропа:

$$Q_{crop}(x, y) = \frac{\sum_{i=1}^K |R_i| Q_{R_i}(x, y)}{\sum_{i=1}^K |R_i|}, \quad (7.5)$$

где  $R_i$  — прямоугольный ROI-кроп, а  $K$  — число валидных ROI на кадре. Если для изображения нет валидной ROI, оно не входит в  $n$  для ROI-only и ROI-crop метрик; поэтому размер выборки для этих метрик может отличаться от числа подготовленных FHD-кадров.

Для PSNR, SSIM, VMAF и  $Q_{JOD}$  большее значение означает лучшее качество. Для LPIPS и DISTS меньшее значение означает лучшее качество. Поэтому improvement нормализован так, чтобы положительный знак всегда означал преимущество ROI-aware над baseline:

$$\Delta_m^{(j)} = \begin{cases} m(x_j, r_j) - m(x_j, b_j), & m \in \{\text{PSNR, SSIM, VMAF, } Q_{JOD}\}, \\ m(x_j, b_j) - m(x_j, r_j), & m \in \{\text{LPIPS, DISTS}\}. \end{cases} \quad (7.6)$$

Агрегированные значения в таблицах далее вычисляются как среднее, медиана, стандартное отклонение и доля положительных случаев:

$$\bar{\Delta}_m = \frac{1}{n} \sum_{j=1}^n \Delta_m^{(j)}, \quad (7.7)$$

$$\text{positive\_share}_m = \frac{1}{n} \sum_{j=1}^n \mathbf{1}\{\Delta_m^{(j)} > 0\}. \quad (7.8)$$

Именно поэтому отрицательная global/background метрика и положительная ROI-crop метрика могут одновременно быть корректным результатом: первая фиксирует потери вне ROI или по всему кадру, а вторая — выигрыш внутри приоритетной области.

## 7.6 Результаты ROI-aware кодирования

Положительное значение `improvement` в таблицах 7.4–7.5 означает преимущество ROI-aware над baseline. Отрицательное значение означает, что соответствующая метрика снизилась относительно baseline при сопоставимом размере потока.

**Таблица 7.4:** Средние значения `improvement` для global, background и thesis-weighted метрик

| Режим | Global PSNR | Global VMAF | Global $Q_{JOD}$ | Background PSNR | Background SSIM | Weighted PSNR | Weighted SSIM |
|-------|-------------|-------------|------------------|-----------------|-----------------|---------------|---------------|
| Text  | -0.206      | -0.305      | -0.044           | -0.382          | -0.0019         | 0.059         | -0.0002       |
| Face  | -0.115      | -0.226      | -0.021           | -0.204          | -0.0013         | 0.021         | 0.0004        |
| Cloth | -0.138      | -0.115      | -0.043           | -1.148          | -0.0085         | 0.080         | 0.0001        |

Background — область вне ROI; Weighted — thesis-weighted метрика, где ROI имеет повышенный вес. Положительное значение `improvement` означает преимущество ROI-aware над baseline.

**Таблица 7.5:** Средние значения `improvement` для ROI-only и ROI-crop-weighted метрик

| Режим | ROI PSNR | ROI SSIM | Crop PSNR | Crop VMAF | Crop $Q_{JOD}$ | Crop LPIPS | Crop DISTS |
|-------|----------|----------|-----------|-----------|----------------|------------|------------|
| Text  | 3.331    | 0.0253   | 3.028     | 4.451     | 0.166          | 0.0107     | 0.0466     |
| Face  | 3.284    | 0.0303   | 2.871     | 5.857     | 0.415          | 0.0300     | 0.0408     |
| Cloth | 0.547    | 0.0017   | 0.971     | 0.449     | 0.034          | 0.0023     | 0.0028     |

Crop обозначает ROI-crop-weighted метрики: качество считается по кропам ROI и агрегируется с учетом их площади. Для LPIPS и DISTS знак `improvement` развернут, поэтому положительное значение также означает преимущество ROI-aware.

**Таблица 7.6:** Доля положительных случаев для ключевых ROI-crop-weighted `improvement`

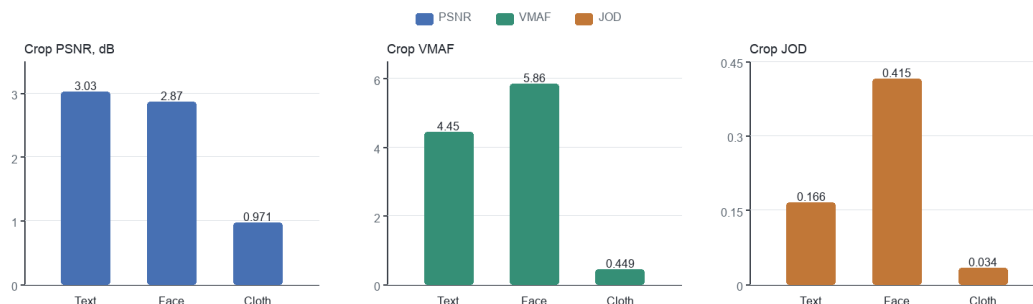
| Режим | Crop PSNR | Crop VMAF | Crop $Q_{JOD}$ | Crop LPIPS | Crop DISTS |
|-------|-----------|-----------|----------------|------------|------------|
| Text  | 1.00      | 1.00      | 1.00           | 1.00       | 1.00       |
| Face  | 1.00      | 1.00      | 1.00           | 1.00       | 1.00       |
| Cloth | 0.95      | 0.90      | 1.00           | 0.95       | 0.95       |

Positive share — доля изображений, где ROI-aware оказался лучше baseline по соответствующей ROI-crop-weighted метрике.

Таблица 7.4 показывает поведение всего кадра и фона. В ней отрицательные global/background значения не являются ошибкой: при `match_baseline` общий размер потока остается почти неизменным, поэтому усиление ROI компенсируется более жестким квантованием менее важных областей. Наиболее показателен Cloth: background PSNR снижается на 1.148 dB, но weighted PSNR остается положительным. Это означает, что перераспределение качества попадает в целевую область, хотя глобальное усреднение по кадру частично скрывает локальный выигрыш.

Таблица 7.5 фиксирует результат там, где ROI-aware кодирование должно проявляться сильнее всего: внутри масок ROI и на ROI-кропах. Text и Face дают крупный локальный прирост по PSNR и VMAF, потому что текстовые области и лица компактнее, лучше согласованы с детектором и сильнее выигрывают от дополнительного битового бюджета. Cloth

также показывает положительные значения по всем ROI-crop метрикам, но меньшей величины: это более сложная и размытая категория, где объект часто содержит неоднородные ткани, складки, границы одежды и фон. Таблица 7.6 дополняет средние значения: положительная доля близка к единице, следовательно, эффект не сводится к нескольким удачным изображениям.



**Рисунок 7.2:** ROI-crop-weighted improvement по PSNR, VMAF и  $Q_{JOD}$  для трех режимов.

Рисунок 7.2 показывает главный локальный эффект: ROI-crop метрики положительны во всех трех режимах. Самый большой ROI-crop VMAF improvement зафиксирован для Face (+5.857), затем Text (+4.451); для Cloth эффект меньше по абсолютной величине (+0.449), но остается положительным и устойчивым по доле положительных случаев.



**Рисунок 7.3:** Сопоставление ROI-only PSNR, background PSNR и thesis-weighted PSNR improvements.

Рисунок 7.3 фиксирует ожидаемый компромисс режима `match_baseline`: ROI-only PSNR растет, background PSNR снижается, а thesis-weighted PSNR остается положительным. Это означает перераспределение качества от фона к ROI при сопоставимом размере потока, а не равномерное улучшение всего кадра.

## 7.7 Почему эффект ограничен

Небольшая абсолютная величина части изменений не означает слабость эффекта. Эксперимент поставлен в строгом режиме `match_baseline`: ROI-aware ветка не получает дополнительного битового бюджета, а только перераспределяет уже имеющийся бюджет между ROI и фоном. В такой постановке крупный рост качества внутри ROI почти неиз-

бежно сопровождается снижением качества вне ROI. Поэтому ожидаемый результат — не улучшение всех метрик сразу, а положительный локальный эффект при контролируемом компромиссе по фону.

Второй фактор — выбранный baseline. В работе использован не искусственно слабый baseline, а фиксированная HEVC-конфигурация на `libx265` с `preset medium` и `CRF 32`. Такой baseline уже умеет адаптивно распределять биты по сложным участкам кадра, поэтому ROI-aware слой соревнуется не с примитивным кодированием, а с сильным кодировщиком. Это делает прирост более честным: результат показывает добавленную ценность ROI side data сверх стандартной rate-control логики HEVC.

Третий фактор — консервативная политика `qoffset`. В работе не подбирался максимально агрессивный `qoffset`, который мог бы искусственно зависить локальный эффект ценой деградации фона. Напротив, параметры выбраны так, чтобы не приводить к чрезмерной деградации фона и сохранять сопоставимый размер потока. Для Text и Face смещение сильнее, потому что эти области компактны и субъективно важны. Для Cloth используется более мягкая защита: эта настройка дает меньший численный выигрыш, но лучше соответствует производственной логике, где текстурированные ROI имеют меньший приоритет, чем текст и лица.

Разница между Cloth и режимами Text/Face объясняется также природой самих областей. Текст и лица имеют более устойчивую семантическую структуру: небольшое повышение качества в буквах, контурах лица и деталях глаз быстро отражается в ROI-crop метриках. Одежда и фактуры распределены иначе: они занимают большую и менее однородную площадь, содержат высокочастотные детали, складки, границы с фоном и участки разной важности. Кроме того, прямоугольная ROI для одежды чаще захватывает лишнюю площадь, поэтому часть выделенного бюджета уходит не только на действительно значимые фактуры. Поэтому меньший эффект Cloth является ожидаемым свойством более сложного режима, а не признаком неработоспособности подхода.

Именно поэтому основной вывод по эксперименту формулируется не как «ROI-aware улучшает весь кадр», а как «ROI-aware стабильно улучшает области интереса при сопоставимом размере потока». Метод демонстрирует управляемое перераспределение качества в пользу областей интереса на фиксированной HEVC-конфигурации и при ограниченном бюджете, а дальнейший рост эффекта может достигаться отдельным подбором `qoffset`, приоритетов ROI и видеоспецифических temporal-механизмов.

## 7.8 Интерпретация по режимам

В режиме Text ROI ветка ROI-aware кодирования дала устойчивый локальный выигрыш: ROI-only PSNR вырос на 3.331 dB, ROI-only SSIM — на 0.0253, ROI-crop VMAF — на 4.451, а доля положительных случаев по ключевым ROI-crop метрикам равна 1.00. Одновременно global VMAF снизился на 0.305, background PSNR — на 0.382 dB. Такой результат соответствует цели: текстовые области защищены лучше, а часть битового бюджета снята с фона.

Для Face ROI локальный эффект также устойчив: ROI-only PSNR вырос на 3.284 dB, ROI-only SSIM — на 0.0303, ROI-crop VMAF — на 5.857, ROI-crop  $Q_{JOD}$  — на 0.415. Глобальные метрики при этом немного снизились: global PSNR на 0.115 dB, global VMAF на 0.226. Это согласуется с режимом `match_baseline`: локальная защита лиц достигается за счет перераспределения битов, а не за счет роста общего размера потока.

Для Cloth ROI глобальные метрики слегка снижаются: global PSNR на 0.138 dB, global SSIM на 0.0008, global VMAF на 0.115, global  $Q_{JOD}$  на 0.043. При этом ROI-only и ROI-crop-weighted метрики демонстрируют положительный эффект: ROI-only PSNR +0.547 dB, ROI-crop PSNR +0.971 dB, ROI-crop VMAF +0.449, ROI-crop  $Q_{JOD}$  +0.034. Background PSNR снижается на 1.148 dB, а background SSIM на 0.0085; это является прямым следствием приоритизации областей одежды и фактур при сопоставимом размере потока.

## 7.9 Выводы по главе

Покадровая схема сравнения `baseline` и `ROI-aware` на основе `FFmpeg addroi` и `libx265` реализована и проверена для трех режимов `IDet`: `Text`, `Face` и `Cloth`. Для всех режимов использован `CRF baseline 32`, `preset medium` и режим `match_baseline`; средний `size_ratio` находится около 1.0, поэтому сравнение выполнено при сопоставимом размере потока.

Эксперимент подтверждает локальный эффект `ROI side data`: ROI-only и ROI-crop-weighted метрики улучшаются во всех трех режимах. Ухудшение `background` и части `global metrics` не противоречит цели метода, потому что при фиксированном битовом бюджете кодировщик перераспределяет качество от фона к ROI. Полученный результат подтверждает применимость `ROI side data` для приоритизации значимых областей. Полноценное сравнение на видеопотоке требует отдельного этапа с пространственно-временным сглаживанием, `tracking ROI`, `GOP`-структурой и оценкой сетевой доставки, поэтому в данной работе оно вынесено за границы эксперимента.

## Глава 8. Экспериментальное исследование производительности IDet

### 8.1 Назначение benchmark-эксперимента

Экспериментальное исследование производительности IDet направлено на проверку того, может ли ROI-детекция использоваться в CPU-first видеоконвейере без нарушения целевого бюджета задержки. В отличие от главы, посвященной обучению YOLO-модели, здесь оценивается не качество отдельного ML-checkpoint, а поведение runtime-конфигураций IDet: задержка обработки кадра, устойчивость хвоста распределения, качество геометрического покрытия ROI и стоимость разных режимов разбиения изображения на плитки.

Для оценки использовались grid-search эксперименты по трем режимам библиотеки:

- 1) **text ROI** — детекция текстовых областей;
- 2) **face ROI** — детекция лиц;
- 3) **textured ROI** — детекция одежды и фактурных областей.

Каждая конфигурация характеризуется набором runtime-параметров: размером входа модели, схемой разбиения на плитки, числом потоков ONNX Runtime, числом потоков tiled-обработки и режимом I/O Binding. Для каждой точки фиксировались значения  $p50$ ,  $p90$ ,  $p95$  и  $p99$  задержки, а также area-based проху качества относительно reference-прогона: AreaRecall, AreaPrecision, AreaF<sub>1</sub> и ExtraAreaRatio, определенные ранее в разделе о метриках покрытия ROI.

Важно подчеркнуть, что reference-прогон в данном эксперименте не является ручной ground truth-разметкой. Он используется как инженерный проху: более дорогая или более консервативная конфигурация задает ожидаемую область покрытия, а быстрые варианты сравниваются с ней по площади пересечения. Такой подход позволяет оценивать компромисс между скоростью и сохранением ROI-геометрии без необходимости вручную размечать каждый benchmark-кадр.

### 8.2 Дисциплина измерения задержки

Для корректного измерения latency важно отделять чистый benchmark от диагностических запусков. В чистом режиме отключались отрисовка, сохранение результатов и подробный вывод:

```
--is_draw 0 --is_dump 0 --verbose 0.
```

Именно этот режим использовался для измерения  $p50/p90/p95/p99$ . Отдельный короткий запуск с `--is_dump 1` применялся только для выгрузки ROI-геометрии и расчета area-based метрик. Эти два режима нельзя смешивать: dump и диагностический вывод добавляют I/O-операции, которые искажают задержку hot path.

Время внешнего Python-скрипта, запускающего grid search, не трактуется как latency IDet. Оно может сохраняться как служебное поле, но финальные выводы о скорости должны

строиться по значениям, которые печатает сам `idet_app`. Такой подход исключает влияние `subprocess`, парсинга `stdout` и файловых операций на измерение производительности `runtime`-слоя.

Для `real-time` сценария ключевым является не среднее значение, а хвост распределения. Средняя задержка может быть приемлемой, но редкие пики приводят к пропуску `deadline` кадра, накоплению очереди или необходимости отключать часть детекторов. Поэтому в данной работе основной метрикой выбора `runtime`-конфигурации является  $p99$ , а  $p50$  используется как характеристика типичного поведения.

Измерения производительности IDet выполнялись не на обучающей GPU-станции, а на сервере с двумя сокетами Kunpeng 920B, 224 GB RAM и двумя 48-ядерными ARM CPU. Итого сервер предоставляет 96 физических CPU-ядер, однако `benchmark` в этой работе намеренно не использует всю машину как единый вычислительный ресурс. Цель эксперимента — найти рабочие точки, пригодные для производственного видеоконвейера, где часть ядер должна оставаться доступной кодировщику, сетевому вводу-выводу и другим сервисным задачам.

По этой причине измерения трактуются как проверка конфигураций в заданном CPU-бюджете, а не как поиск абсолютного максимума скорости на сервере. Для двухсокетной NUMA-системы это особенно важно: выход за пределы локальной группы ядер может дать больше параллелизма, но одновременно увеличить межсокетные обращения к памяти и шум в хвосте задержки.

### 8.3 Рабочие точки IDet

По результатам `grid-search` эксперимента для дальнейшего ROI-aware кодирования выбрана одна основная рабочая точка на режим. Эти точки уже использованы в главе 7 и приведены в таблице 7.3. Здесь они рассматриваются с позиции производительности: выбор выполнялся не по одному минимальному  $p99$ , а по компромиссу между задержкой, `recall`, `precision`, `Area F1` и лишней защищаемой площадью.

В `grid-search` скрипте была задана верхняя граница в 32 потока на конфигурацию. Это не предел сервера Kunpeng 920B: на тестовой машине доступно 96 CPU-ядер. Ограничение в 32 потока выбрано как инженерный бюджет для одной ROI-ветки: оно остается внутри локального 48-ядерного сокета, снижает риск NUMA-штрафов и оставляет запас для видеокодировщика. Поэтому значения в таблице 7.3 следует читать как лучшие найденные рабочие точки внутри заданного бюджета, а не как абсолютный потолок производительности оборудования.

Выбранный Text ROI профиль использует сетку  $5 \times 6$ , вход  $64 \times 128$ , `threads_intra=2` и `tile_omp=30`. Он задействует весь заданный бюджет 32 потоков и дает  $p99 = 8.032$  ms при `Area F1 0.788`. Для текста это оправдано: `recall 0.886` важнее минимального  $p99$ , потому что пропущенный текст не получает защиты в кодировщике.

Для Face ROI используется сетка  $3 \times 4$ , вход  $128 \times 192$ , `threads_intra=8` и `tile_omp=12`. Профиль дает  $p99 = 7.553$  ms, `Area Recall 0.916`, `Area Precision 0.929` и `Area`

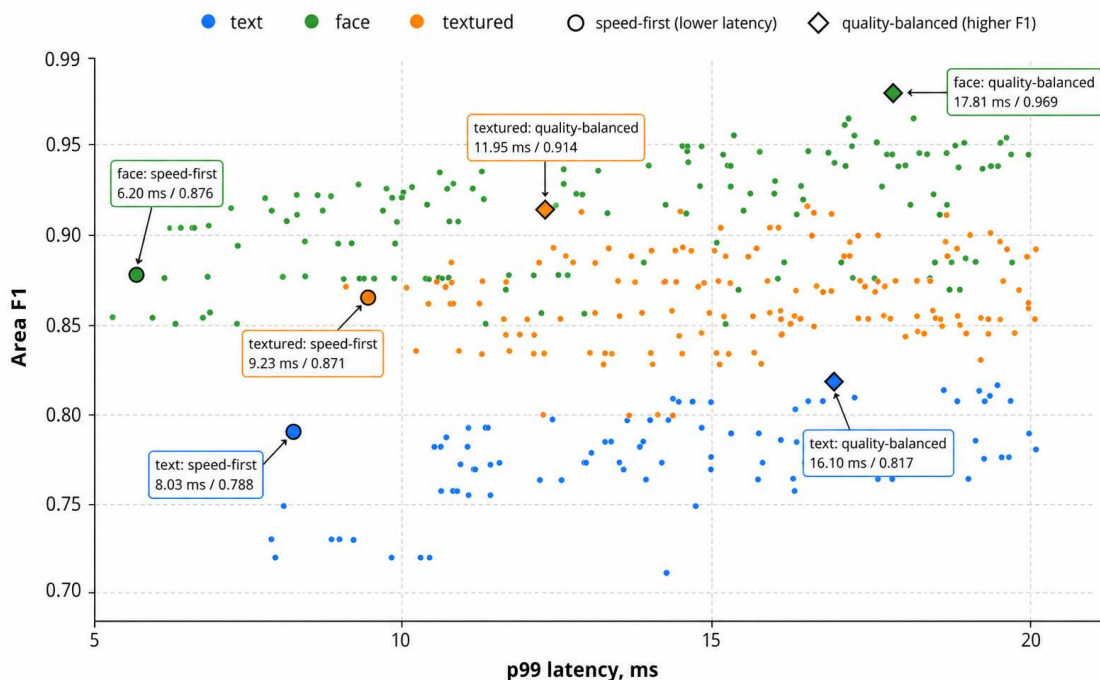


F1 0.922. Он лучше соответствует задаче качества, чем ultra-speed варианты с меньшим покрытием лиц, и остается ниже целевого бюджета 10 ms.

Для Cloth ROI используется сетка  $2 \times 4$ , вход  $128 \times 128$ , `threads_intra=8` и `tile_omp=8`. Профиль дает  $p99 = 9.230$  ms, Area Recall 0.920 и Area F1 0.871. Extra Area Ratio 0.192 является приемлемым для текстурированных областей: небольшое расширение ROI дешевле, чем пропуск одежды или фактур, которые затем не смогут получить повышенный приоритет в кодировщике.

## 8.4 Компромисс между задержкой и качеством покрытия

На рисунке 8.1 каждая точка соответствует одной конфигурации grid search. По оси задержки используется  $p99$ , поскольку именно эта метрика отражает риск нарушения deadline кадра. По оси качества используется  $\text{AreaF}_1$ , объединяющая полноту покрытия reference ROI и аккуратность candidate ROI.



**Рисунок 8.1:** Компромисс между  $p99$ -задержкой и Area F1 для text, face и textured ROI: график показывает, какие режимы укладываются в целевой latency budget и где качество покрытия требует более дорогой конфигурации.

Рисунок 8.1 показывает, что универсальной настройки IDet нет. Для Text ROI повышение покрытия достигается за счет более агрессивного tiling и большего плиточного параллелизма. Для Face ROI выбранная точка находится в balanced-области: она не минимизирует  $p99$  любой ценой, но дает более надежное покрытие лиц. Для Cloth ROI выбранный профиль находится у границы целевого бюджета 10 ms и сохраняет достаточно высокий recall.

Tiled inference может снижать задержку отдельного кадра, но эффект зависит от модели и схемы плиток. Слишком мелкое разбиение увеличивает overhead stitching и NMS, а слишком крупные плитки уменьшают выигрыш от параллелизма. Поэтому параметры `tiles_rc`, `fixed_hw` и `tile_omp` должны подбираться совместно.

## 8.5 Интерпретация по режимам

Text ROI использует наиболее высокий бюджет потоков CPU среди выбранных профилей. Это соответствует характеру задачи: текст часто занимает малую площадь, поэтому важнее не потерять область, чем добиться максимальной точности границ. Для последующего кодирования такая стратегия повышает шанс защитить все читаемые фрагменты.

Face ROI показывает самый высокий Area F1 среди выбранных точек. Это важно для субъективного качества: заметные артефакты на лице воспринимаются сильнее, чем такие же артефакты в фоне. Выбранная рабочая точка сохраняет баланс между качеством покрытия и p99, не уходя в дорогие quality-first конфигурации.

Cloth ROI работает ближе всего к верхней границе latency budget. Это ожидаемо для текстурированных областей: модель YOLO и постобработка одежды тяжелее, чем выбранная SCRFD-конфигурация для детекции лиц. Тем не менее выбранная точка укладывается в 10 ms и затем в главе 7 дает положительный ROI-срок эффект при кодировании.

## 8.6 I/O Binding и локальность памяти

I/O Binding в ONNX Runtime позволяет заранее связать входные и выходные тензоры с конкретными буферами и уменьшить лишние перемещения данных между этапами исполнения [27]. В CPU-only сценарии это особенно важно для tail latency: непредсказуемые копирования и неудачная локальность памяти могут ухудшать p95/p99 даже при приемлемом среднем времени.

Профилирование использовалось как диагностический инструмент. Сравнивались режимы с включенным и выключенным I/O Binding при зафиксированном состоянии NUMA balancing. В таблице 8.1 показано, что I/O Binding существенно повышает долю локального memory traffic. Например, для textured tiled доля локальных обращений выросла с 48.01% до 94.31%, для face tiled — с 55.49% до 94.88%, для text tiled — с 48.73% до 91.43%.

**Таблица 8.1:** Влияние I/O Binding на локальность памяти

| Режим           | Доля локального memory traffic, % |             |               | Общий memory traffic, GB |             |
|-----------------|-----------------------------------|-------------|---------------|--------------------------|-------------|
|                 | С binding                         | Без binding | Прирост, п.п. | С binding                | Без binding |
| Textured single | 77.22                             | 41.01       | 36.21         | 15.82                    | 35.75       |
| Textured tiled  | 94.31                             | 48.01       | 46.30         | 35.56                    | 45.68       |
| Face single     | 84.63                             | 56.47       | 28.16         | 9.41                     | 14.40       |
| Face tiled      | 94.88                             | 55.49       | 39.39         | 22.93                    | 20.77       |
| Text single     | 52.35                             | 44.87       | 7.48          | 31.72                    | 40.24       |
| Text tiled      | 91.43                             | 48.73       | 42.70         | 23.65                    | 25.94       |

Рисунок 8.2 подтверждает главный эффект I/O Binding: он повышает локальность обращений, но не гарантирует меньший общий объем memory traffic. Например, для tiled-режима лиц общий трафик с I/O Binding выше, чем без него. Для закрепленных CPU-измерений и NUMA-систем решающим является не сам объем трафика, а снижение зависимости от межузловых обращений, которое делает хвост задержки более управляемым.

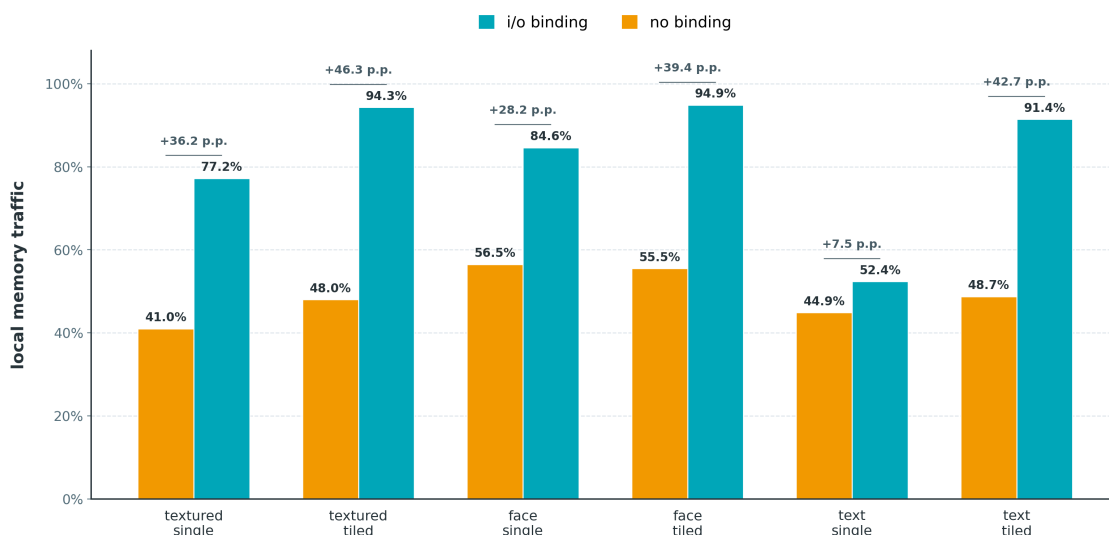


Рисунок 8.2: Доля локального memory traffic при включенном и выключенном I/O Binding.

## 8.7 NUMA-аспекты CPU-only исполнения

На серверных CPU итоговая задержка зависит не только от числа потоков, но и от размещения потоков и памяти. Automatic NUMA balancing в Linux может быть полезен для обычных приложений, но в закрепленных измерениях он способен добавлять шум из-за миграции страниц по page fault и периодического сканирования памяти [41]. Поэтому состояние `kernel.numa_balancing`, `cpuset`, `memory binding` и политика размещения потоков должны фиксироваться в протоколе эксперимента.

Для IDet это особенно важно из-за сочетания двух уровней параллелизма: ONNX Runtime использует собственные потоки `intra-op/inter-op`, а `tiled`-режим добавляет внешний параллелизм по плиткам. Если оба уровня настроены слишком агрессивно, возникает переподписка потоков: несколько пулов конкурируют за одни и те же ядра. Поэтому параметры `threads_intra`, `threads_inter` и `tile_omp` должны рассматриваться как единая политика среды выполнения.

Практический вывод состоит в том, что оптимальные настройки нельзя переносить между машинами без повторной проверки. Конфигурация, оптимальная для одного `cpuset` или NUMA-узла, может стать неоптимальной при изменении числа доступных ядер, памяти или версии ONNX Runtime. Поэтому таблицы измерений фиксируют не только задержку и качество, но и полный набор параметров потоков.

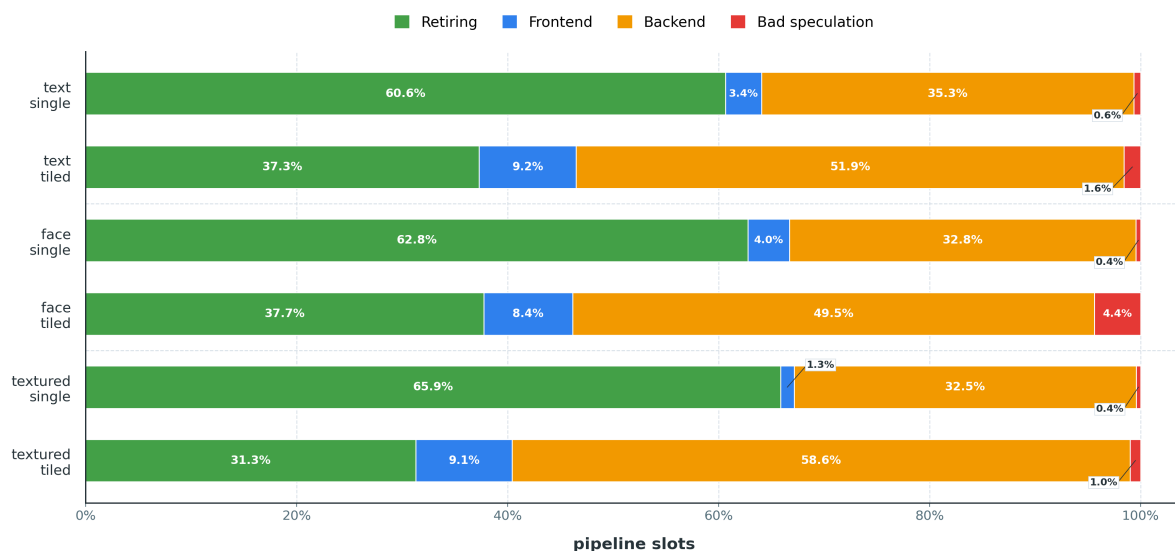
## 8.8 Top-Down Microarchitecture Analysis

Top-Down Microarchitecture Analysis позволяет классифицировать, как процессор расходует слоты конвейера исполнения [44, 45]. В этой методике:

- 1) *Retiring* соответствует полезно завершенным инструкциям;
- 2) *Frontend Bound* отражает ограничения подачи инструкций;
- 3) *Backend Bound* связан с ожиданием данных, памяти или исполнительных ресурсов;

4) *Bad Speculation* показывает потери из-за неверных предсказаний и сбросов конвейера исполнения.

На рисунке 8.3 показана реконструированная сводка для диагностических прогонов без I/O Binding. Режимы single-shot имеют более высокую долю *Retiring*: примерно 60.65% для текста, 62.77% для лиц и 65.88% для текстурированных ROI. В tiled-режиме возрастает доля *Backend Bound*: для текста — до 51.94%, для лиц — до 49.47%, для текстурированных ROI — до 58.60%.



**Рисунок 8.3:** Top-Down Microarchitecture Analysis для single/tiled режимов без I/O Binding: распределение pipeline slots помогает отделить вычислительную нагрузку от задержек памяти и фронта.

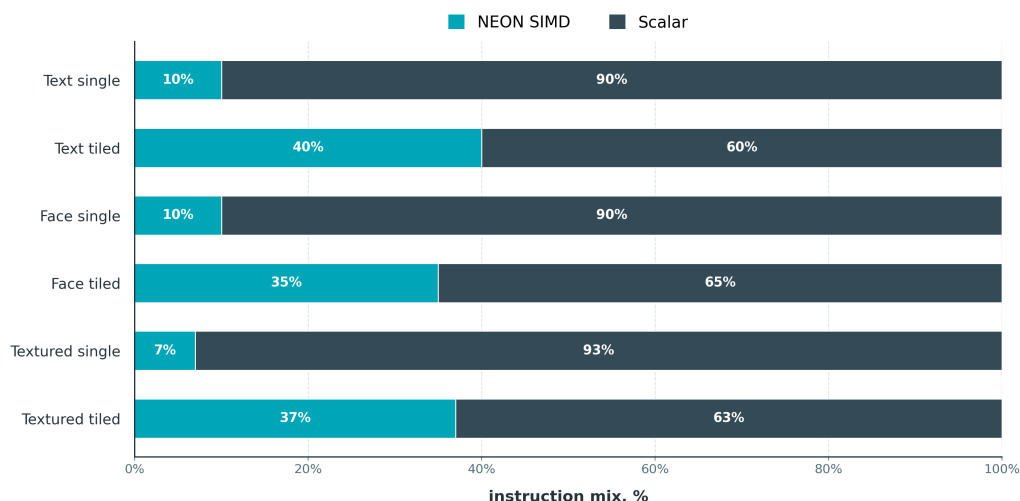
Рисунок 8.3 показывает, что tiled-режимы повышают параллелизм, но чаще становятся backend-bound. Это объясняет, почему увеличение числа плиток и потоков не всегда улучшает p99.

Такой результат согласуется с природой tiled-инференса. Разбиение на плитки повышает параллелизм и позволяет лучше использовать CPU, но одновременно увеличивает давление на память, синхронизацию, планирование среды выполнения и постобработку. Поэтому рост параллелизма не гарантирует автоматического снижения p99: конфигурация может стать backend-bound и упереться не в арифметику, а в доступ к данным.

Дополнительно tiled-режимы увеличивают долю NEON SIMD-инструкций. Для текстовых ROI она возрастает примерно с 10–14% до 40%, для ROI лиц — примерно с 9–14% до 33–34%, для текстурированных ROI — примерно с 6–8% до 36–37%. Это показывает, что tiled-инференс действительно активнее использует SIMD-вычисления, но итоговый выбор конфигурации все равно должен основываться на p99 и качестве покрытия одновременно.

Рисунок 8.4 подтверждает, что tiled inference активнее нагружает векторные вычисления, но выбирать конфигурацию только по SIMD-доле нельзя: решающими остаются p99 и качество покрытия.

Для углубленного анализа hotspot-ов может использоваться Linux perf, предоставляющий sampling и hardware performance counters [42]. На ARM-системах дополнительно может



**Рисунок 8.4:** Доля NEON SIMD-инструкций в single и tiled режимах.

применяться Statistical Profiling Extension, если она доступна в конкретной аппаратной и системной конфигурации [43]. Если профилировщик сообщает о недоступности SPE, это не является ошибкой IDet, но ограничивает глубину микроархитектурного анализа.

## 8.9 Ограничения текущих benchmark-данных

Полученные таблицы измерений подтверждают пригодность отдельных режимов IDet к развертыванию, но относятся именно к слою ROI-детекции. У этих данных есть несколько ограничений.

Во-первых, метрики покрытия по площади сравнивают ROI-кандидаты с референсным прогоном, а не с ручной разметкой видео. Поэтому они показывают согласованность быстрых конфигураций с выбранным референсом, но не измеряют абсолютную семантическую правильность ROI.

Во-вторых, измерения выполнялись на отдельных изображениях. Videopоследовательность добавляет межкадровые эффекты: размытие движения, изменение масштаба, частичное перекрытие объектов, нестабильность детектора и необходимость временного сглаживания. Поэтому результаты на изображениях должны быть дополнены прогоном на видеокорпусе.

В-третьих, задержка в этой главе относится к слою ROI-детекции. Она не включает полный видеокодировщик, расчет VMAF/SSIM, передачу метаданных ROI в бэкэнд кодировщика и возможные накладные расходы интеграционного слоя. Поэтому результаты этой главы подтверждают, что IDet формирует ROI в реалистичном CPU-бюджете, а качество после кодирования оценивается отдельно в покадровом эксперименте главы 7.

## 8.10 Выводы по главе

Экспериментальное исследование показывает, что IDet имеет практические рабочие конфигурации для всех трех режимов ROI. Текстовые ROI достигают  $p99 = 8.032\text{ ms}$ , ROI лиц —  $p99 = 7.553\text{ ms}$ , текстурированные ROI —  $p99 = 9.230\text{ ms}$ . Эти значения относятся к отдельным веткам ROI-детекции и не означают, что все три детектора должны запускаться

последовательно на каждом кадре в рамках того же бюджета. Полный трехрежимный видеоконвейер требует планировщика запусков, переноса ROI между кадрами и отдельного end-to-end измерения.

Выбор рабочих точек отражает разную цену ошибок. Text ROI делает акцент на recall, Face ROI дает наилучший Area F1 среди выбранных профилей, а Cloth ROI сохраняет высокий recall/F1 при задержке ниже 10 ms. Эти профили далее использованы в покадровом сравнении кодирования.

I/O Binding существенно повышает локальность памяти в NUMA-сценарии, особенно для tiled-режимов. Top-Down Microarchitecture Analysis показывает, что tiled-инференс активнее использует SIMD, но одновременно может становиться backend-bound. Поэтому оптимизация IDet должна учитывать не только время инференса модели, но и весь критический путь: предобработку, планирование среды выполнения, локальность памяти, постобработку, NMS, сшивание плиток и слияние ROI.

Главный практический вывод главы состоит в следующем: IDet нельзя настраивать одной универсальной конфигурацией. Для производственной интеграции нужны профили политики среды выполнения: скоростной профиль для live-сценариев, сбалансированный профиль качества для nearline/offline обработки и адаптивное планирование для полного трехрежимного конвейера.

## Глава 9. Производственная интеграция, ограничения и план развития IDet

### 9.1 Граница производственной интеграции

Результаты предыдущих глав показывают, что IDet может формировать ROI в CPU-ориентированном режиме и имеет рабочие точки с приемлемым  $p99$  для отдельных веток текста, лиц и текстурированных ROI. Однако производственная интеграция требует рассматривать библиотеку не как набор независимых детекторов, а как часть видеоконвейера. В таком конвейере ROI-детекция должна получать кадры из потока, работать в заданном бюджете задержки, возвращать стабильную геометрию областей и не создавать непредсказуемую нагрузку на память, CPU и кодировщик.

В текущей архитектуре публичный контракт IDet остается геометрическим: библиотека возвращает список четырехугольников `VecQuad`. Это решение упрощает API и отделяет компьютерное зрение от политики кодирования. Производственная ROI-aware система строится поверх этого слоя: последующий компонент добавляет тип области, приоритет, ограничение площади, временное сглаживание и отображение в `qoffset` или другую управляющую информацию для кодировщика. Такое разделение важно, потому что оно не привязывает IDet к конкретной версии FFmpeg, x265 или другому бэкэнду кодирования.

Следовательно, производственная интеграция IDet должна состоять из трех уровней:

- 1) **Слой извлечения ROI** — собственно IDet, выполняющий детекцию текста, лиц и текстурированных областей;
- 2) **Слой управления ROI** — слияние областей, приоритизация, ограничение площади и стабилизация во времени;
- 3) **Слой интеграции с кодировщиком** — преобразование метаданных ROI в `qoffset`, ROI side data или резервную политику.

Такой подход позволяет развивать систему поэтапно. На первом этапе достаточно стабильно получать ROI и измерять их качество. На втором этапе можно использовать ROI для offline-анализа качества. На третьем этапе ROI подключаются к кодировщику и проверяются сначала покадрово, как в главе 7, а затем на полноценном видеопотоке с учетом *temporal mechanisms*.

### 9.2 Режимы эксплуатации

Предлагаемая система может использоваться в нескольких режимах эксплуатации. Они отличаются жесткостью требований к задержке и способом использования ROI.

Для данной ВКР наиболее важен сценарий реального или близкого к реальному времени, потому что именно он задает основные инженерные ограничения. Однако offline-анализ и assisted encoding также полезны: они позволяют проверить корректность ROI, подобрать

**Таблица 9.1:** Режимы эксплуатации ROI-aware системы на базе IDet

| Режим               | Основная цель                                | Особенности использования ROI                                                                                                                                              |
|---------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Offline-анализ      | Анализ качества и подготовка отчетов.        | ROI используются для расчета локальных метрик, визуального сравнения и подбора политики кодирования. Жесткого бюджета задержки нет.                                        |
| Assisted encoding   | Улучшение VoD или nearline-транскодирования. | ROI передаются кодировщику, но допускается более высокая задержка. Можно использовать сбалансированные профили качества.                                                   |
| Real-time streaming | Live или интерактивный видеопоток.           | ROI должны формироваться в жестком бюджете задержки. Нужны скоростные профили, периодический запуск детекторов, временное переиспользование и строгое ограничение площади. |

qoffset-политику и провести расширенные baseline vs ROI-aware эксперименты без риска нарушить live-конвейер.

### 9.3 Профили среды выполнения

Результаты главы с измерениями показывают, что универсальной конфигурации IDet для всех сценариев нет. Поэтому производственная система должна поддерживать несколько профилей среды выполнения.

**Таблица 9.2:** Предлагаемые профили среды выполнения IDet

| Профиль          | Цель                                 | Характеристика                                                                                                                |
|------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Speed-first      | Минимизировать $p99$ задержку.       | Использует быстрые tiled-конфигурации, ограничивает размер входа и число кандидатов, допускает умеренное снижение Area F1.    |
| Quality-balanced | Повысить качество покрытия ROI.      | Использует более дорогие конфигурации с лучшими Area Recall/Area F1, подходит для nearline или режима периодического запуска. |
| Adaptive         | Подстраиваться под состояние потока. | Меняет частоту запуска детекторов, переносит ROI между кадрами, отключает тяжелые ветки при росте задержки.                   |
| Diagnostic       | Собирать отладочные данные.          | Разрешает выгрузку ROI, подробный вывод и дополнительные метрики, но не используется для чистого измерения задержки.          |

Speed-first профиль подходит для live-сценариев, где важнее предсказуемость задержки. Quality-balanced профиль полезен для проверки максимального качества ROI и для offline/nearline кодирования. Adaptive профиль является наиболее перспективным для полной трехрежимной системы: он позволяет запускать дорогие детекторы не на каждом кадре, а между запусками использовать временное сглаживание и перенос областей.

### 9.4 Планировщик трех режимов ROI

Последовательный запуск детекторов текста, лиц и текстур на каждом кадре может превысить целевой бюджет задержки. Поэтому производственная версия IDet должна ис-



пользовать планировщик режимов. Его задача — решать, какие детекторы запускать на текущем кадре, какие ROI можно переиспользовать с предыдущих кадров, а какие области необходимо обновить немедленно.

Простейшая политика может быть задана следующим образом:

- 1) текстовые ROI запускаются чаще в сценариях с субтитрами, интерфейсом или презентациями;
- 2) ROI лиц запускаются чаще в talking-head и video conference сценариях;
- 3) текстурированные ROI запускаются реже, если одежда и крупные объекты стабильны во времени;
- 4) при превышении бюджета задержки тяжелые ветки временно отключаются;
- 5) ROI с высокой временной стабильностью переносятся на несколько кадров вперед;
- 6) при резкой смене сцены все ROI пересчитываются заново.

Такой планировщик превращает IDet из статической цепочки моделей в адаптивный компонент среды выполнения. Это особенно важно для текстурированных ROI: фактура одежды обычно меняется медленнее, чем текстовые overlay-элементы или положение лица, поэтому нет необходимости пересчитывать ее на каждом кадре в live-режиме.

## 9.5 Инварианты production-конвейера

Для надежной интеграции IDet в видеокодирование необходимо зафиксировать инварианты, которые должны выполняться независимо от выбранной модели и профиля среды выполнения.

- 1) координаты всех ROI должны находиться внутри границ исходного кадра;
- 2) суммарная площадь защищаемых областей не должна превышать заданный бюджет площади  $\alpha$ ;
- 3) пересекающиеся ROI должны разрешаться по приоритету: текст > лицо > текстурированная область > фон;
- 4) отсутствие ROI не должно считаться ошибкой конвейера;
- 5) сбой одного детектора не должен останавливать весь видеоконвейер;
- 6) адаптер кодировщика должен иметь резервный путь, если нативные метаданные ROI недоступны;
- 7) диагностические режимы не должны использоваться для финального измерения задержки;
- 8) параметры потоков и NUMA-состояние должны фиксироваться в протоколе измерений.

Эти инварианты отделяют исследовательский прототип от компонента, пригодного для практического встраивания. В дипломной работе они важны как часть инженерного результата: система должна быть не только точной на валидационном разбиении, но и предсказуемой при встраивании в реальный конвейер.

## 9.6 Ограничения текущей реализации и данных

Несмотря на высокие метрики YOLO26n и успешные CPU-only измерения отдельных режимов, текущие результаты имеют ограничения.

Первое ограничение связано со смещением домена. DeepFashion2 и Fashionpedia состоят из fashion-изображений, тогда как реальный видеостриминг содержит размытие движения, шум веб-камеры, предварительное сжатие, неполные фигуры, нестандартное освещение, несколько людей в кадре и сильные изменения масштаба. Поэтому модель текстурированных ROI должна дополнительно проверяться на видеокорпусе, близком к целевым сценариям: лекции, видеоконференции, обзоры товаров, talking-head и фрагменты прямых трансляций.

Второе ограничение связано с прямоугольной геометрией ROI. Bbox удобен для кодировщика и дешев в обработке, но может включать фон вокруг объекта. Если текстурированная ROI слишком широкая, часть битов будет потрачена на второстепенные области. Поэтому для текстурированных ROI особенно важны AreaPrecision, ExtraAreaRatio и отсечение в последующем ROI-слое.

Третье ограничение связано с тем, что покадровый эксперимент не проверяет временную стабильность. Для видео важно не только качество ROI на одном кадре, но и отсутствие мерцания между кадрами. Поэтому перенос результата на live-видеопоток требует временных метрик: стабильности координат, устойчивости присутствия ROI, частоты исчезновения областей и влияния сглаживания.

Четвертое ограничение связано с отсутствием полноценного сравнения на видеопотоке. Такой этап выходит за рамки работы, потому что требует не только повторного кодирования последовательностей, но и реализации пространственно-временного сглаживания ROI, tracking между кадрами, настройки GOP-структуры, контроля межкадрового предсказания и отдельной оценки доставки. Это уже самостоятельная инженерная задача с заметными вычислительными и временными затратами.

## 9.7 Риски ложных ROI

Ошибки ROI-детекции имеют разную цену в зависимости от типа области. Для текстовых ROI опасен пропуск текста: пользователь сразу замечает потерю читаемости. Для ROI лиц опасны и пропуски, и сильные искажения геометрии, потому что артефакты лица субъективно заметны. Для текстурированных ROI основной риск иной: модель может выделить визуально сложный фон, мебель, ковер или шумную поверхность и тем самым потратить битовый бюджет на область, которая не является приоритетной.

Поэтому текстурированные ROI должны иметь меньший приоритет, чем текст и лица, а также жестко ограничиваться по площади. Если текстурированная ROI пересекается с текстовой ROI или ROI лица, приоритет должен отдаваться более важной области. Если текстурированная ROI занимает слишком большую часть кадра, последующий слой должен уменьшить `qoffset`, обрезать область или исключить часть кандидатов.

Для производственной политики важен не только порог уверенности, но и сочетание трех факторов:

- 1) уверенность детектора;
- 2) доля площади кадра;
- 3) тип ROI и его приоритет для пользователя.

Такой подход лучше соответствует видеокодированию, чем простое применение object detection threshold. В задаче кодировщика модель не обязана идеально классифицировать одежду; она должна формировать полезные области для перераспределения качества.

## 9.8 Практическая значимость работы

Практическая значимость работы состоит в том, что предложенная архитектура может быть развита до полноценного ROI extraction layer для облачного видеостриминга. В рамках работы уже сформированы основные элементы такой системы:

- 1) CPU-ориентированная C++ библиотека IDet с единым геометрическим контрактом VecQuad;
- 2) трехрежимная архитектура ROI: текст, лица и текстурированные области;
- 3) воспроизводимый конвейер YOLO для подготовки, аудита, обучения и экспорта модели;
- 4) согласованная 7-классовая таксономия DeepFashion2 и Fashionpedia;
- 5) обученная легкая YOLO26n-модель для текстурированных ROI;
- 6) контракт разворачивания PT-to-ONNX для переноса модели в ONNX Runtime;
- 7) CPU-only измерения с анализом  $p50/p95/p99$  и качества покрытия по площади;
- 8) покадровый pipeline baseline vs ROI-aware на базе FFmpeg addroi и libx265;
- 9) численная оценка ROI-aware эффекта при сопоставимом размере потока.

Таким образом, результат работы находится на стыке машинного обучения и системной инженерии. Модель текстурированных ROI не рассматривается как самостоятельный fashion-детектор; ее назначение — стать частью управляемого видеоконвейера, где качество детекции, задержка, память и влияние на кодировщик оцениваются совместно.

## 9.9 План дальнейшего развития

Дальнейшее развитие работы целесообразно выполнять в несколько этапов.

- 1) **Видео-корпус для проверки смещения домена.** Необходимо собрать небольшой набор видеофрагментов из целевых сценариев: лекции, видеоконференции, обзоры товаров, talking-head и прямые трансляции.
- 2) **Временное сглаживание.** Следует реализовать перенос ROI между кадрами, сглаживание уверенности и контроль мерцания.
- 3) **Адаптивный планировщик.** Нужно добавить политику запуска детекторов с разной частотой в зависимости от сцены, нагрузки и бюджета задержки.

- 4) **Адаптер кодировщика.** Необходимо реализовать практический путь передачи ROI в кодировщик через нативные метаданные ROI или резервную политику.
- 5) **Видеоэксперимент baseline и ROI-aware.** Следует расширить покадровый результат до видеопотока с фиксированными preset, GOP, fps и temporal mechanisms.
- 6) **Варианты INT8/ORT.** Нужно отдельно проверить FP32, QDQ INT8, QOperator INT8 и целевые ORT-артефакты по качеству и p99 задержке.
- 7) **Расширение телеметрии.** Желательно фиксировать не только задержку, но и площадь ROI, число областей, стабильность во времени и влияние на отклонение битрейта.

Этот план позволяет последовательно перейти от текущей исследовательско-инженерной версии к ROI-aware системе, пригодной для практического развертывания. При этом каждый этап имеет проверяемый критерий успеха: корректность ROI, стабильность во времени, соблюдение бюджета задержки, совместимость с кодировщиком и подтвержденный ROI-выигрыш при сопоставимом размере потока.

## 9.10 Выводы по главе

Производственная интеграция IDet требует рассматривать библиотеку не как отдельный детектор, а как часть видеокодировочного конвейера. Для этого необходимо разделить ROI extraction, ROI management и encoder integration. Такое разделение сохраняет простой публичный контракт IDet и позволяет менять кодировщик, qoffset-политику и fallback-режимы без изменения детекторов.

Основные риски связаны со смещением домена, временной нестабильностью ROI, ложными текстурированными ROI и возможным превышением бюджета задержки при одновременном запуске всех режимов. Эти риски не отменяют предложенный подход, но задают требования к дальнейшей реализации: адаптивный планировщик, временное сглаживание, ограничение площади и строгие CPU-only измерения.

Практическая ценность работы состоит в том, что она формирует воспроизводимый путь от ML-модели к системному ROI-aware компоненту: от подготовки датасетов и обучения YOLO26n до развертывания через ONNX Runtime, CPU-only измерений и покадровой проверки ROI side data на реальном кодировщике.

## Заключение

В выпускной квалификационной работе рассмотрена задача повышения воспринимаемого качества облачного видеостриминга в условиях ограниченных вычислительных ресурсов. Основная идея работы состоит в использовании ROI-aware подхода: значимые области кадра выделяются до этапа кодирования, после чего последующий интеграционный слой может передать кодировщику информацию о приоритете этих областей. Такой подход дополняет классические механизмы адаптивного стриминга и видеокодирования пространственным уровнем оптимизации: качество перераспределяется не только между сегментами видео, но и внутри отдельного кадра.

В работе предложена архитектура трехрежимного ROI-конвейера на базе библиотеки IDet. Библиотека рассматривается как CPU-ориентированный C++ слой между компьютерным зрением и видеокодированием. В ее архитектуре выделены три типа областей интереса: текстовые области, лица и текстурированные области одежды или материалов. Текстовые ROI важны для читаемости, ROI лиц — для сохранения субъективно значимых человеческих областей, текстурированные ROI — для защиты визуально богатых фактур, потеря которых воспринимается как снижение резкости и детализации.

Центральным инженерным результатом является системное описание IDet как компонента среды выполнения. В работе зафиксированы границы ответственности библиотеки, жизненный цикл обработки кадра, публичный геометрический контракт VecQuad, роль адаптеров детекторов, модель владения буферами, политика потоков, требования к критическому пути без динамических аллокаций, I/O Binding и NUMA-aware исполнению. Отдельно подчеркнуто, что типизация ROI, приоритеты и `qoffset` являются последующим слоем поверх фактического публичного API IDet, а не уже существующим публичным классом библиотеки. Это разделение предотвращает смешение реализованной части и проектируемого слоя интеграции с кодировщиком.

Для третьего режима IDet подготовлен детектор текстурированных ROI на базе YOLO26n. Для его обучения использован воспроизводимый конвейер обучения YOLO, включающий конвертацию DeepFashion2 и Fashionpedia в YOLO-формат, согласование таксономии до семи классов, аудит датасетов, запуск обучения, анализ поклассовых метрик и подготовку PT-to-ONNX контракта для развертывания. Объединенная таксономия позволила использовать преимущества обоих датасетов: масштаб DeepFashion2 и дополнительное разнообразие Fashionpedia. При этом в работе отдельно рассмотрены дисбаланс классов, геометрия bbox, центральное смещение объектов и ограничения переноса fashion-датасетов на реальные видеосценарии.

Экспериментальные результаты обучения показали, что легкая модель YOLO26n достигает высокого качества детекции на объединенной 7-классовой таксономии:  $\text{precision} = 0.903$ ,  $\text{recall} = 0.873$ ,  $\text{mAP}@0.5 = 0.938$  и  $\text{mAP}@0.5:0.95 = 0.818$ . Более тяжелая YOLO26s в отдельных экспериментах дает прирост точности, но имеет существенно большую вычислительную стоимость: около 20.5 GFLOPs против 5.2 GFLOPs у YOLO26n. Поэтому для

CPU-ориентированной IDet выбран YOLO26n как более практичный компромисс между качеством и задержкой.

Отдельный вклад работы состоит в формализации ROI-aware видеокодирования как ограниченной оптимизационной задачи. Введены ограничения на задержку, вычислительный бюджет, битрейт и суммарную площадь защищаемых областей. Рассмотрены ROI-взвешенные метрики качества, метрики покрытия по площади, временное сглаживание, штраф за мерцание ROI и отображение приоритета ROI в qoffset. Такой формальный слой нужен для того, чтобы работа не сводилась к обучению детектора объектов, а описывала полную задачу перераспределения качества в видеопотоке.

В экспериментальной части по производительности проведен CPU-only grid-search для режимов текстовых ROI, ROI лиц и текстурированных ROI. Для каждого режима выбрана рабочая точка с учетом компромисса между задержкой и покрытием областей: Text ROI дает  $p99 = 8.032\text{ ms}$  и  $\text{AreaF}_1 = 0.788$ , Face ROI —  $p99 = 7.553\text{ ms}$  и  $\text{AreaF}_1 = 0.922$ , Cloth ROI —  $p99 = 9.230\text{ ms}$  и  $\text{AreaF}_1 = 0.871$ . Эти значения подтверждают практическую применимость отдельных веток IDet в заданном CPU-бюджете, но не являются end-to-end задержкой полного трехрежимного видеоконвейера. Для одновременной работы всех режимов требуется планировщик, задающий частоту запуска детекторов и правила переиспользования ROI между кадрами.

Профилирование показало, что I/O Binding существенно повышает локальность памяти в NUMA-сценарии, особенно для режимов с разбиением кадра на плитки. Top-Down Microarchitecture Analysis показал, что tiled inference увеличивает долю SIMD-инструкций, но одновременно может усиливать backend-bound поведение. Поэтому оптимизация IDet должна рассматривать не только модельный инференс, но и весь критический путь: подготовку входа, планирование ONNX Runtime, локальность памяти, постобработку, NMS, сшивание плиток и слияние ROI.

Практическая значимость работы заключается в создании воспроизводимого пути от ML-модели к системному ROI-aware компоненту, который повышает качество значимых областей кадра за счет защиты ROI. В рамках работы описаны и связаны между собой: CPU-ориентированная библиотека IDet, конвейер обучения YOLO, объединенная 7-классовая таксономия, обученная YOLO26n-модель для текстурированных ROI, PT-to-ONNX контракт развертывания, CPU-only измерения и покадровый pipeline baseline vs ROI-aware на базе FFmpeg addroi и libx265.

Покадровый эксперимент выполнен при CRF=32. Preset был зафиксирован как medium, а режим match\_baseline обеспечивал сопоставимый размер потока. Проверка проведена на подготовленных FHD-наборах: 3000 изображений для Text ROI, 3000 изображений для Face ROI и 2696 изображений для Cloth ROI. Результаты показали, что ROI-aware кодирование перераспределяет качество в пользу областей интереса при сопоставимом размере потока. Показатель ROI-crop VMAF вырос на 4.451 в режиме Text ROI, на 5.857 в режиме Face ROI и на 0.449 в режиме Cloth ROI. ROI-only PSNR также вырос во всех трех режимах: на 3.331 dB для Text, на 3.284 dB для Face и на 0.547 dB для Cloth.

При этом часть `global/background` метрик снизилась, например `background PSNR` для `Cloth` уменьшился на 1.148 dB. Это не ошибка эксперимента, а ожидаемый компромисс: в режиме `match_baseline` кодировщик отдает больше битов ROI и меньше фону. Меньший эффект `Cloth` объясняется более мягким `qoffset`, большей неоднородностью текстурированных областей и тем, что одежда чаще занимает широкую прямоугольную ROI с примесью фона. Поэтому основной вывод работы состоит не в максимизации отдельной метрики, а в подтверждении управляемого локального выигрыша на фиксированной HEVC-конфигурации без увеличения размера потока.

Основное ограничение текущих результатов состоит в покадровой постановке эксперимента. Поэтому корректный вывод состоит не в утверждении о полном видеопотоке, а в зафиксированном `frame-level` эффекте ROI `side data`. Сравнение на реальном видеопотоке сознательно вынесено за рамки работы: оно требует видеокорпуса из целевых сценариев, пространственно-временного сглаживания ROI, `tracking` между кадрами, настройки GOP, контроля межкадрового предсказания и отдельной оценки доставки. Это существенно более дорогой этап по вычислениям, данным и инженерной реализации.

## Список литературы

- [1] Spiteri K., Urgaonkar R., Sitaraman R. K. BOLA: Near-Optimal Bitrate Adaptation for Online Videos. IEEE INFOCOM, 2016. DOI: <https://doi.org/10.1109/INFOCOM.2016.7524428>.
- [2] Mao H., Netravali R., Alizadeh M. Neural Adaptive Video Streaming with Pensieve. ACM SIGCOMM, 2017. DOI: <https://doi.org/10.1145/3098822.3098843>.
- [3] Huynh-Thu Q., Ghanbari M. Scope of Validity of PSNR in Image/Video Quality Assessment. Electronics Letters, 2008, Vol. 44, No. 13, pp. 800–801. DOI: <https://doi.org/10.1049/el:20080522>.
- [4] Wang Z., Bovik A. C., Sheikh H. R., Simoncelli E. P. Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE Transactions on Image Processing, 2004, Vol. 13, No. 4, pp. 600–612. DOI: <https://doi.org/10.1109/TIP.2003.819861>.
- [5] Netflix. VMAF: Video Multi-Method Assessment Fusion. URL: <https://github.com/Netflix/vmaf>.
- [6] Bampis C. G., Li Z., Bovik A. C. SpatioTemporal Feature Integration and Model Fusion for Full Reference Video Quality Assessment. IEEE Transactions on Circuits and Systems for Video Technology, 2019. URL: <https://arxiv.org/abs/1804.04813>.
- [7] Zhang R., Isola P., Efros A. A., Shechtman E., Wang O. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. CVPR, 2018. URL: <https://arxiv.org/abs/1801.03924>.
- [8] Ding K., Ma K., Wang S., Simoncelli E. P. Image Quality Assessment: Unifying Structure and Texture Similarity. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, Vol. 44, No. 5, pp. 2567–2581. DOI: <https://doi.org/10.1109/TPAMI.2020.3045810>.
- [9] Mantiuk R. K., Denes G., Chapiro A., Kaplanyan A., Rufo G., Bachy R., Lian T., Patney A. FovVideoVDP: A Visible Difference Predictor for Wide Field-of-View Video. ACM Transactions on Graphics, 2021, Vol. 40, No. 4. DOI: <https://doi.org/10.1145/3450626.3459831>.
- [10] ITU-T. Recommendation P.910: Subjective Video Quality Assessment Methods for Multimedia Applications. URL: <https://www.itu.int/rec/T-REC-P.910>.
- [11] Wiegand T., Sullivan G. J., Bjøntegaard G., Luthra A. Overview of the H.264/AVC Video Coding Standard. IEEE Transactions on Circuits and Systems for Video Technology, 2003, Vol. 13, No. 7, pp. 560–576. DOI: <https://doi.org/10.1109/TCSVT.2003.815165>.



- [12] ITU-T. Recommendation H.265: High Efficiency Video Coding. URL: <https://www.itu.int/rec/T-REC-H.265>.
- [13] Ватолин Д. С. Курс «Методы сжатия медиаданных». Лаборатория компьютерной графики и мультимедиа ВМК МГУ. URL: <https://www.compression.ru/dv/course/>.
- [14] Тропченко А. Ю., Тропченко А. А. Методы сжатия изображений, аудиосигналов и видео: учебное пособие. СПб: СПбГУ ИТМО, 2009. URL: <https://books.ifmo.ru/file/pdf/574.pdf>.
- [15] FFmpeg Developers. AVRegionOfInterest Struct Reference. URL: <https://ffmpeg.org/doxygen/trunk/structAVRegionOfInterest.html>.
- [16] FFmpeg Developers. FFmpeg Filters Documentation: addroi Video Filter. URL: <https://ffmpeg.org/ffmpeg-filters.html#addroi>.
- [17] x265 Project. Command Line Options. URL: <https://x265.readthedocs.io/en/stable/cli.html>.
- [18] Zhou X., Li Z., Zhang Y., Wang S. Region-of-Interest Based Coding Scheme for Live Videos. Applied Sciences, 2024, Vol. 14, No. 9, Article 3823. DOI: <https://doi.org/10.3390/app14093823>.
- [19] Huang S.-F., Chen M.-J., Tai K.-H., Li M.-S. Region-of-Interest Determination and Bit-Rate Conversion for H.264 Video Transcoding. EURASIP Journal on Advances in Signal Processing, 2013, Article 112. DOI: <https://doi.org/10.1186/1687-6180-2013-112>.
- [20] Meddeb M., Cagnazzo M., Pesquet-Popescu B. Region-of-Interest-Based Rate Control Scheme for High-Efficiency Video Coding. APSIPA Transactions on Signal and Information Processing, 2014, Vol. 3, e16. DOI: <https://doi.org/10.1017/ATSIP.2014.20>.
- [21] Hadizadeh H., Bajić I. V. Saliency-Aware Video Compression. IEEE Transactions on Image Processing, 2014, Vol. 23, No. 1, pp. 19–33. DOI: <https://doi.org/10.1109/TIP.2013.2282897>.
- [22] Liao M., Wan Z., Yao C., Chen K., Bai X. Real-Time Scene Text Detection with Differentiable Binarization. AAAI, 2020. URL: <https://arxiv.org/abs/1911.08947>.
- [23] Liao M., Zou Z., Wan Z., Yao C., Bai X. Real-Time Scene Text Detection with Differentiable Binarization and Adaptive Scale Fusion. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022. DOI: <https://doi.org/10.1109/TPAMI.2022.3155612>.
- [24] Guo J., Deng J., Lattas A., Zafeiriou S. Sample and Computation Redistribution for Efficient Face Detection. ICLR, 2022. URL: <https://arxiv.org/abs/2105.04714>.

- [25] ONNX Project. Open Neural Network Exchange. URL: <https://onnx.ai/>.
- [26] ONNX Runtime Documentation. URL: <https://onnxruntime.ai/docs/>.
- [27] ONNX Runtime. I/O Binding for Performance. URL: <https://onnxruntime.ai/docs/performance/tune-performance/iobinding.html>.
- [28] ONNX Runtime. Graph Optimizations. URL: <https://onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html>.
- [29] ONNX Runtime. Tune Performance. URL: <https://onnxruntime.ai/docs/performance/tune-performance/>.
- [30] ONNX Runtime. Thread Management. URL: <https://onnxruntime.ai/docs/performance/tune-performance/threading.html>.
- [31] ONNX Runtime. Quantize ONNX Models. URL: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>.
- [32] Ultralytics. Model Export with Ultralytics YOLO. URL: <https://docs.ultralytics.com/modes/export/>.
- [33] Ultralytics. ONNX Export for YOLO Models. URL: <https://docs.ultralytics.com/integrations/onnx/>.
- [34] Ultralytics. YOLO Performance Metrics. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/>.
- [35] Ultralytics. Data Augmentation using Ultralytics YOLO. URL: <https://docs.ultralytics.com/guides/yolo-data-augmentation/>.
- [36] Singh A., Natarjan V., Shah M., Jiang Y., Chen X., Batra D., Parikh D., Rohrbach M. TextOCR: Towards Large-Scale End-to-End Reasoning for Arbitrary-Shaped Scene Text. CVPR, 2021. URL: <https://arxiv.org/abs/2105.05486>.
- [37] Yang S., Luo P., Loy C. C., Tang X. WIDER FACE: A Face Detection Benchmark. CVPR, 2016. URL: <https://arxiv.org/abs/1511.06523>.
- [38] Ge Y., Zhang R., Wu L., Wang X., Tang X., Luo P. DeepFashion2: A Versatile Benchmark for Detection, Pose Estimation, Segmentation and Re-Identification of Clothing Images. CVPR, 2019. URL: <https://arxiv.org/abs/1901.07973>.
- [39] Jia M., Shi M., Sirotenko M., Cui Y., Cardie C., Hariharan B., Adam H., Belongie S. Fashionpedia: Ontology, Segmentation, and an Attribute Localization Dataset. ECCV, 2020. DOI: [https://doi.org/10.1007/978-3-030-58452-8\\_19](https://doi.org/10.1007/978-3-030-58452-8_19).
- [40] NVIDIA. GeForce RTX 4090 Specifications. URL: <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/>.

- [41] Red Hat. Automatic NUMA Balancing. Red Hat Enterprise Linux 7 Virtualization Tuning and Optimization Guide. URL: [https://docs.redhat.com/en/documentation/red\\_hat\\_enterprise\\_linux/7/html/virtualization\\_tuning\\_and\\_optimization\\_guide/sect-virtualization\\_tuning\\_optimization\\_guide-numa-auto\\_numa\\_balancing](https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_tuning_and_optimization_guide/sect-virtualization_tuning_optimization_guide-numa-auto_numa_balancing).
- [42] Linux Kernel. perf: Linux Profiling with Performance Counters. URL: <https://perf.wiki.github.io/main/>.
- [43] Arm. Statistical Profiling Extension. URL: [https://learn.arm.com/learning-paths/cross-platform/windowsperf\\_sampling\\_cpython\\_spe/overview/](https://learn.arm.com/learning-paths/cross-platform/windowsperf_sampling_cpython_spe/overview/).
- [44] Yasin A. A Top-Down Method for Performance Analysis and Counters Architecture. IEEE International Symposium on Performance Analysis of Systems and Software, 2014. DOI: <https://doi.org/10.1109/ISPASS.2014.6844459>.
- [45] Arm. Compare Arm Neoverse and Intel x86 Top-Down Performance Analysis with PMU Counters. URL: <https://learn.arm.com/learning-paths/cross-platform/topdown-compare/>.
- [46] Parshin D. IDet: CPU-first C++ Library for Text, Face and Textured ROI Detection. Author's software implementation, 2026. URL: <https://github.com/De-Par/IDet.git>.
- [47] Parshin D. Generic YOLO Training Pipeline for Dataset Conversion, Training, Export and Quantization. Author's software implementation, 2026. URL: <https://github.com/De-Par/yolo-training-pipeline.git>.

## Приложения

Ниже собраны вспомогательные материалы, которые нужны для воспроизводимости работы, но перегружают основной текст. Основные выводы по датасетам, YOLO-моделям, измерениям IDet и производственной интеграции приведены в соответствующих главах. Здесь представлены расширенные сводки, дополнительные матрицы ошибок, параметры запусков и чек-листы контроля артефактов.

### A.1 Расширенная сводка аудита датасетов

Таблица A.1 используется как контрольная точка качества данных перед обучением. В ней сопоставлены масштаб каждого корпуса, среднее число объектов на изображение и состояние label-файлов после конвертации в YOLO-формат. Отсутствие пустых, пропущенных, orphan-label и некорректных строк разметки означает, что дальнейшие различия между моделями объясняются архитектурой, составом датасета и режимом обучения, а не техническими ошибками подготовки корпуса.

**Таблица A.1:** Расширенная сводка аудита датасетов с 7-классовой таксономией после конвертации в YOLO-формат: число изображений, объектов и результат проверки label-файлов

| Датасет         | Изображения | Объекты | Объектов/изобр. | Метки  |        |              |
|-----------------|-------------|---------|-----------------|--------|--------|--------------|
|                 |             |         |                 | Всего  | Пустые | Некорректные |
| DeepFashion2-7c | 224114      | 364675  | 1.6272          | 224114 | 0      | 0            |
| Fashionpedia-7c | 45445       | 74724   | 1.6443          | 45445  | 0      | 0            |
| DF2+FP-7c       | 269559      | 439399  | 1.6301          | 269559 | 0      | 0            |

### A.2 Распределение классов по датасетам

Таблица A.2 раскрывает, как меняется баланс классов после объединения DeepFashion2 и Fashionpedia. Для ROI-задачи эта сводка важна не как fashion-аналитика, а как проверка устойчивости детектора: частые классы формируют основной обучающий сигнал, а редкие категории повышают риск пропусков textured ROI и требуют отдельной интерпретации поклассовых AP.

**Таблица A.2:** Распределение объектов по 7-классовой таксономии в DeepFashion2-7c, Fashionpedia-7c и объединенном корпусе DF2+FP-7c

| Класс          | DeepFashion2-7c | Fashionpedia-7c | DF2+FP-7c | Комментарий                    |
|----------------|-----------------|-----------------|-----------|--------------------------------|
| top_sleeved    | 126230          | 24803           | 151033    | Доминирующий класс             |
| outerwear      | 16153           | 9135            | 25288     | Усилен после объединения       |
| top_sleeveless | 20515           | 741             | 21256     | Самый редкий в DF2+FP-7c       |
| shorts         | 40783           | 2862            | 43645     | Устойчивый класс               |
| trousers       | 64973           | 12728           | 77701     | Один из крупнейших классов     |
| skirt          | 37357           | 5208            | 42565     | Средний по объему класс        |
| dress          | 58664           | 19247           | 77911     | Второй по масштабу в DF2+FP-7c |

Из таблицы A.2 следует, что объединение наборов расширяет корпус, но не делает его равномерным: top\_sleeved, dress и trousers остаются наиболее представленными

категориями, тогда как `top_sleeveless` остается редким классом. Поэтому агрегированная mAP не является достаточным основанием для выбора модели: ее нужно читать вместе с per-class AP, confusion matrix и последующими area-based метриками покрытия ROI.

### А.3 Ключевые параметры обучающих запусков

Таблицы А.3 и А.4 разделяют два уровня воспроизводимости: состав запуска и параметры оптимизации. Первую таблицу следует читать как описание экспериментальной сетки, а вторую — как фиксацию факторов, которые могут менять сходимость и итоговую локализацию bbox. Полные конфигурации сохраняются в артефактах запуска; здесь вынесены только параметры, без которых сравнение YOLO26n и YOLO26s становится неоднозначным.

**Таблица А.3:** Ключевые параметры обучающих запусков YOLO: архитектура, источник данных, длительность обучения и размер batch

| Модель  | Датасет         | Эпохи | Batch |
|---------|-----------------|-------|-------|
| YOLO26n | DeepFashion2-7c | 180   | 48    |
| YOLO26n | DF2+FP-7c       | 300   | 48    |
| YOLO26s | DeepFashion2-7c | 150   | 24    |
| YOLO26s | DF2+FP-7c       | 300   | 32    |

Таблица А.3 показывает, что сравнение построено не только вокруг архитектуры, но и вокруг источника данных. Финальные варианты на объединенном наборе обучались дольше, поскольку DF2+FP-7c содержит больше разнообразных примеров, а batch для YOLO26s уменьшен из-за более высокой вычислительной стоимости. Это важно при интерпретации результатов: выигрыш YOLO26s нельзя рассматривать отдельно от роста стоимости обучения и последующего CPU-инференса.

**Таблица А.4:** Параметры оптимизации и аугментаций YOLO-запусков: размер входа, начальный learning rate, сила mosaic и момент отключения mosaic

| Модель  | Датасет         | Размер входа | LR0    | Mosaic | Close mosaic | Примечание                         |
|---------|-----------------|--------------|--------|--------|--------------|------------------------------------|
| YOLO26n | DeepFashion2-7c | 640          | 0.0009 | 0.8    | 10           | Базовый запуск на DeepFashion2-7c  |
| YOLO26n | DF2+FP-7c       | 640          | 0.0009 | 0.6    | 15           | Финальный легкий кандидат для IDet |
| YOLO26s | DeepFashion2-7c | 640          | 0.0009 | 0.6    | 15           | Тяжелая модель для сравнения       |
| YOLO26s | DF2+FP-7c       | 640          | 0.0009 | 0.6    | 15           | Тяжелый вариант на DF2+FP-7c       |

Таблица А.4 фиксирует одинаковый размер входа и начальный learning rate, а также различия в mosaic-аугментации. Для textured ROI это не формальная деталь: mosaic влияет на геометрию объектов, частоту частичных bbox и устойчивость модели к изменению масштаба. Поэтому таблица объясняет, при каких условиях были получены mAP и recall, а не просто перечисляет параметры запуска.

### А.4 Сводка результатов обучения YOLO

Таблица А.5 связывает качество модели с ее вычислительной ценой. Для задачи IDet важны не только precision, recall и mAP, но и размер модели, GFLOPs и вероятность уложиться в бюджет задержки. Поэтому таблица используется как аргумент в пользу легкой модели: она показывает, какую цену приходится платить за прирост качества у YOLO26s.

**Таблица A.5:** Сводка качества и вычислительной стоимости YOLO-запусков: precision, recall, mAP и относительная тяжесть модели

| Модель  | Датасет         | P     | R     | mAP50 | mAP50-95 | Парам., млн | GFLOPs | Комментарий                    |
|---------|-----------------|-------|-------|-------|----------|-------------|--------|--------------------------------|
| YOLO26n | DeepFashion2-7c | 0.915 | 0.888 | 0.949 | 0.829    | 2.38        | 5.2    | Сильная легкая baseline-модель |
| YOLO26n | DF2+FP-7c       | 0.903 | 0.873 | 0.938 | 0.818    | 2.38        | 5.2    | Финальный легкий кандидат      |
| YOLO26s | DeepFashion2-7c | 0.935 | 0.906 | 0.959 | 0.850    | 9.47        | 20.5   | Точнее, но существенно тяжелее |
| YOLO26s | DF2+FP-7c       | 0.920 | 0.886 | 0.944 | 0.837    | 9.47        | 20.5   | Умеренный прирост качества     |

Из таблицы A.5 видно, что YOLO26s улучшает mAP, но требует примерно в четыре раза больше параметров и GFLOPs. Для CPU-first сценария это существенное ограничение: добавочная точность должна конкурировать с p95/p99-задержкой, стоимостью предобработки и постобработки, а также параллельной работой других ROI-веток. Поэтому YOLO26n рассматривается как более практичный кандидат для IDet: он немного уступает по качеству, но лучше согласуется с ограниченным вычислительным бюджетом.

## A.5 Матрицы ошибок и метрики обучения YOLO-экспериментов

В основном тексте приведены итоговые численные метрики, нормализованная матрица ошибок для выбранной модели и сравнение диагональных значений confusion matrix. В приложение вынесены полные диагностические визуализации для четырех YOLO-запусков: матрицы ошибок и графики метрик после обучения.

Матрица ошибок позволяет оценить, какие классы модель чаще всего путает между собой, а график метрик обучения показывает динамику качества, устойчивость сходимости, поведение loss-функций и момент выбора лучшего checkpoint. Такое представление важно, поскольку итоговые значения precision, recall и mAP фиксируют только финальную точку, но не показывают, насколько стабильно модель к ней пришла.

На рисунке A.1 видно, что YOLO26n на DeepFashion2-7c уверенно отделяет основные классы одежды, но ошибки в сторону background сохраняют риск пропуска части textured ROI.

По рисунку A.2 видно, что кривые обучения стабилизируются без резких провалов. Поэтому итоговые метрики для YOLO26n + DeepFashion2-7c можно интерпретировать как результат сходимости, а не случайный выброс последней эпохи.

Рисунок A.3 показывает, что на объединенном наборе диагональ остается высокой, но top\_sleeveless и outerwear остаются самыми чувствительными классами.

Рисунок A.4 показывает, что объединенный датасет не ломает обучение YOLO26n: метрики выходят на плато, а loss-кривые снижаются. Это подтверждает пригодность объединенной таксономии.

На рисунке A.5 видно, что YOLO26s улучшает диагональ для ряда классов, но не устраняет полностью ошибки близких категорий одежды и переходы в background.

Рисунок A.6 показывает устойчивую сходимость YOLO26s, но ее преимущество нужно сопоставлять с четырехкратной стоимостью относительно YOLO26n.



Рисунок А.1: Нормализованная матрица ошибок YOLO26n на DeepFashion2-7c.

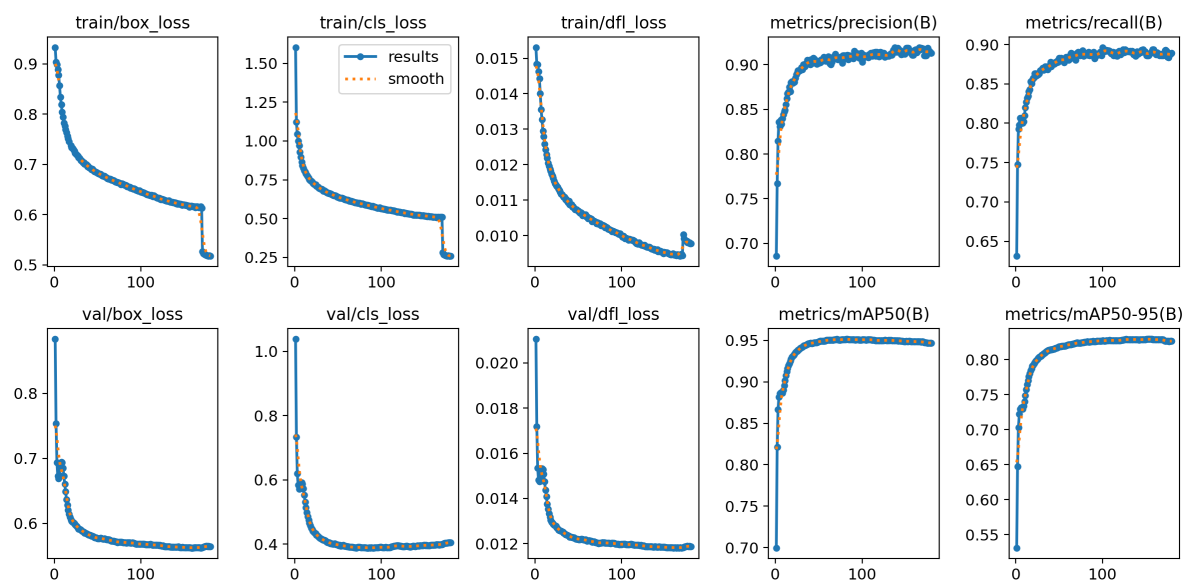


Рисунок А.2: Динамика обучения YOLO26n на DeepFashion2-7c.

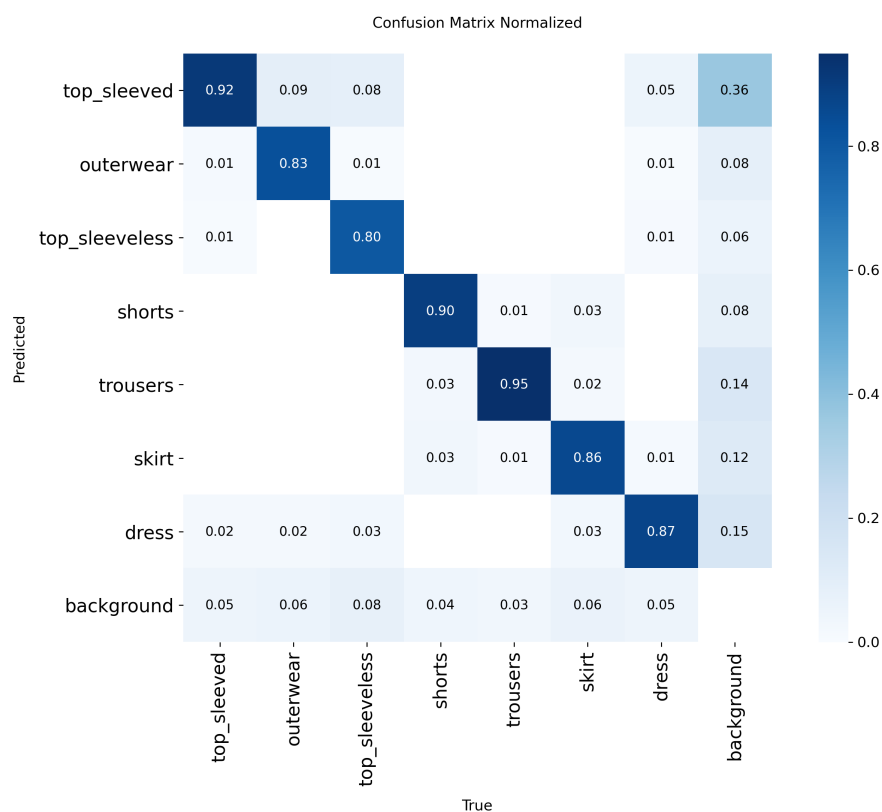


Рисунок А.3: Нормализованная матрица ошибок YOLO26n на DF2+FP-7c.

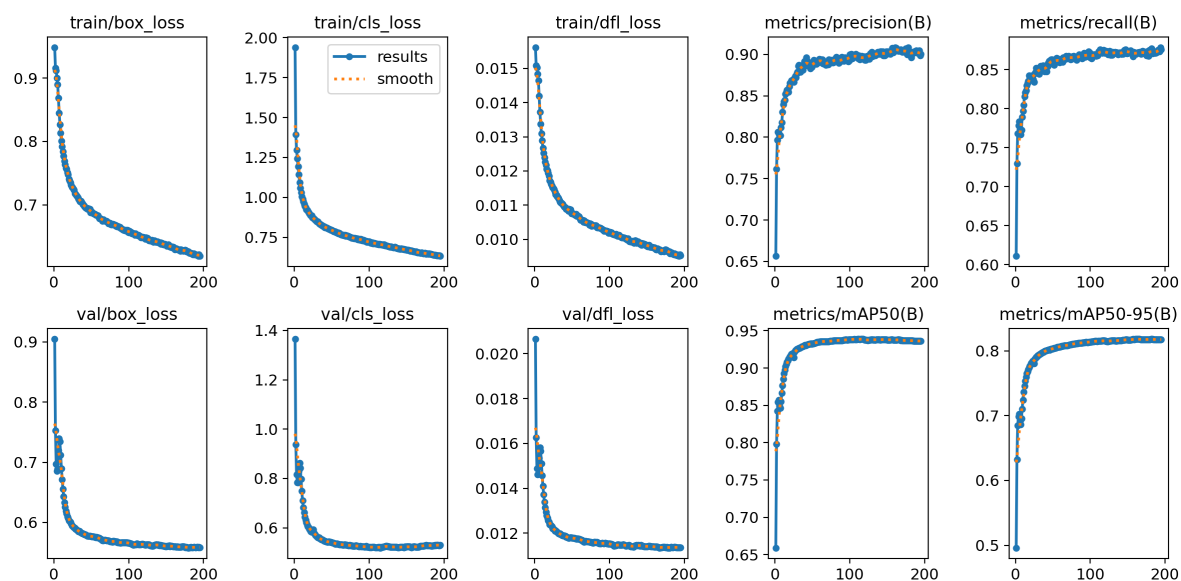


Рисунок А.4: Динамика обучения YOLO26n на DF2+FP-7c.





Рисунок А.5: Нормализованная матрица ошибок YOLO26s на DeepFashion2-7c.

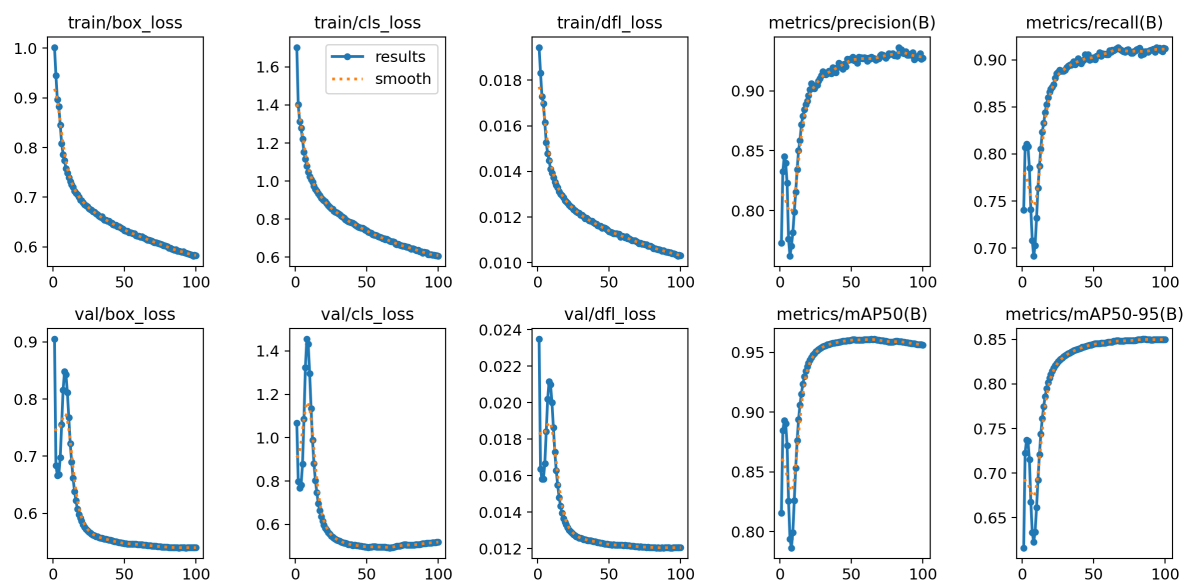
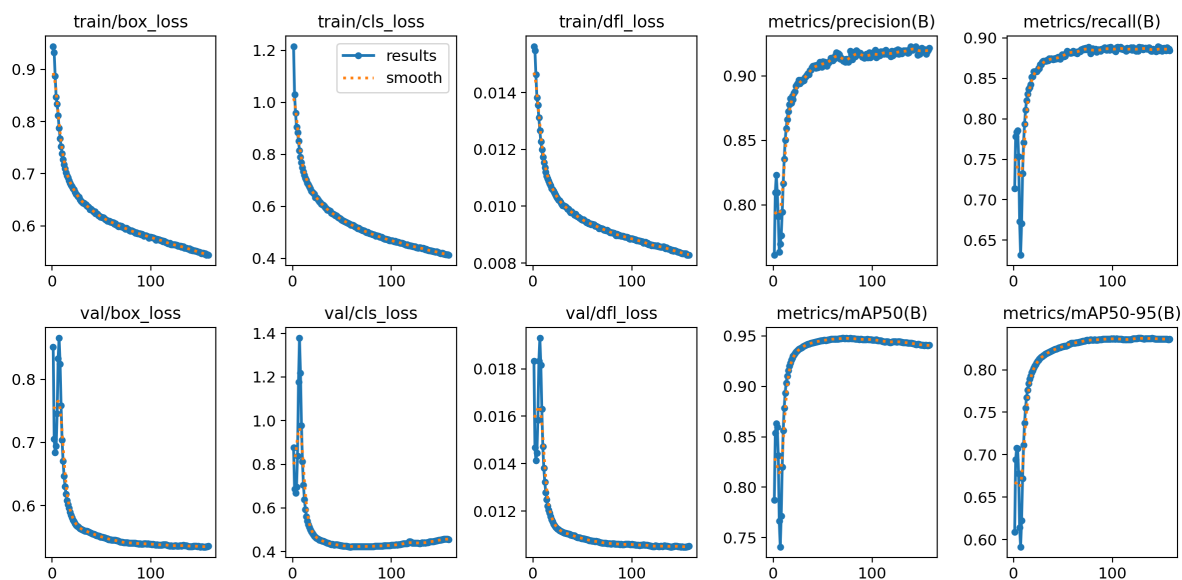


Рисунок А.6: Динамика обучения YOLO26s на DeepFashion2-7c.



**Рисунок А.7:** Нормализованная матрица ошибок YOLO26s на DF2+FP-7с.

На рисунке А.7 видно, что на объединенном наборе YOLO26s сохраняет сильную диагональ, но практическое преимущество над YOLO26n остается умеренным для CPU-first задачи.



**Рисунок А.8:** Динамика обучения YOLO26s на DF2+FP-7с.

Рисунок А.8 подтверждает работоспособность обучения YOLO26s на объединенном наборе, но не отменяет deployment-ограничение по размеру и GFLOPs.

Сравнение всех диагностических рисунков показывает, что интерпретация качества модели должна учитывать как итоговую структуру ошибок, так и динамику обучения. Для задачи textured ROI особенно важны не только ошибки между близкими классами одежды, но и случаи перехода объекта в background. Если модель путает близкие типы одежды, но сохраняет корректное покрытие области, downstream-кодировщик все равно может использовать такую область как textured ROI. Гораздо опаснее ситуации, когда значимая область не выделяется вовсе или когда ROI становится чрезмерно широкой и начинает расходовать битовый бюджет на второстепенные фрагменты кадра.

## A.6 Визуализация labels для датасетных конфигураций

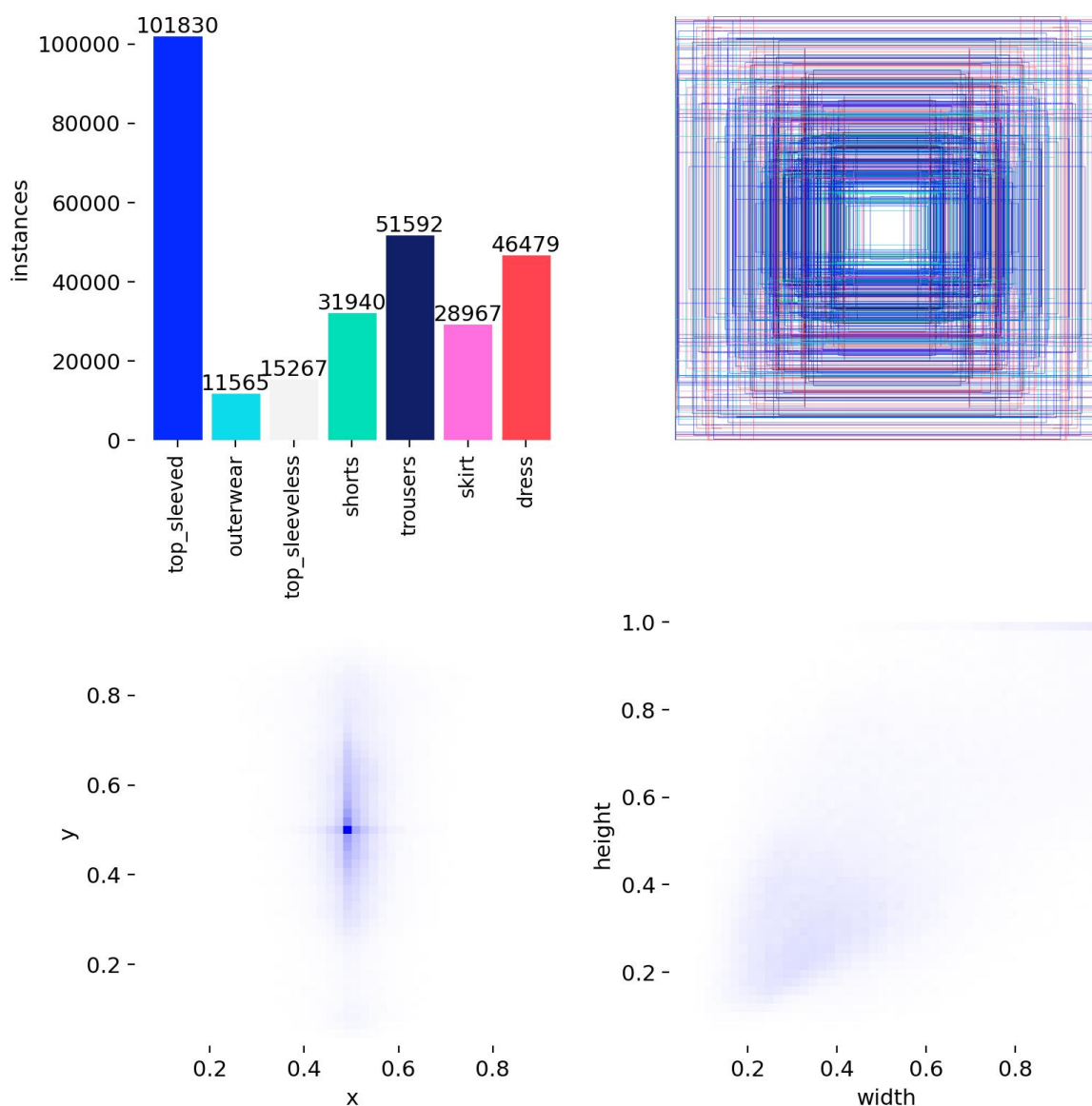
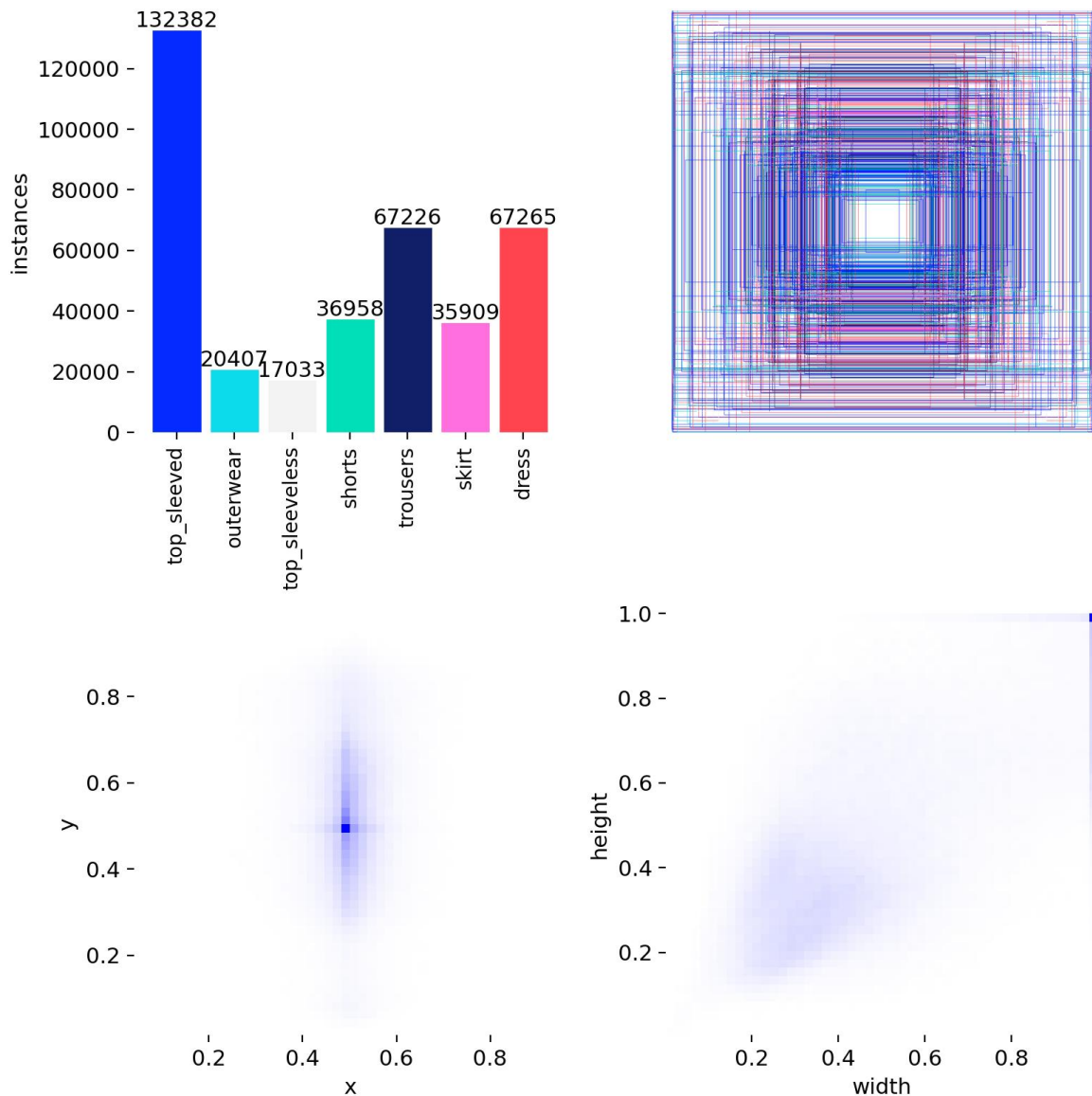


Рисунок A.9: Визуализация labels для DeepFashion2-7c.

Рисунок A.9 показывает структуру разметки DeepFashion2-7c: распределение классов, центры bbox, ширину и высоту bbox, а также совместное распределение геометрии объектов. Для этого набора характерны концентрация объектов около центра кадра, преобладание

крупных bbox и перекос в пользу верхней одежды. В textured ROI это полезно для локализации одежды, но требует контроля area budget: крупные области могут конкурировать с text/face ROI.



**Рисунок А.10:** Визуализация labels для объединенного набора DF2+FP-7с.

Рисунок А.10 показывает структуру labels после объединения DeepFashion2 и Fashionpedia в единую 7-классовую таксономию. Объединенный набор содержит больше изображений и объектов, расширяет геометрию примеров, но не устраняет дисбаланс классов: наиболее частые категории формируют большую часть обучающего сигнала, тогда как редкие классы требуют отдельного контроля через per-class AP и confusion matrix.

## В.1 Команда чистого измерения задержки

Листинг В.1 показывает параметризованный запуск `idet_app` для чистого измерения задержки. В этом режиме отключены отрисовка, выгрузка геометрии и подробный вывод, поэтому результат отражает задержку пути среды выполнения, а не стоимость диагностической записи.

**Листинг В.1:** Параметризованный запуск `idet_app` для чистого измерения задержки

```
1 idet_app \  
2   --mode cloth \  
3   --model <onnx-model> \  
4   --image <input-image> \  
5   --bind_io 1 \  
6   --runtime_policy 1 \  
7   --soft_mem_bind 1 \  
8   --cpu_placement latency \  
9   --bench_iters 30 \  
10  --warmup_iters 5 \  
11  --is_draw 0 \  
12  --is_dump 0 \  
13  --verbose 0 \  
14  --fixed_hw 128x128 \  
15  --tiles_rc 2x4 \  
16  --threads_intra 8 \  
17  --threads_inter 1 \  
18  --tile_omp 8
```

## В.2 Отдельный запуск для ROI-геометрии

Для расчета метрик покрытия по площади требуется получить ROI-геометрию. Этот запуск выполняется отдельно от чистого измерения задержки, поскольку выгрузка геометрии добавляет операции вывода и не должна входить в оценку задержки.

**Листинг В.2:** Отдельный запуск для выгрузки ROI-геометрии

```
1 idet_app \  
2   --mode cloth \  
3   --model <onnx-model> \  
4   --image <input-image> \  
5   --bind_io 1 \  
6   --runtime_policy 1 \  
7   --soft_mem_bind 1 \  
8   --cpu_placement latency \  
9   --bench_iters 1 \  
10  --warmup_iters 0 \  
11  --is_draw 0 \  
12  --is_dump 1 \  
13  --verbose 0 \  
14  --fixed_hw 128x128 \  

```

```
15 --tiles_rc 2x4 \  
16 --threads_intra 8 \  
17 --threads_inter 1 \  
18 --tile_omp 8
```

### В.3 Алгоритмический протокол измерений

Листинг В.3 фиксирует общую процедуру grid-search измерений. Она разделяет измерение задержки и выгрузку ROI, а также явно указывает, что сортировка конфигураций выполняется по *p99* при контроле качества покрытия по площади.

**Листинг В.3:** Алгоритмический протокол grid-search измерений

```
1 1. Build a full-image or conservative reference configuration.  
2  
3 2. For each candidate configuration:  
4   a. run idet_app with:  
5       --is_draw 0 --is_dump 0 --verbose 0;  
6   b. parse p50_ms, p90_ms, p95_ms, p99_ms  
7       from idet_app stdout;  
8   c. reject candidates above the latency threshold;  
9   d. run a separate dump pass with:  
10      --is_draw 0 --is_dump 1 --verbose 0;  
11   e. extract candidate Quads from stdout;  
12   f. compute AreaRecall, AreaPrecision, AreaF1  
13       and ExtraAreaRatio against reference boxes.  
14  
15 3. Sort accepted candidates by p99_ms.  
16  
17 4. Inspect the Pareto trade-off between p99 latency  
18 and area-based quality metrics.
```

## С.1 Чек-лист PT-to-ONNX артефакта

Таблица С.1 задает минимальный чек-лист проверки артефакта развертывания перед использованием в IDet. Он нужен для того, чтобы обучающая контрольная точка не подменяла собой модель, готовую к работе в среде выполнения.

**Таблица С.1:** Чек-лист проверки PT-to-ONNX артефакта перед включением в CPU-first контур IDet

| Проверка               | Требование                                                                             |
|------------------------|----------------------------------------------------------------------------------------|
| Формат                 | Модель экспортирована в ONNX и загружается через ONNX Runtime.                         |
| Форма входа            | Размер входа совпадает с предобработкой в IDet или явно поддерживает динамические оси. |
| Контракт выходов       | Формы выходов и порядок тензоров зафиксированы в отчете.                               |
| Class names            | Список классов совпадает с 7-классовой таксономией.                                    |
| Соответствие PT и ONNX | Проверены близость координат, уверенности и числа кандидатов на тестовых изображениях. |
| Домены операторов      | Переносимый ONNX не содержит целевых доменов, если используется как общий контракт.    |
| Варианты оптимизации   | Raw, slimmed, FP32, INT8 и ORT-optimized артефакты не смешиваются между собой.         |
| Метаданные             | Сохранены YAML-описание датасета, параметры экспорта, opset, хэш весов.                |
| Измерения              | Для каждого варианта развертывания измерены p50/p95/p99 и метрики покрытия по площади. |

## D.1 Визуальное сравнение single-shot и tiled-режимов

Рисунки D.1–D.3 показывают не просто визуальную разницу между single-shot и tiled-режимом, а характер ошибок, который затем должен учитываться в ROI-политике. Для текста важны мелкие надписи и риск дублей на границах плиток, для лиц — полнота в группе и стабильность bbox, для одежды — площадь защищаемых областей и недопущение чрезмерного расхода битового бюджета. Поэтому эти изображения дополняют численные benchmark-результаты и помогают интерпретировать, почему одна и та же p99-задержка может давать разное качество покрытия ROI.

Важно учитывать, что геометрия итоговых box-областей и визуальное качество детекции зависят не только от выбранного режима *single-shot* или *tiled*. На результат также влияют используемая модель, размер входа, сетка разбиения кадра, overlap между плитками, порог уверенности, параметры NMS, правила гетар координат и последующее слияние областей внутри IDet. Поэтому приведенные изображения следует трактовать как иллюстрацию поведения конкретных конфигураций детектора, а не как универсальное свойство всех tiled-режимов.



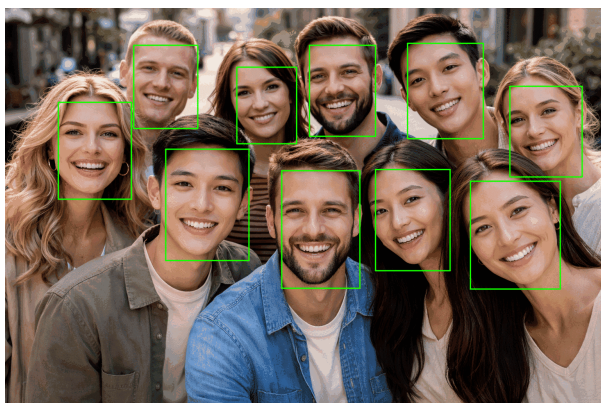
Рисунок D.1: Сравнение single-shot и tiled-режима для текстовых ROI.

Рисунок D.1 показывает типичный компромисс текстовой ветки: tiled-режим лучше сохраняет локальные надписи на разных масштабах, но добавляет границы плиток, вдоль которых возможны дубли и разрывы одной текстовой области. Для кодировщика это означает, что постобработка должна объединять близкие текстовые ROI и ограничивать лишнюю защищаемую площадь, иначе мелкие улучшения читаемости могут быть куплены избыточным числом областей.

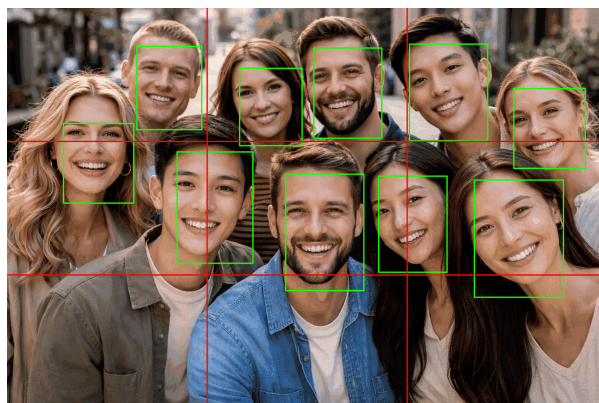
Рисунок D.2 показывает, что tiled-режим повышает шанс найти отдельные лица в плотной группе, особенно когда часть лиц занимает небольшую долю кадра. Побочный эффект — более дробная и неоднородная геометрия: соседние лица могут попадать в разные плитки, а границы bbox становятся менее согласованными. В видеопотоке такую геометрию нужно сглаживать во времени, иначе ROI-подсказки кодировщику будут мерцать между соседними кадрами.

Рисунок D.3 показывает, что для textured ROI tiled-режим меняет не только полноту, но и площадь будущей защиты в кодировщике. На объектах одежды это особенно важно:





(a) Single-shot



(b) Tiled

**Рисунок D.2:** Сравнение single-shot и tiled-режима для ROI лиц.



(a) Single-shot



(b) Tiled

**Рисунок D.3:** Сравнение single-shot и tiled-режима для текстурированных ROI.

слишком широкие или раздробленные области начинают конкурировать с text/face ROI за общий area budget. Поэтому tiled-инференс в IDet рассматривается как параметризуемый runtime-режим: его нельзя выбирать только по визуальному впечатлению. Окончательная конфигурация должна подтверждаться area recall, area precision, ExtraAreaRatio и p95/p99-задержкой.